



TRABAJO DE FIN DE MÁSTER
MÁSTER UNIVERSITARIO EN INGENIERÍA INFORMÁTICA

GeneRAG

Desarrollo y evaluación de un sistema basado en RAG de consultas
ciudadanas sobre ayudas gubernamentales

Autor

Pablo Valenzuela Álvarez

Tutor

David Griol Barres



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, septiembre de 2025



GeneRAG

Desarrollo y evaluación de un sistema basado en RAG de consultas
ciudadanas sobre ayudas gubernamentales

Autor

Pablo Valenzuela Álvarez

Tutor

David Griol Barres

GeneRAG: Desarrollo y evaluación de un sistema basado en RAG de consultas ciudadanas sobre ayudas gubernamentales

Palabras clave: Generación Aumentada por Recuperación, RAG, Grandes Modelos de Lenguaje, LLM, Modelos de Embedding, Inteligencia Artificial, IA, Ollama, OpenAI, Mistral, LangChain, Chroma, Streamlit, BOE, DANA

Resumen

En este Trabajo de Fin de Máster, hemos diseñado, desarrollado y evaluado un sistema de generación aumentada por recuperación o RAG. Aplicando diversas técnicas en la construcción, configuración y evaluación, hemos conseguido obtener un sistema capaz de responder preguntas con bastante precisión y de forma muy eficiente.

La base de un sistema RAG son los grandes modelos de lenguaje o LLM. Estos están capacitados para responder preguntas generalistas, pero no funcionan de igual modo con datos con los que no han sido entrenados o privados. Para solucionar esta problemática, un sistema RAG necesita acceder a una base de conocimiento externa para construir una respuesta específica usando estos nuevos datos. Estas bases de conocimiento suelen ser bases de datos vectoriales que guardan la relación semántica entre los documentos, y los recuperan de forma rápida y eficiente ante una consulta de usuario.

En este proyecto hemos implementado nuestro propio sistema RAG junto a una interfaz web en la que podemos realizar consultas. También se ha elaborado un amplio conjunto de pruebas de rendimiento de las diferentes alternativas con las que podemos configurar un sistema RAG (modelos de lenguaje, embedding o estrategias de búsqueda) con el objetivo de averiguar la mejor combinación de elementos. Para ello, se han definido cuatro agentes críticos encargados de evaluar las respuestas generadas por el RAG.

Durante la realización de este proyecto hemos sopesado diferentes opciones para realizar nuestro sistema RAG. Hemos explorado y descartado plataformas con las que solíamos trabajar dado a que se han quedado obsoletas ante el auge de los grandes modelos de lenguaje actualmente. Librerías como LangChain (diseñada para trabajar con modelos de lenguaje), o Chroma (base de datos vectorial); han facilitado el desarrollo del proyecto gracias a su flexibilidad y adaptabilidad a diferentes modelos de lenguaje: desde modelos locales de Ollama, externos y gratuitos de Mistral o de pago con OpenAI.

Por último, la base de conocimiento usada para el proyecto contiene documentos del Boletín Oficial Español (BOE) con referencias a lo sucedido en la DANA (Depresión Aislada en Niveles Altos) de finales del año 2024. Cuando el proyecto comenzó, este evento estaba muy reciente y pensamos que era una buena idea la de proporcionar una plataforma de ayuda a los ciudadanos con dudas o preguntas sobre la DANA. Nuestro sistema RAG está diseñado solo para este cometido, y a través de su interfaz web, responde con claridad y precisión cualquier duda siempre que tenga datos sólidos para ello.

GeneRAG: Project Subtitle

Keywords: Retrieval augmented generation, RAG, large language models, LLM, embedding models, artificial intelligence, AI, Ollama, OpenAI, Mistral, LangChain, Chroma, Streamlit

Abstract

In this Master's Thesis, we have designed, developed and evaluated a retrieval augmented generation (RAG) system. By applying various techniques in construction, configuration and evaluation, we have achieved a system capable of answering question with accuracy and efficiency.

The foundation of an RAG system lies in Large Language Models (LLMs). These models are trained to answer general questions but may not perform equally well with unfamiliar or private data. To address this issue, a RAG system needs access to an external knowledge base to construct specific responses using new data. These knowledge bases typically consist of vector databases that store semantic relationships between documents and retrieve them quickly and efficiently in response to user queries.

In this project, we implemented our own RAG system alongside a web interface for conducting queries. We also develop extensive performance testing of different configurations for RAG systems (language models, embeddings, search strategies) to determine the optimal combination of elements. To facilitate this, we defined four critical agents tasked with evaluating the responses generated by the RAG system.

Throughout the course of this project, we considered various options for implementing our RAG system. We explored and discarded platforms that had become obsolete in light of the current dominance of large language models. Libraries such as LangChain (designed for working with language models) and Chroma (a vector database) facilitated project development due to their flexibility and adaptability to different language models: from local models like Ollama's, to external free models like Mistral's, and paid models like those from OpenAI.

Finally, the knowledge base used for the project contains documents from the Official State Gazette (BOE) with references to the events of the late 2024 cut-off low (DANA) weather phenomenon. At the project's inception, this event was recent, prompting us to create a platform to assist citizens with questions about DANA. Our RAG system is designed specifically for this purpose, providing clear and precise answers through its web interface, provided it has solid data to draw upon.

Yo, **Pablo Valenzuela Álvarez**, alumno de la titulación Máster Universitario en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 76652136J, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Máster en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Pablo Valenzuela Álvarez

8 de septiembre de 2025

D. **David Griol Barres**, Profesor del Departamento de Lenguaje y Sistemas Informáticos de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***GeneRAG: Desarrollo y evaluación de un sistema basado en RAG de consultas ciudadanas sobre ayudas gubernamentales***, ha sido realizado bajo su supervisión por **Pablo Valenzuela Álvarez** , y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 8 de septiembre de 2025.

Los directores:

David Griol Barres

Agradecimientos

Quiero agradecer especialmente a David Griol Barres y Zoraida Callejas Carrión por su guía, disponibilidad y valiosos consejos a lo largo de este trabajo.

A mis compañeros y compañeras de máster, gracias por el apoyo constante, las discusiones enriquecedoras y la camaradería durante este proceso. También agradezco profundamente a mi familia, por su paciencia, ánimo y confianza en mí.

Finalmente, a todas las personas que, de una forma u otra, contribuyeron a la realización de este trabajo: gracias.

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	3
1.3	Estructura de la memoria	4
2	Estado del Arte	5
2.1	Generación Aumentada por Recuperación (RAG)	5
2.1.1	Diferencia con los LLMs clásicos	6
2.1.2	¿Como funciona un RAG?	7
2.1.3	Beneficios del RAG	9
2.1.4	Desafíos y limitaciones de RAG	9
2.1.5	Tendencias futuras de RAG	10
2.2	Evolución de los LLMs hasta nuestros días	10
2.2.1	LLMs usados en el proyecto	15
2.3	Modelos de Embedding	18
2.3.1	Nomic Embed (nomic-embed-text)	19
2.3.2	BGE-M3	20
2.3.3	Artic Embed 2.0 (snowflake-arctic-embed2	21
2.3.4	Jina Embeddings v2 Base ES (jina-embeddings-v2-base-es)	22
2.3.5	Text Embedding 3 (text-embedding-3-small)	23
2.4	Alternativa al uso de LLM: RASA Pro	24
2.4.1	Licencia de uso	24
2.4.2	Facilidad de uso	25
2.4.3	Rendimiento	26
2.4.4	Conclusión respecto a Rasa Pro	28
2.5	Ollama	29
2.5.1	Características principales de Ollama	30
2.5.2	Instalación y funcionamiento de Ollama	31
2.5.3	Alternativas: LM Studio	32
2.6	ChromaDB	35
2.6.1	Características de ChromaDB	35
2.6.2	Funcionamiento de ChromaDB	36
2.6.3	Alternativas: Pinecone	37
2.7	LangChain	41
2.7.1	Características e integraciones de LangChain	42
2.7.2	Tutorial básico de LangChain	42
2.7.3	Alternativas: LlamaIndex	43

2.8	Streamlit	45
2.8.1	Funcionalidades clave de Streamlit	46
2.8.2	Ejemplo de aplicación LLM en Streamlit	46
2.9	Aplicaciones alternativas al proyecto propuesto	49
2.9.1	Ajuda Dana	49
2.9.2	TuCocheDana	50
2.9.3	Justicio	51
2.9.4	Conclusión sobre las aplicaciones alternativas	52
2.10	Métodos de evaluación de nuestro RAG	53
2.10.1	Estrategias de búsqueda usadas	53
2.10.2	Embeddings usados	53
2.10.3	LLMs usados	54
2.10.4	Métricas de evaluación usadas	54
3	Desarrollo del Proyecto	55
3.1	Definición de la arquitectura	55
3.2	Planificación y presupuesto	56
3.2.1	Metodología seguida: Scrum	56
3.2.2	Fases del proyecto	58
3.2.3	Presupuesto	60
3.3	Construcción del corpus de documentos	62
3.3.1	Script de descarga de sumarios	64
3.3.2	Script para la descarga de documentos	64
3.3.3	Automatización de procesos de descarga	65
3.4	Desarrollo del chatbot	68
3.4.1	Clase Chatbot: constructor	68
3.4.2	Interfaz de carga y actualización de la base de datos	71
3.4.3	Interfaz del chatbot: Recuperación y generación de información	75
3.5	Conclusiones sobre el desarrollo de la aplicación	85
4	Evaluación del Proyecto	87
4.1	Generación del conjunto de preguntas	87
4.2	Configuración de las alternativas de evaluación	89
4.2.1	Modelos evaluadores y configuración inicial	89
4.2.2	Agentes críticos	91
4.2.3	Generación de evaluaciones	93
4.3	Casos de uso de la aplicación	97
4.4	Resultados de las evaluaciones del RAG	99
4.4.1	Resultados de Mistral Small 3.2	104
4.4.2	Resultados de Llama 3.2	113
4.4.3	Resultados de GPT-4.1	114
4.5	Conclusión sobre los resultados	115
5	Conclusiones y Líneas Futuras	119
5.1	Conclusiones	119
5.2	Líneas futuras	120

Índice de figuras

Figura 1	Pasarela provisional en la zona del barranco del Poyo en Picanya (Valencia), el 27 de noviembre [1]	1
Figura 2	Árbol de la evolución de los LLMs modernos [7]	3
Figura 3	Comparación entre los tres paradigma más utilizados de RAG [8] . .	6
Figura 4	Diferencia básica entre el mecanismo de respuesta de LLM y de RAG. Los LLM responden usando los datos con los que han sido entrenados, mientras que el RAG accede a una base de datos para responder con datos más específicos	7
Figura 5	Proceso de creación del Vector Store. Se observa todo el proceso: carga, troceado, embedding y almacenamiento [11]	8
Figura 6	Fases del proceso de funcionamiento de RAG, se observan las fases de recuperación de información y generación [4]	8
Figura 7	Gráfica de las cuatro etapas en la evolución de los modelos de lenguaje [12]	11
Figura 8	Fine-tuning en tareas de clasificación de flores usando el modelo pre-entrenado Restnet16 [6, 12]	12
Figura 9	Esquema de las direcciones de investigación donde podemos usar LLMs [6], [12]	14
Figura 10	Tabla comparativa del modelo Llama 3.2 en diferentes benchmarks [15]	16
Figura 11	Tabla comparativa del modelo Mistral Small 3.1 en diferentes benchmarks [17]	17
Figura 12	Rendimiento del conjunto de modelos GPT-4.1 [19]	18
Figura 13	Precio de los modelos GPT-4.1 [19]	18
Figura 14	Ejemplo básico de funcionamiento de un modelo de embeddings [20]. Recibe objetos de datos, que pueden ser de todo tipo, y los transforma en una representación vectorial	19
Figura 15	Rendimiento comparativo de Nomic Embed y otros modelos en distintos benchmarks [21]	20
Figura 16	Rendimiento comparativo del modelo bge-m3 con otros modelos [22]	21
Figura 17	Métricas obtenidas por los modelos de Artic Embed en comparación con otros, usando el benchmark MTEB-R en inglés. [23]	22
Figura 18	Comparación de varios modelos de embedding con JINA. [24]	23
Figura 19	Evaluaciones de los modelos de embedding de OpenAI [25]	23
Figura 20	Evaluaciones de los modelos de embedding de OpenAI con distintos tamaños [25]	23
Figura 21	Comparación de las dos estructuras	25

Figura 22	Contenido de los ficheros flows.yml y patterns.yml	25
Figura 23	Ejemplo de una respuesta con prompt integrado	26
Figura 24	El inspector nos muestra un grafo con el flujo de conversación	27
Figura 25	Maneras en las que puedes conversar en una aplicación hecha con Rasa Pro	28
Figura 26	Sitio web oficial de Ollama: https://ollama.com/	29
Figura 27	Fichero <i>modelfile</i> básico, edita la temperatura y la ventana del con- texto del modelo. Tiene de base el LLM <i>Llama3.2</i>	30
Figura 28	Lista de comandos disponibles de Ollama	31
Figura 29	Prueba del CLI de Ollama, ejecuta el modelo <i>Llama3.2</i> para hacer una consulta	31
Figura 30	Pequeño programa escrito en <i>Python</i> , usando la librería <i>LangChain</i> . Pide al modelo <i>Llama3.2</i> una consulta	32
Figura 31	Pantalla principal de LM Studio, la pestaña activa nos permite cha- tear con un LLM	32
Figura 32	Selección y carga de una versión de <i>Llama3.2</i> en LM Studio	32
Figura 33	Developer. Pestaña que permite controlar la API de LM Studio, se observa el modelo cargado, la dirección IP de esta y los logs	33
Figura 34	My Models. Pestaña que permite ver los LLMs descargados en LM Studio	33
Figura 35	Discover. Pestaña que permite descargar cualquier LLM disponible en LM Studio	34
Figura 36	Pequeño código en Python, para la prueba de la API de LM Studio	34
Figura 37	Esquema básico del funcionamiento de ChromaDB	35
Figura 38	ChromaDB, puesta en marcha mediante <i>Python</i>	36
Figura 39	ChromaDB. Resultados de la consulta, se observan las distancias de los documentos recuperados	37
Figura 40	Pinecone. Inicialización a través de su clave API	38
Figura 41	Pinecone. Creación de un índice	38
Figura 42	Pinecone. Vista del índice en la web	38
Figura 43	Pinecone. Agregación de documentos	39
Figura 44	Pinecone. Vista de documentos en la web	39
Figura 45	Pinecone. Búsqueda semántica y resultados	40
Figura 46	Pinecone. Reranking y resultados	40
Figura 47	Esquema que muestra las diferentes aplicaciones en las que podemos usar LangChain [38]	41
Figura 48	LangChain. Aplicación sencilla que usa mistral-large (LLM desarro- llado por MistralAI) para traducir un texto	43
Figura 49	LangChain. Resultado de la traducción del texto de la figura 48 . . .	43
Figura 50	Loaders, se encarga de cargar documentos [42]	44
Figura 51	Splitter, se encarga de trocear los documentos [42]	44
Figura 52	Indexing, se encarga de almacenar los documentos en la base de datos vectorial [42]	44
Figura 53	Chains/Query, se encarga de hacer una consulta a la base de datos vectorial [42]	45
Figura 54	Ejemplo aplicación LLM en Streamlit. Prompt con dos varitables . .	47
Figura 55	Ejemplo aplicación LLM en Streamlit. Llamada al LLM	47

Figura 56	Ejemplo aplicación LLM en Streamlit. Programa principal	48
Figura 57	Interfaz del ejemplo aplicación LLM en Streamlit. Bienvenida	48
Figura 58	Interfaz del ejemplo aplicación LLM en Streamlit. Opciones	49
Figura 59	Interfaz del ejemplo aplicación LLM en Streamlit. Respuesta con memoria	49
Figura 60	El voluntómetro muestra las solicitudes de ayuda y los voluntarios que hay en cada lugar [47]	50
Figura 61	Interfaz de Justicio en el BOE del día 3 de septiembre de 2025 [49] .	52
Figura 62	Arquitectura de nuestro sistema RAG	55
Figura 63	Jira. Plantilla Scrum para el proyecto	56
Figura 64	Jira. Conexión con la plataforma de control de versiones GitHub . .	57
Figura 65	Jira. Pestaña Backlog, tareas del sprint 9 y pendientes [56]	57
Figura 66	Jira. Tablero con tareas	58
Figura 67	Cronograma del proyecto. Extraído del proyecto en Jira [56]	59
Figura 68	BOE. API web desde la que descargar sumarios por día [57]	63
Figura 69	BOE. Resultados de la búsqueda de un día [57]	63
Figura 70	Script para descargar el sumario del día	64
Figura 71	Fragmento del script encargado de obtener documentos relacionados con la DANA	65
Figura 72	GitHub Actions. Ejemplo de funcionamiento de un flujo de trabajo [59]	65
Figura 73	GitHub Actions. Fragmento del flujo de trabajo que muestra el uso del comando cron	66
Figura 74	GitHub Actions. Fragmento del flujo de trabajo que muestran los trabajos que son ejecutados	66
Figura 75	Pestaña de acciones del proyecto. Se muestran una parte de las acciones automatizadas	67
Figura 76	Vista en detalle de una acción. Se observa el tiempo empleado en realizar cada paso	67
Figura 77	Clase Chatbot. Parámetros del constructor	68
Figura 78	Clase Chatbot. Configuración del LLM	69
Figura 79	Clase Chatbot. Configuración de varios parámetros: text_splitter, modelo de embedding y modelo de recuperación	70
Figura 80	Clase Chatbot. Configuración de la estrategia de búsqueda	70
Figura 81	Clase Chatbot. Configuración del prompt	71
Figura 82	Clase Chatbot. Configuración del reranker	71
Figura 83	Interfaz. Código de la página de carga y actualización de la base de datos	72
Figura 84	Interfaz. Diseño de la página de carga y actualización de la base de datos	73
Figura 85	Clase Chatbot. Fragmentos de código de algunos de los métodos usados para actualizar la base de datos	73
Figura 86	Interfaz. Cuadro de dialogo de tiempo de espera	74
Figura 87	Interfaz. Cuadros de dialogo. Tiempos para bases de datos con documentos de tamaño 512	74

Figura 88	Interfaz. Cuadros de diálogo. Tiempos para bases de datos con documentos de tamaño 1024	75
Figura 89	Interfaz. Tiempo de embedding de <i>text-embedding-3-small</i>	75
Figura 90	Interfaz. Página principal	76
Figura 91	Fragmentos de código. Declaración de la variable Chatbot en la interfaz principal y clase EMBEDDING	77
Figura 92	Fragmentos de código. Caja de selección de los modelos de lenguaje y clase LLM.	77
Figura 93	Función de la interfaz. Modificar el modelo de lenguaje	78
Figura 94	Clase Chatbot. Modificar el modelo de lenguaje	78
Figura 95	Fragmentos de código. Selección de radio para las estrategias de búsqueda y función de la interfaz para cambiarlas	78
Figura 96	Clase Chatbot. Modificar la estrategia de búsqueda	79
Figura 97	Fragmento de código correspondiente a la interfaz del chatbot	80
Figura 98	Función <i>query</i> de la interfaz del chatbot. Hace los procesos de recuperación y generación de información del sistema RAG	81
Figura 99	Método <i>answer_query</i> de la clase Chatbot, retorna la respuesta del sistema RAG	81
Figura 100	Prompt por defecto usado en el proyecto	82
Figura 101	Interfaz web del chatbot	82
Figura 102	Interfaz web con la barra lateral izquierda plegada	83
Figura 103	Interfaz web con pregunta	83
Figura 104	Interfaz web generando respuesta	84
Figura 105	Interfaz web con respuesta generada y referencias	84
Figura 106	Configuración LLM de generación del conjunto de preguntas	87
Figura 107	Prompt usado en la generación del conjunto de preguntas	88
Figura 108	Carga de datos y preparación del modelo de lenguaje	88
Figura 109	Bucle para la generación del conjunto de preguntas	88
Figura 110	Muestra del conjunto de preguntas [64]	89
Figura 111	Configuración de los modelos de lenguaje de pruebas	90
Figura 112	Variables de pruebas	90
Figura 113	Configuración de los chatbots de pruebas	91
Figura 114	Agente crítico del accuracy	91
Figura 115	Agente crítico del faithfulness	92
Figura 116	Agente crítico del groundedness	92
Figura 117	Agente crítico de la relevancia	93
Figura 118	Carga de dataset de preguntas	93
Figura 119	Columnas para el fichero de la evaluación y variables que contendrán los resultados de las evaluaciones	94
Figura 120	Función <i>get_response</i> y <i>get_response_rr</i> , ambas usadas para generar respuestas en las pruebas	94
Figura 121	Fragmento 1 del bucle de evaluación de preguntas: se guardan datos del dataset de preguntas, se obtienen la respuesta generada y el contexto, y se formatean los agentes críticos	94
Figura 122	Fragmento 2 del bucle de evaluación de preguntas. Se van generando una a una las evaluaciones de los agentes	95

Figura 123	Impresión de los resultados de los modelos en formato csv	95
Figura 124	Contenido de la carpeta de evaluaciones <i>GPT-4.1</i> en <i>text-embedding-3-small</i>	96
Figura 125	Muestra del fichero de evaluaciones correspondiente al modelo de lenguaje <i>GPT-4.1</i> y modelo de embedding <i>text-embedding-3-small</i> , con la estrategia de búsqueda TF-IDF	96
Figura 126	Modelos de lenguaje disponibles en la interfaz web	98
Figura 127	Ejemplos de cambio de estrategia de búsqueda desde la interfaz	98
Figura 128	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding BGE-M3 y tamaños de documento 512	106
Figura 129	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding JINA y tamaños de documento 512	106
Figura 130	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Nomic-Embed-Text y tamaños de documento 512	106
Figura 131	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Snowflakev2 y tamaños de documento 512	107
Figura 132	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding BGE-M3 y tamaños de documento 1024	109
Figura 133	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding JINA y tamaños de documento 1024	109
Figura 134	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Nomic-Embed-Text y tamaños de documento 1024	109
Figura 135	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Snowflakev2 y tamaños de documento 1024	110
Figura 136	Modificación hecha en el prompt	110
Figura 137	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding BGE-M3, tamaños de documento 1024 y el prompt modificado	112
Figura 138	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding JINA, tamaños de documento 1024 y el prompt modificado	112
Figura 139	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Nomic-Embed-Text, tamaños de documento 1024 y el prompt modificado	112
Figura 140	Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Snowflakev2, tamaños de documento 1024 y el prompt modificado	113
Figura 141	Gráfica de rendimiento del modelo Llama 3.2 usando el modelo de embedding Jina y tamaños de documento 1024	114
Figura 142	Gráfica de rendimiento del modelo GPT-4.1 usando el modelo de embedding text-embedding-3-small y tamaños de documento 1024	115

Índice de tablas

Tabla 3.1	Recursos hardware utilizados	61
Tabla 3.2	Presupuesto de los recursos hardware utilizados	61
Tabla 3.3	Recursos software utilizados	61
Tabla 3.4	Presupuesto de los recursos software utilizados	61
Tabla 3.5	Presupuesto de los recursos personales utilizados	62
Tabla 3.6	Coste total del proyecto	62
Tabla 4.1	Plantilla usada en los casos de uso	97
Tabla 4.2	Caso de uso 01, consulta de usuario	97
Tabla 4.3	Caso de uso 02, cambio de LLM	98
Tabla 4.4	Caso de uso 03, cambio de estrategia de búsqueda	98
Tabla 4.5	Caso de uso 04, creación de base vectorial	99
Tabla 4.6	Caso de uso 05, actualización desde carpeta	99
Tabla 4.7	Caso de uso 06, actualización desde fichero	99
Tabla 4.8	Comparativa de respuestas a pregunta 1	101
Tabla 4.9	Comparativa de respuestas a pregunta 2	102
Tabla 4.10	Comparativa de respuestas a pregunta 3	103
Tabla 4.11	Resultados de Mistral Small 3.2 en embeddings con 512 de tamaño de documentos	105
Tabla 4.12	Resultados de Mistral Small 3.2 en embeddings con 1024 de tamaño de documentos	108
Tabla 4.13	Resultados de Mistral Small 3.2 en embeddings 1024 de tamaño de documentos y el prompt modificado	111
Tabla 4.14	Resultados de Llama 3.2 en embeddings 1024 de tamaño de documentos	113
Tabla 4.15	Resultados de GPT-4.1 en embeddings 1024 de tamaño de documentos	115

Capítulo 1 : Introducción

El objetivo de este primer capítulo es describir el contexto en que está ubicado este Trabajo de Fin de Máster. Explicaremos los problemas que pretendemos resolver usando tecnologías actuales como los modelos de lenguaje a gran escala y los usos que les podemos dar para ayudar a la población. Para ello, marcaremos unos objetivos claros en los que apoyarnos. Por último, presentaremos la estructura que seguirá la memoria.

1.1. Motivación

Las catástrofes naturales que estamos viviendo estos últimos años, incendios a gran escala o episodios de lluvias torrenciales asociados a Depresiones Aisladas en Niveles Altos (DANA), representan uno de los mayores retos a los que se enfrenta la sociedad actual en el contexto del cambio climático. A finales de pasado año, la Comunidad Valenciana sufrió uno de estos episodios, el cual supuso un punto de inflexión tanto por la intensidad de las precipitaciones como por el impacto social, económico y ambiental que generó [1]. Calles anegadas, infraestructuras interrumpidas (ver ejemplo en la figura 1), miles de vecinos afectados y un gran volumen de información dispersa en medios de comunicación, redes sociales u organismos oficiales conformaron un panorama en el la correcta gestión y acceso al conocimiento veraz resultaron esenciales [2].



Figura 1: Pasarela provisional en la zona del barranco del Poyo en Picanya (Valencia), el 27 de noviembre [1]

En este contexto, surge la motivación de este Trabajo de Fin de Máster: **desarrollar un sistema RAG** (Retrieval-Augmented-Generation) ([3, 4]) que permita organizar, recuperar y generar información de valor sobre la DANA de Valencia. La elección de este enfoque no es casual, pues las técnicas usadas en un RAG combinan la capacidad de búsqueda eficiente en grandes repositorios de documentos con la generación en lenguaje natural contextualizado y coherente.

La relevancia social de este trabajo se refuerza por el hecho de que la información utilizada

procede de fuentes oficiales, en particular del Boletín Oficial de Estado (BOE) [5]. Aunque el BOE constituye la referencia legal sobre ayudas, disposiciones administrativas y medidas de emergencia, su consulta resulta compleja para la mayoría de ciudadanos, más aún en momentos de vulnerabilidad. Una persona afectada por la DANA que necesite conocer qué ayudas solicitar, qué requisitos cumplir o qué plazos respetar se enfrenta a textos jurídicos densos y de difícil navegación. Un sistema RAG aplicado sobre estos documentos permite transformar ese lenguaje administrativo en respuestas claras, comprensibles y accesibles.

Para hacer efectiva esta idea, el trabajo incluye el **desarrollo de una interfaz de usuario interactiva**, que permite a cualquier persona formular preguntas en lenguaje natural y recibir respuestas de forma precisa y siempre fundamentadas en la documentación oficial. Esta interfaz no solo es un complemento, sino un componente clave del proyecto, ya que convierte un sistema técnico (como lo es el RAG) en una herramienta práctica con la que cualquier ciudadano, periodista o institución puede interactuar de manera directa. En definitiva, la interfaz actúa como puente entre la complejidad de la inteligencia artificial y la necesidad de simplicidad en la experiencia del usuario final.

Más allá del desarrollo de una aplicación RAG completa, este TFM también persigue un objetivo de carácter académico y técnico: **evaluar el sistema RAG** de manera rigurosa mediante métricas específicas. El ámbito de los sistemas RAG es muy dinámico y ofrece múltiples alternativas en cuanto a modelos y algoritmos de recuperación, embeddings y grandes modelos de lenguaje (LLMs). Analizar y comparar diferentes configuraciones resulta fundamental para determinar cuál ofrece el mejor equilibrio entre precisión, coherencia y utilidad práctica. Este segundo objetivo garantiza no solo la validez técnica del trabajo, sino también su capacidad para generar conocimiento transferible a aplicaciones futuras.

El desarrollo de este proyecto se apoya en la evolución reciente de los modelos de lenguaje ([6, 7]), que han alcanzado un nivel de madurez que los convierte en herramientas aptas para aplicaciones de gran impacto social. Muestra de ello es la figura 2, donde vemos la gran cantidad de modelos que han ido surgiendo hasta 2023. Los modelos más modernos no solo son capaces de procesar y contextualizar información de manera avanzada, sino que también pueden interactuar con el usuario de manera fluida y natural. Por lo tanto, la integración de estas capacidades en un sistema RAG con interfaz hace posible el diseño de asistentes inteligentes que realmente ayudan en situaciones críticas, consiguiendo a la vez que el ciudadano este informado con fuentes fiables.

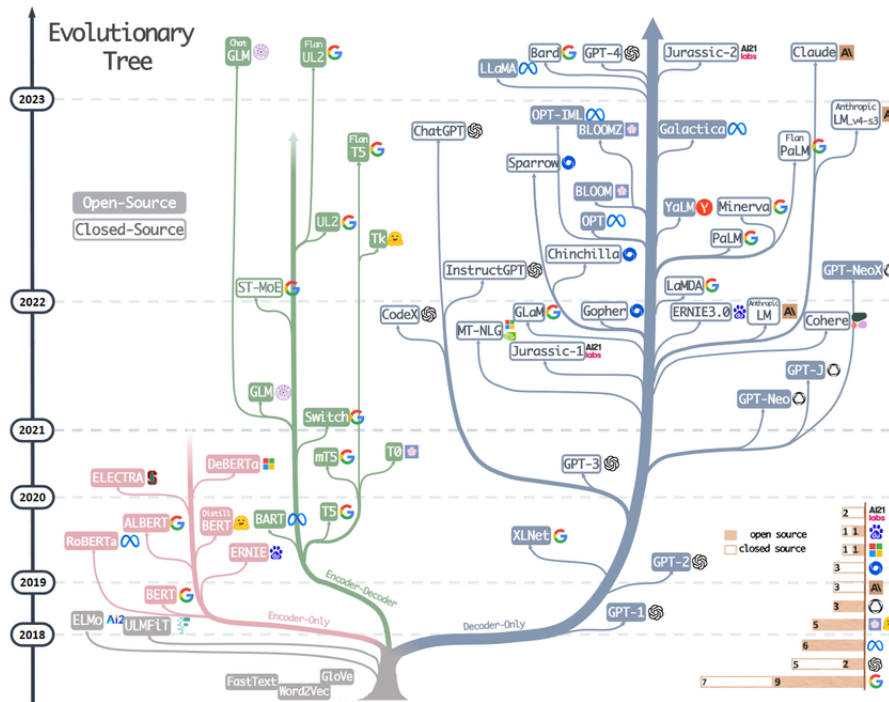


Figura 2: Árbol de la evolución de los LLMs modernos [7]

En conjunto, la motivación de este Trabajo de Fin de Máster combina tres vertientes complementarias:

- **Social**, orientada a facilitar el acceso a información oficial a ciudadanos en momentos de crisis ambiental o climática.
- **Práctica**, a través de una interfaz accesible que permite a cualquier usuario interactuar con el sistema RAG sin necesidad de conocimientos previos.
- **Técnica**, centrada en la evaluación comparativa de las diferentes configuraciones que admite un sistema RAG, buscando la máxima eficiencia.

De este modo, aunque el caso de estudio se centra en la DANA de Valencia de 2024, la metodología y la aplicación desarrollada pueden extrapolarse a otros contextos de crisis. La motivación última es contribuir, desde la investigación universitaria, a un futuro más informado y preparado, en el que la inteligencia artificial actúe como herramienta efectiva para mejorar la vida de la ciudadanía.

1.2. Objetivos

En esta sección comentaremos los dos objetivos definidos para este proyecto: la construcción de un RAG y su evaluación. Describiremos cada uno, desde su justificación hasta su futura implementación.

El primer objetivo de este trabajo es el desarrollo de un sistema RAG completo, es decir, un sistema que se apoye en información recuperada de una base de conocimiento para generar respuestas más ricas con la ayuda de un modelo de lenguaje. El cometido de nuestro sistema RAG, es el de ayudar a la población a obtener información de cualquier

índole (medidas aprobadas, subvenciones, plazos, etc) que esté relacionada con el fenómeno meteorológico DANA, sucedido a finales de 2024.

Para este fin, hemos usado documentos del Boletín Oficial de Estado (BOE) relacionados con la DANA para generar nuestra fuente de conocimiento. A partir de aquí, se desarrollará una aplicación con interfaz capaz de recuperar información relevante a la consulta de usuario y de generar una respuesta completa usando esa información. El objetivo es conseguir que nuestro sistema responda con precisión a la duda planteada por la persona en cuestión.

El segundo objetivo del trabajo, es evaluar el rendimiento de nuestro sistema RAG. Dada la gran cantidad de parámetros ajustables tanto en el modelo de recuperación de información como de elección de modelos de lenguaje para la generación de respuesta, es interesante observar como de distintas son las respuestas ante estos cambios.

En consecuencia, para la evaluación debemos crear un conjunto de preguntas y respuestas sobre los que aplicar después distintas métricas. Dichas métricas, no son más que *prompts* adaptados que evalúan la precisión, uso de contexto o relevancia de este, en la respuesta generada. Los resultados obtenidos, probando distintas configuraciones en el modelo de recuperación y diferentes modelos de lenguaje, nos indicaran las combinaciones recomendables, o las a evitar.

1.3. Estructura de la memoria

La estructura de la memoria consta de cinco capítulos finalizando con la bibliografía. El contenido de cada capítulo se describe en la siguiente lista:

- **Capítulo 1. Introducción:** Presenta el proyecto de forma general, explicando la motivación que ha llevado a su realización, los objetivos principales que se persiguen y la organización de la memoria para facilitar su lectura.
- **Capítulo 2. Estado del Arte:** Revisión de los conceptos teóricos relevantes y descripción de las herramientas, librerías y plataformas empleadas en el desarrollo. Se incluyen además alternativas consideradas, justificando la elección final.
- **Capítulo 3. Desarrollo del Proyecto:** Exposición del proceso seguido para la construcción de la aplicación RAG, detallando la planificación inicial, la metodología aplicada y las fases de implementación.
- **Capítulo 4. Evaluación del Proyecto:** Análisis del funcionamiento de la aplicación a partir de métricas y un conjunto de preguntas diseñadas para tal fin. Los resultados se presentan en tablas y gráficas que facilitan su interpretación.
- **Capítulo 5. Conclusiones y Líneas Futuras:** Recoge las conclusiones generales derivadas del trabajo, así como posibles mejoras y ampliaciones que podrían explorarse en desarrollos posteriores.
- **Bibliografía:** Relación de las fuentes y referencias consultadas a lo largo de la elaboración del proyecto.

Capítulo 2 : Estado del Arte

En este capítulo explicamos todos los conceptos aparecidos en el proyecto. Mostramos las aplicaciones, tecnologías, herramientas y librerías usadas durante el proceso de desarrollo del mismo, explicando en cada sección las razones y ventajas que tienen sobre otras.

2.1. Generación Aumentada por Recuperación (RAG)

La generación aumentada por recuperación o **RAG** [3, 4] por sus siglas en inglés (Retrieval-Augmented Generation), es el concepto principal sobre el que se asienta nuestro proyecto final de máster. Esta técnica complementa la generación de texto que proporcionan los grandes modelos de lenguaje o **LLMs** (Large Language Models) con información proveniente de datos privados a través de un modelo de recuperación de información, diseñado para buscar en grandes conjuntos de datos o bases de conocimiento. Las respuestas obtenidas de este modo resultan más legibles y más coherentes para el usuario.

La tecnología RAG ha evolucionado rápidamente en los últimos años, siguiendo distintas etapas ligadas al desarrollo de los grandes modelos de lenguaje (LLM). En sus inicios, surgió junto con la arquitectura *Transformer* y se centró en mejorar los modelos pre-entrenados mediante la incorporación de conocimiento externo. Posteriormente, la aparición de *ChatGPT* marcó un punto de inflexión, al mostrar el potencial del in-context learning y orientar la investigación hacia cómo ofrecer mejor información a los LLM para resolver tareas más complejas y dependientes de conocimiento [8].

Hoy día, la investigación sobre el paradigma RAG continúa evolucionando, tanto que puede ser categorizada en tres etapas [8] (ver figura 3):

1. **Naive RAG:** enfoque inicial basado en un esquema tradicional de indexación, recuperación y generación. Es útil, pero presenta problemas de precisión en la recuperación, *alucinaciones* en la generación o redundancia e incoherencia en la integración de información.
2. **Advanced RAG:** surge para intentar superar las limitaciones de la etapa anterior, mejorando tanto indexación (segmentación fina, uso de metadatos, ...) como la post-recuperación (*reranking*). Así reducimos la sobrecarga de información que la damos al LLM y aumentamos la relevancia de esta.
3. **Modular RAG:** representa la etapa más flexible y avanzada. Introduce nuevos módulos y patrones. **Nota:** esta modalidad no ha investigado en este trabajo.

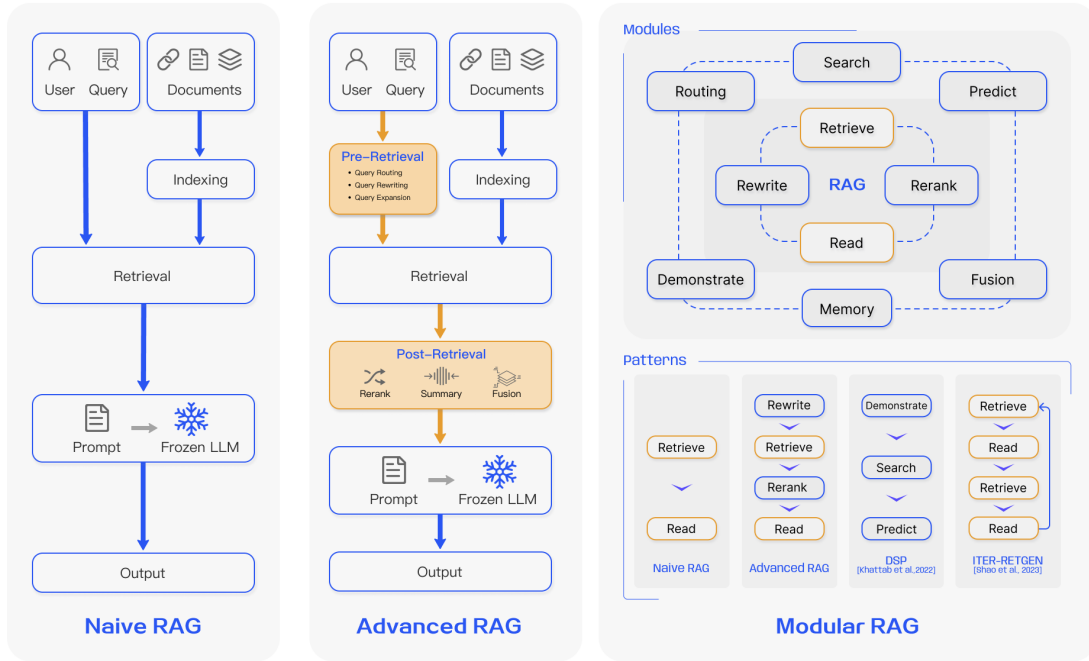


Figura 3: Comparación entre los tres paradigma más utilizados de RAG [8]

En las siguientes secciones de este apartado iremos desggranando poco a poco más información sobre los conceptos clave LLM-RAG, centrándonos especialmente en el objeto protagonista de este proyecto, el RAG. Mostraremos en que mejora a los LLMs clásicos, explicaremos su funcionamiento y observaremos las ventajas y limitaciones que posee.

2.1.1. Diferencia con los LLMs clásicos

Los LLMs tradicionales, como GPT-3 o GPT-4, son modelos pre-entrenados o PLMs (sección 2.2) con grandes conjuntos de datos que generan texto de acuerdo a la información contenida en esos datos [9]. Por lo tanto, su conocimiento esta limitado a la fecha en que fue entrenado, y puede generar respuestas genéricas, no precisas o directamente desfasadas. A este fenómeno se le conoce como *LLM hallucinations* o alucinaciones [10].

Optimizar estos modelos para que contengan la información más actualizada y reciente requiere de realizar re-entrenos constantemente, lo que consume muchos recursos y tiempo, por lo que no es una solución adecuada.

Los sistemas RAG usan datos recuperados en tiempo real de bases de datos externas en el momento de generar su respuesta al usuario. De esta forma el RAG esta provisto de datos recientes y específicos para el contexto del usuario, sin la necesidad de re-entrenar ningún modelo. La figura 4 muestra de forma básica este comportamiento.

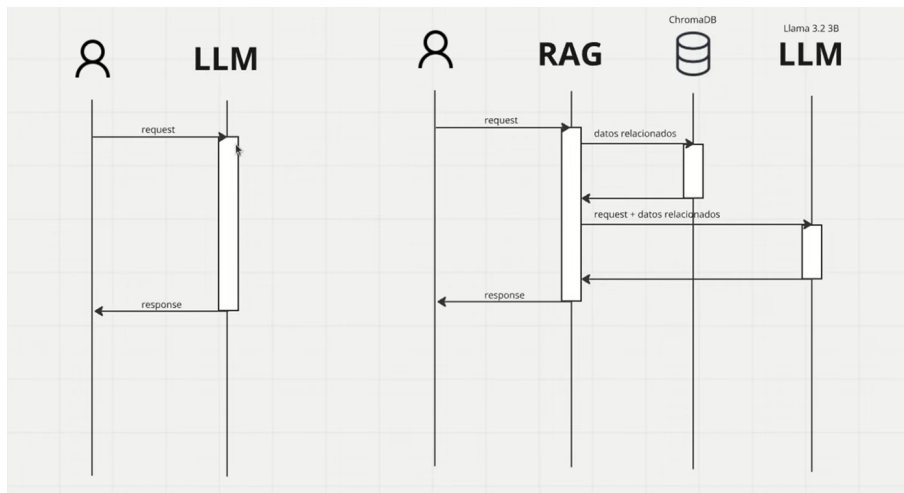


Figura 4: Diferencia básica entre el mecanismo de respuesta de LLM y de RAG. Los LLM responden usando los datos con los que han sido entrenados, mientras que el RAG accede a una base de datos para responder con datos más específicos

Esta diferencia ayuda al modelo de lenguaje usado por el RAG a adaptar su respuesta al contexto propio del usuario, aportando información relevante y actualizada a la que los LLMs de uso general no tienen acceso.

2.1.2. ¿Como funciona un RAG?

Un sistema RAG se puede comprender como un guión en el que hay que seguir unos pasos para conseguir una respuesta contextualizada y coherente para el usuario [3, 4, 8]. Estos pasos se pueden englobar en tres etapas claramente diferenciadas en el proceso de funcionamiento de un RAG, y son las siguientes:

Creación del Vector Store

El Vector Store es la base de datos que almacenará todo el conocimiento necesario para la lógica de negocio del sistema RAG. Los pasos dentro de esta etapa se pueden resumir en tres: conseguir un conjunto de datos, trocear o separar los datos y almacenar los datos usando modelos de embedding.

El conjunto de datos para el RAG es esencial, puesto que en él se reúnen los datos específicos y contextuales que necesitamos y que no poseen los LLMs genéricos. En la mayoría de casos de uso, la información requerida esta contenida en manuales, bases de datos de productos o listas de preguntas frecuentes [3].

El troceado de datos es el proceso en el cual los datos son transformados en piezas más pequeñas y a su vez más manejables, por ejemplo, dividir un documento de 100 páginas en partes más pequeñas puede ayudar a responder más preguntas a los usuarios. Esto también incrementa la eficiencia del RAG, puesto que el sistema puede obtener rápidamente los trozos de información más relevantes en vez procesar un documento entero.

Por último, una vez troceados los datos, necesitamos almacenarlos en nuestra base de datos convertidos en representaciones vectoriales. Esto requiere del uso de los modelos

de embedding (sección 2.3), que son modelos de lenguaje que transforman el texto de los datos en representaciones numéricas, capturando en ellas las relaciones semánticas del texto durante el proceso (ver figura 5).

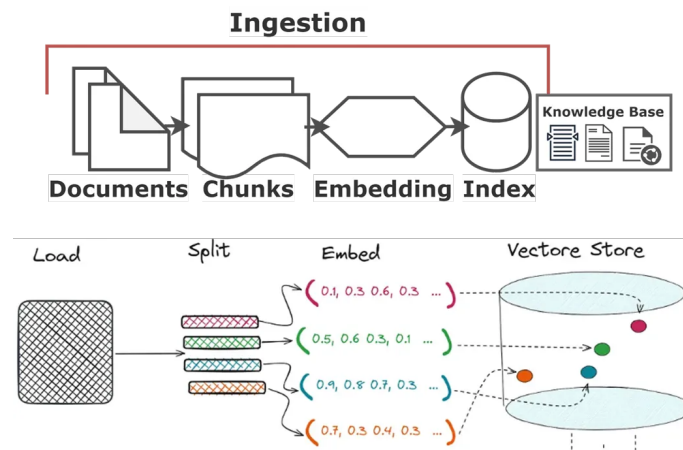


Figura 5: Proceso de creación del Vector Store. Se observa todo el proceso: carga, troceado, embedding y almacenamiento [11]

Recuperación de información

La recuperación de información es la etapa donde entra una consulta al sistema RAG, esta debe ser también vectorizada usando los mismos modelos de embedding que los usados al crear la Vector Store, para garantizar la uniformidad en la respuesta.

Una vez vectorizada la consulta el sistema compara ambos embeddings, identifica y retorna las piezas de documentos más parecidas usando medidas de comparación como la similitud coseno o distancia euclídea entre otras [3]. La figura 6 muestra parte de este proceso.

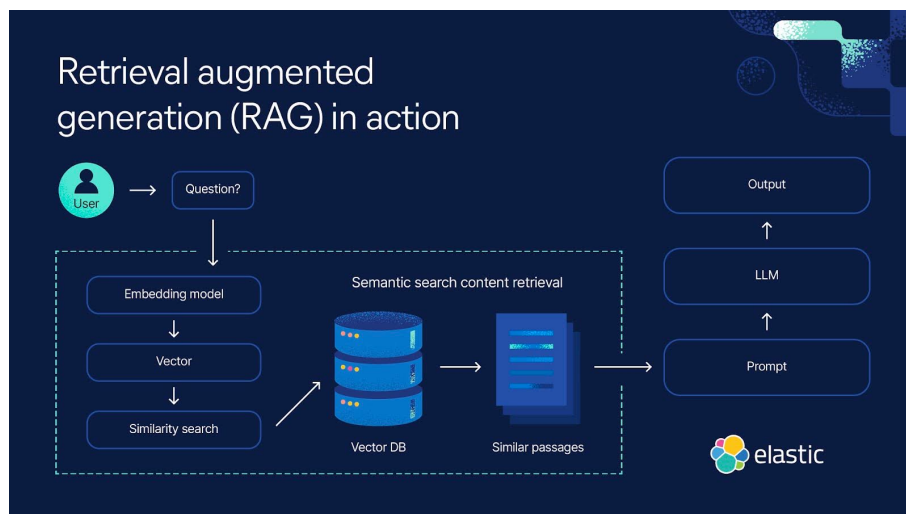


Figura 6: Fases del proceso de funcionamiento de RAG, se observan las fases de recuperación de información y generación [4]

Generación

La generación es la etapa que siempre sigue a la recuperación de información. Los documentos recuperados de la consulta inicial son aportados al LLM a través de un prompt [4], que ayuda a generar mediante una serie de instrucciones una respuesta coherente. En la figura 6 se puede ver parte de este proceso.

Las respuestas obtenidas de esta forma son, en general, más precisas y relevantes dado el contexto, puesto que han sido generadas a partir de los documentos recuperados más relevantes de la base de conocimiento que posee el RAG.

2.1.3. Beneficios del RAG

Un sistema RAG ofrece múltiples ventajas frente a los modelos de lenguaje que operan de manera aislada. Uno de los principales beneficios es que permite al modelo acceder a información actualizada mediante la integración de fuentes externas, lo que garantiza que las respuestas generadas sean recientes y relevantes para el usuario [4]. Esta capacidad de actualización continua resulta clave en entornos donde el conocimiento cambian o evolucionan rápidamente.

Otro aspecto destacable es la eficiencia en términos de recursos. Los sistemas basados en RAG suelen requerir menos capacidad de cómputo y almacenamiento, lo que los convierte en una opción más rentable para muchas aplicaciones. Además, al basarse en documentos recuperados durante la consulta, un RAG puede citar sus fuentes y proporcionarlas al usuario, lo que aumenta la transparencia y permite verificar la precisión de las respuestas ofrecidas.

En cuanto a la personalización, aunque los modelos de lenguaje ya permiten generar respuestas adaptadas, los RAG va un paso más allá al incorporar mecanismos de recuperación semántica. Esto le permite referenciar contenidos relevantes a partir del contexto específico de la consulta, mejorando la calidad y precisión de las respuestas. Asimismo, frente a preguntas complejas o ambiguas, donde un modelo tradicional podría generar respuestas erróneas o alucinaciones [10], un RAG se apoya en evidencia externa, lo que reduce significativamente este riesgo.

Por último, los modelos RAG son altamente versátiles y pueden aplicarse a una amplia gama de tareas de procesamiento de lenguaje natural, como sistemas conversacionales, generación de contenido o recuperación de información, consolidándose así como una herramienta poderosa en el desarrollo de soluciones basadas en inteligencia artificial [4].

2.1.4. Desafíos y limitaciones de RAG

Si bien los sistemas RAG ofrecen ventajas significativas frente a los modelos de lenguaje que operan de forma aislada, también presenta diversos desafíos y limitaciones que deben ser considerados. Uno de los más relevantes es la posibilidad de recuperar información incorrecta o no confiable. Dado que un RAG se basa en fuentes externas para elaborar sus respuestas, una recuperación inadecuada puede derivar en resultados imprecisos o incluso engañosos, afectando la calidad general del sistema.

Otro aspecto crítico es el tiempo de búsqueda [4]. Al requerir consultas en grandes bases de

conocimiento o incluso en la web, el proceso de recuperación puede introducir una latencia considerable, lo que impacta en la experiencia del usuario y en la eficiencia del sistema. Esta limitación exige estrategias de optimización tanto en la indexación como en el diseño de los mecanismos de búsqueda.

Además, integrar recuperación y generación en una misma aplicación supone retos importantes en cuanto a diseño e implementación. Es necesario desarrollar soluciones cuidadosas, lo cual puede dificultar la capacitación de los modelos y aumentar la complejidad técnica.

La privacidad representa otro desafío relevante [4], especialmente cuando un RAG opera con datos sensibles o confidenciales. En estos casos, es fundamental garantizar el cumplimiento de normativas y políticas de protección de datos, lo que a su vez puede restringir el conjunto de fuentes disponibles.

Finalmente, cabe destacar que, por su naturaleza centrada en la recuperación de hechos verificables, un RAG puede mostrar limitaciones a la hora de generar contenido imaginativo o creativo [4]. Aunque esto no representa un inconveniente para tareas centradas en la precisión, sí puede reducir su efectividad en aplicaciones donde se espera una mayor capacidad de invención.

2.1.5. Tendencias futuras de RAG

En cuanto al futuro de la generación aumentada por recuperación, las tendencias indican que las tecnologías empleadas para su desarrollo serán cada vez más eficientes y adaptables a una amplia variedad de aplicaciones.

Los modelos RAG seguirán avanzando hacia una mayor personalización [4], integrando conocimientos específicos del usuario para ofrecer respuestas más precisas y relevantes. Esto será especialmente útil en sistemas de recomendación de contenido y asistentes virtuales, donde la adaptación al perfil del usuario es clave. Además de la personalización, se espera que los usuarios adquieran un mayor control sobre el comportamiento de los modelos, con opciones para ajustar estrategias de búsqueda o modificar la estructura y segmentación del contenido de las bases de datos utilizadas durante la recuperación de información.

Otra tendencia relevante es el crecimiento en la escalabilidad de los modelos RAG [4], lo que les permitirá manejar un volumen cada vez mayor de datos e interacciones sin comprometer el rendimiento. Asimismo, la integración de RAG con otras técnicas de inteligencia artificial —como el aprendizaje por refuerzo— promete sistemas más versátiles y sensibles al contexto, capaces de gestionar distintos tipos de datos y tareas de forma simultánea.

Finalmente, a medida que los modelos RAG mejoren en velocidad de recuperación y generación de respuestas [4], será más común su implementación en aplicaciones que demandan baja latencia y tiempos de respuesta en tiempo real, ampliando así su presencia en cualquier entorno.

2.2. Evolución de los LLMs hasta nuestros días

La forma de extraer información de fuentes de datos textuales ha cambiado ostensiblemente en la última década. Como término, el procesamiento del lenguaje natural ha

sobrepasado a la minería de datos como aspecto más importante. Uno de los principales motivos de este cambio fue la aparición de los modelos de lenguaje, que son utilizados como base para multitud de aplicaciones dedicadas a extraer información valiosa de texto en crudo.

Los modelos de lenguaje usan técnicas de aprendizaje automático para predecir palabras mediante probabilidades. Estos modelos aprenden a través de conjuntos de datos textuales, y suelen ser usados para producir textos originales, el reconocimiento de discursos, el reconocimiento óptico de caracteres o el reconocimiento de escritura a mano [6, 12].

La evolución de los modelos de lenguaje ha pasado por cuatro etapas diferenciadas como se pueden observar en la figura 7. A continuación, describiremos brevemente estas:

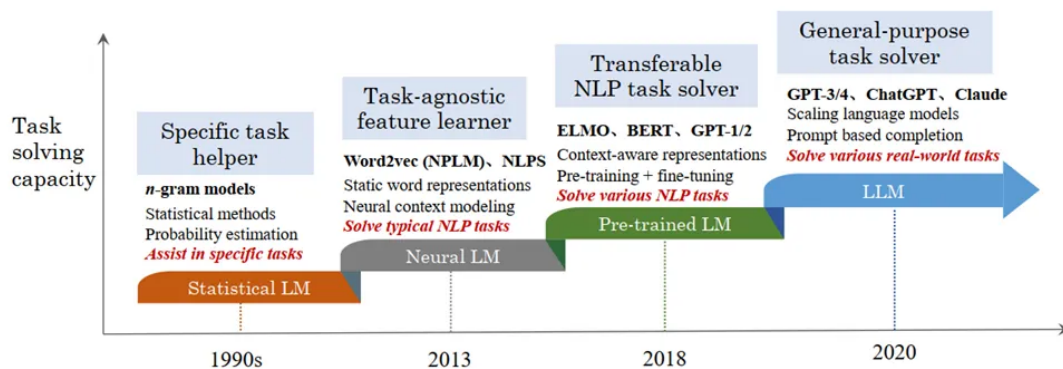


Figura 7: Gráfica de las cuatro etapas en la evolución de los modelos de lenguaje [12]

Modelos de lenguaje estadísticos (SLMs)

Los modelos de lenguaje estadísticos o SLMs surgieron a partir de el año 1990, e intentaban adivinar la siguiente palabra basándose en las palabras que la precedían. Estos modelos son llamados n-gramas [6]: por ejemplo, los bigramas y trigramas predicen según las dos y las tres palabras anteriores respectivamente.

Mientras los SLMs eran buenos para tareas como la búsqueda de información y la comprensión del lenguaje, tenían un gran inconveniente: la predicción basada en muchas palabras previas, los llamados modelos de gran orden, se complicaba en demasía. La razón principal era la necesidad de estimar un valor exponencial de probabilidades de transición, que en resumen, son valores numéricos comprendidos entre 0 y 1 que representan la probabilidad de que una palabra específica (B) aparezca tras una palabra (A) en una secuencia.

Modelos de lenguaje neuronales (NLMs)

Los SLMs eran buenos al predecir el orden de las palabras, pero fallaban a la hora de entender el lenguaje en profundidad. Los nuevos modelos llamados modelos de lenguaje neuronales o NLMs [6] usaban herramientas potentes como redes neuronales, por ejemplo el MLPs (perceptron multicapa) y RNNs (redes neuronales recursivas), que capturaban las relaciones entre palabras. Los modelos como *word2vec* [12] creaban representaciones

vectoriales (también denominado modelo de *embeddings*) para cada palabra, ayudando así a la comprensión de la red de la idea general.

Este pequeño avance hacia la comprensión del significado de las palabras, no sólo el orden, ha revolucionado como los sistemas procesan el lenguaje, permitiendo a su vez ampliar el rango de tareas en el que son más efectivos, como la generación de texto o la traducción automática de idiomas. Más información sobre modelos de embedding en la sección 2.3 de esta memoria.

Modelos de lenguaje pre-entrenados (PLMs)

Los modelos de lenguaje pre-entrenados o PLMs [6], son modelos que han sido entrenados con un conjunto de datos a través de técnicas de aprendizaje no supervisado. Durante la etapa de pre-entrenamiento, el modelo aprende a comprender y generar texto natural mediante el procesamiento de grandes cantidades de texto; como por ejemplo: libros, artículos, páginas web, o cualquier contenido de texto disponible en la red. La figura 8 muestra un ejemplo de uso para tareas de clasificación de imágenes.

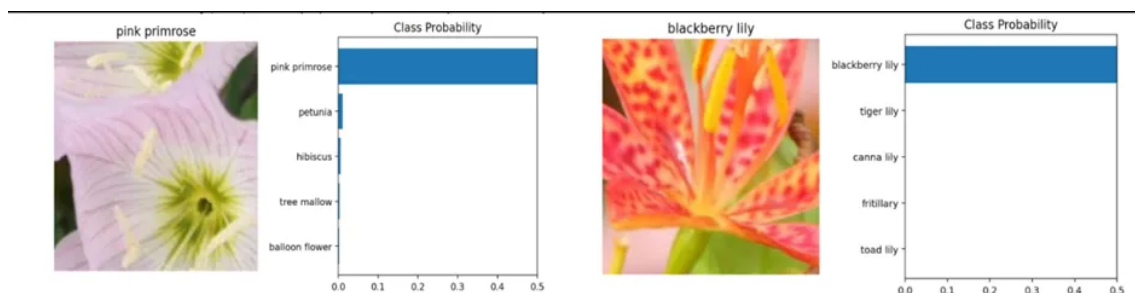


Figura 8: Fine-tuning en tareas de clasificación de flores usando el modelo pre-entrenado Restnet16 [6, 12]

Los modelos pioneros como *ELMo* [12] (Embeddings from Language Models) y *BERT* [12] (Bidirectional Encoder Representations from Transformers) revolucionaron la forma en la que se procesaba el lenguaje natural (NLP) debido a la introducción de palabras representativas del contexto. Estos modelos agregaban al pre-entrenamiento, arquitecturas bidireccionales como *Transformers*.

Este pre-entrenamiento permitía a los modelos aprender las representaciones numéricas de palabras contextuales, gracias a los *embeddings* (sección 2.3) y por consiguiente mejorar significativamente el rendimiento del NLP realizando pequeños ajustes (el llamado *fine-tuning*). Modelos como *GPT-2* [12] (Generative Pre-trained Transformer 2) y *BART* [12] (Bidirectional and Auto-Regressive Transformers) surgieron en consecuencia y abrieron el camino a lo que hoy se conoce como los grandes modelos de lenguaje.

Grandes modelos de lenguaje (LLMs)

Los grandes modelos de lenguaje son el resultado de la ampliación progresiva de los PLMs. Este crecimiento en tamaño, acompañado del uso de conjuntos de entrenamiento cada vez más extensos y diversos, ha demostrado mejorar significativamente el rendimiento en múltiples tareas. Modelos recientes como *GPT-4* han evidenciado comportamientos

distintos respecto a sus predecesores más pequeños, mostrando una mayor capacidad para resolver tareas complejas sin necesidad de entrenamiento adicional.

Actualmente, los LLMs están generando un impacto considerable en la comunidad de inteligencia artificial [6]. Su desarrollo ha ampliado las expectativas sobre el potencial de estos modelos, hasta el punto de fomentar discusiones sobre una inteligencia artificial con capacidades que podrían igualar o incluso superar el razonamiento humano. Como ejemplo ilustrativo, el modelo *GPT-4.5* ha sido recientemente citado por superar el test de Turing con un 73 % de éxito [13]. Estas capacidades abren la puerta a aplicaciones que van desde el acceso avanzado al conocimiento y la automatización de tareas complejas, hasta la aceleración de descubrimientos científicos o el impulso económico.

Una de las características más destacadas de los LLMs es la aparición de capacidades emergentes [6], es decir, habilidades que no estaban presentes en modelos más pequeños y que surgen como resultado del aumento de escala y del refinamiento en el entrenamiento.

Capacidades emergentes

El *in-context learning* (aprendizaje en contexto) se refiere a la capacidad del modelo para resolver tareas a partir de ejemplos o instrucciones proporcionadas en la propia entrada del usuario (el *prompt*), sin necesidad de re-entrenar el modelo. Esta habilidad permite adaptar la respuesta al contexto ofrecido. Por ejemplo:

Contexto: “Eres un asistente virtual especializado en resolver problemas matemáticos.”

Instrucción: “Calcula la raíz cuadrada de 25.”

De forma similar, el seguimiento de instrucciones es posible gracias a que muchos LLMs han sido ajustados utilizando conjuntos de datos formateados como instrucciones en lenguaje natural. Esto permite al modelo generalizar mejor a tareas nuevas, siempre que estén formuladas de manera instructiva.

Por último, el razonamiento paso a paso hace referencia a la capacidad del modelo para descomponer un problema complejo en pasos más simples, facilitando así su resolución. Al encadenar instrucciones dentro de un *prompt*, se puede guiar al modelo a pensar de forma más estructurada, lo que mejora tanto la precisión como la interpretabilidad de las respuestas generadas.

Aplicaciones de un LLM

En resumen, los grandes modelos de lenguaje tienen aplicaciones muy variadas que abarcan tanto direcciones de investigación como dominios específicos (ver figura 9).

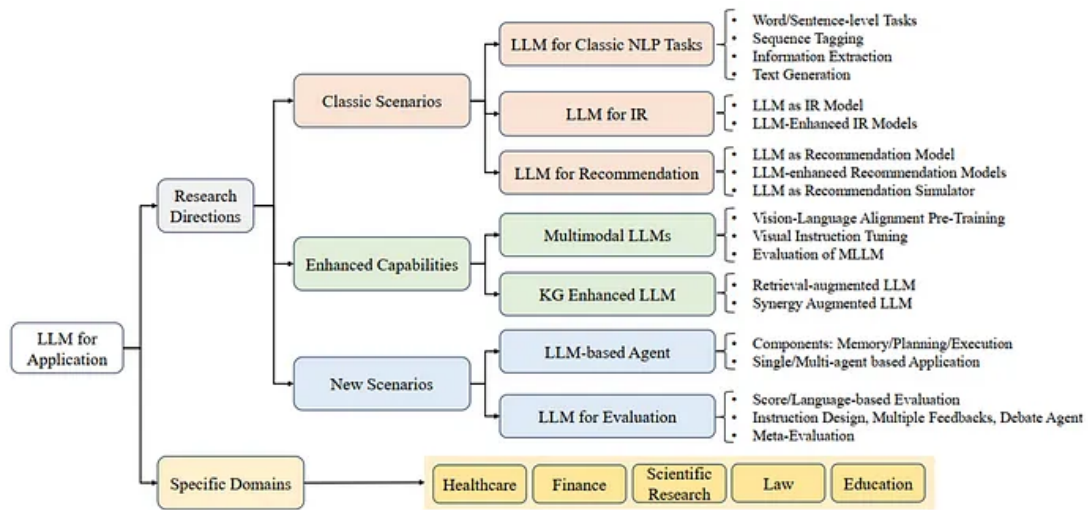


Figura 9: Esquema de las direcciones de investigación donde podemos usar LLMs [6], [12]

En el ámbito de las tareas clásicas del procesamiento del lenguaje natural, los LLMs son capaces de abordar actividades como el análisis a nivel de palabra u oración, el etiquetado de secuencias, la extracción de información y la generación de texto. En cuanto a la recuperación de información, estos modelos se utilizan como herramientas de búsqueda por sí mismas o para mejorar modelos existentes, incrementando la precisión y relevancia de los resultados obtenidos. También desempeñan un papel importante en los sistemas de recomendación, ya sea como modelos principales o como complemento de otros sistemas, simulando recomendaciones y ajustándose a las preferencias de los usuarios, como ocurre en plataformas como Netflix y YouTube [6].

Los LLMs multi-modales representan otro avance importante, al ser capaces de integrar y procesar información proveniente de distintas modalidades, como texto e imágenes, mejorando así la interacción y comprensión de contenido en diferentes formatos. Asimismo, existen LLMs mejorados con grafos de conocimiento, los cuales incorporan datos estructurados para enriquecer su capacidad de comprensión y generación de contenido con un mayor contexto.

Además de estas aplicaciones tradicionales, los LLMs se emplean en nuevos escenarios. Uno de ellos son los agentes basados en LLMs, que cuentan con capacidades de memoria, planificación y ejecución, permitiéndoles realizar tareas de manera autónoma y adaptarse a las solicitudes del usuario y al entorno. Otro caso es el uso de LLMs para evaluación, donde se aplican técnicas de evaluación para puntuar modelos o contenidos.

En cuanto a dominios específicos, los LLMs han demostrado un rendimiento notable. En sanidad, modelos como Med-PaLM [12] han alcanzado un nivel experto en el Examen de Licencia Médica de EE.UU., siendo aprobados por médicos en la resolución de preguntas médicas. En educación, herramientas como ChatGPT han ayudado a estudiantes a mejorar su rendimiento en asignaturas como seguridad informática, generando o refinando respuestas. En el ámbito legal, GPT-4 obtuvo una puntuación dentro del 10% superior en un examen de acceso a la abogacía simulado, evidenciando su capacidad para la interpretación legal y el razonamiento. En finanzas, BloombergGPT [12] [14] demostró un rendimiento destacado en múltiples tareas financieras, sin perder competitividad en ta-

reas de propósito general. Por último, en la investigación científica, los LLMs han sido utilizados en distintas fases del proceso científico, incluyendo la revisión bibliográfica, la generación de hipótesis, el análisis de datos y la redacción de artículos.

2.2.1. LLMs usados en el proyecto

Durante la realización del proyecto, hemos probado varios modelos de lenguaje, incluyendo modelos locales y externos. Los modelos seleccionados finalmente han sido tres, diferenciando las tres estrategias que podemos usar a la hora de usar un LLM, es decir: un modelo local, un modelo gratuito externo y un modelo externo de pago.

En los siguientes apartados se explican con más detenimiento los modelos elegidos y las plataformas en los que están disponibles.

Llama 3.2

Llama 3.2 [15] es modelo de lenguaje multilingüe perteneciente a Meta. Es una colección de modelos pre-entrenados diseñados para seguir instrucciones, con un tamaño de 1 a 3 billones de parámetros (1B y 3B). La versión usada en el proyecto es la 3B.

El modelo está optimizado para mantener conversaciones en varios idiomas, incluyendo tareas como la realización de agentes de búsqueda y resúmenes.

Lightweight instruction-tuned benchmarks				
Category Benchmark	Llama 3.2 1B	Llama 3.2 3B	Gemma 2 2B IT (measured)	Phi-3.5-mini IT (measured)
General				
MMLU (5-shot)	49.3	63.4	57.8	69.0
Open-rewrite eval (0-shot, rougeL)	41.6	40.1	31.2	34.5
TLDR9+ (best, 1-shot, rougeL)	16.8	19.0	13.9	12.8
IFEval	59.5	77.4	61.9	59.2
Tool Use				
BFCL V2	25.7	67.0	27.4	58.4
Nexus	13.5	34.3	21.0	26.1
Math				
GSM8K (8-shot, CoT)	44.4	77.7	62.5	86.2
MATH (0-shot, CoT)	30.6	48.0	23.8	44.2
Reasoning				
ARC Challenge (0-shot)	59.4	78.6	76.7	87.4
GPQA (0-shot)	27.2	32.8	27.5	31.9
Hellaswag (0-shot)	41.2	69.8	61.1	81.4
Long Context				
InfiniteBench/En.MC (128k)	38.0	63.3	—	39.2
InfiniteBench/En.QA (128k)	20.3	19.8	—	11.3
NIH/Multi-needle	75.0	84.7	—	52.7
Multilingual				
MGSM (0-shot, CoT)	24.5	58.2	40.2	49.8

Figura 10: Tabla comparativa del modelo Llama 3.2 en diferentes benchmarks [15]

La plataforma donde esta disponible una versión de este modelo es Ollama (ver sección 2.5). Es la opción que usaremos en el proyecto como base de pruebas de un modelo local, ya que la plataforma Ollama permite la descarga de multitud de modelo gratuitos.

Mistral Small 3.2

Mistral Small 3.2 [16] es la última actualización disponible del modelo Mistral Small. Este modelo mejora al anterior (Mistral Small 3.1 [17]) a la hora de dar respuestas precisas a instrucciones, generar menos texto repetitivo o generaciones infinitas.

El modelo Mistral Small 3.2 cuenta con un tamaño de 24 billones de parámetros (24B), algo considerable para ser usado localmente. Mistral AI ofrece este y otros modelos de forma gratuita a través de su API (con ciertas restricciones ratio/preguntas), por esa razón, es el modelo escogido para realizar las pruebas con un modelo externo gratuito.

Nota: La siguiente figura muestra el rendimiento del modelo 3.1. Mistral Small 3.2 es relativamente moderno (junio de 2025), y a día de hoy, no disponemos de un blog/página oficial que informe sobre su rendimiento actual.

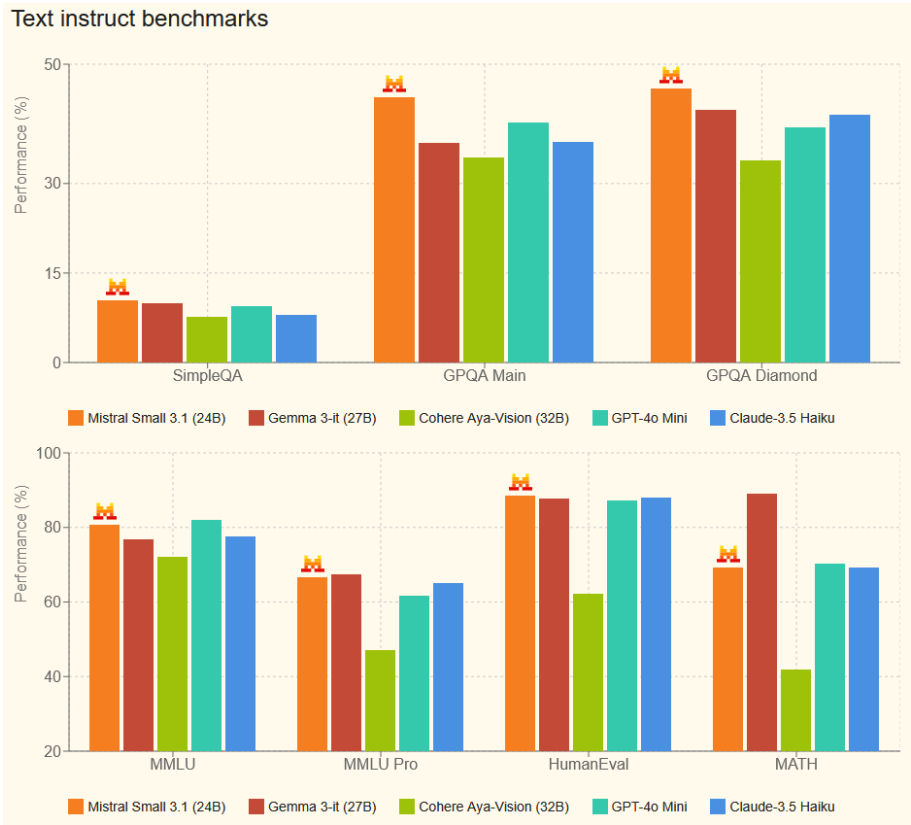


Figura 11: Tabla comparativa del modelo Mistral Small 3.1 en diferentes benchmarks [17]

GPT-4.1

El conjunto de modelos GPT-4.1 [18] esta disponible en tres tamaños: GPT-4.1, GPT-4.1 Mini y GPT-4.1 Nano. Están destinados a programadores que necesitan un mayor rendimiento, un contexto más amplio y un seguimiento de las instrucciones más predecible. Cada modelo admite hasta 1 millón de tokens de contexto, un salto apreciable dado que los modelos anteriores permitían un tamaño de contexto de 128 mil.

GPT-4.1 mejora las capacidades de GPT-4o, manteniendo la latencia más o menos en el mismo rango. En la práctica, significa que podemos obtener mejor rendimiento sin pagar un coste en capacidad de respuesta (ver figura 13 para ver los precios). En la siguiente figura, GPT-4.1 adelanta en inteligencia a GPT-4o manteniendo una velocidad similar. De igual manera, la versión *mini* es más inteligente siendo un poco más lenta.

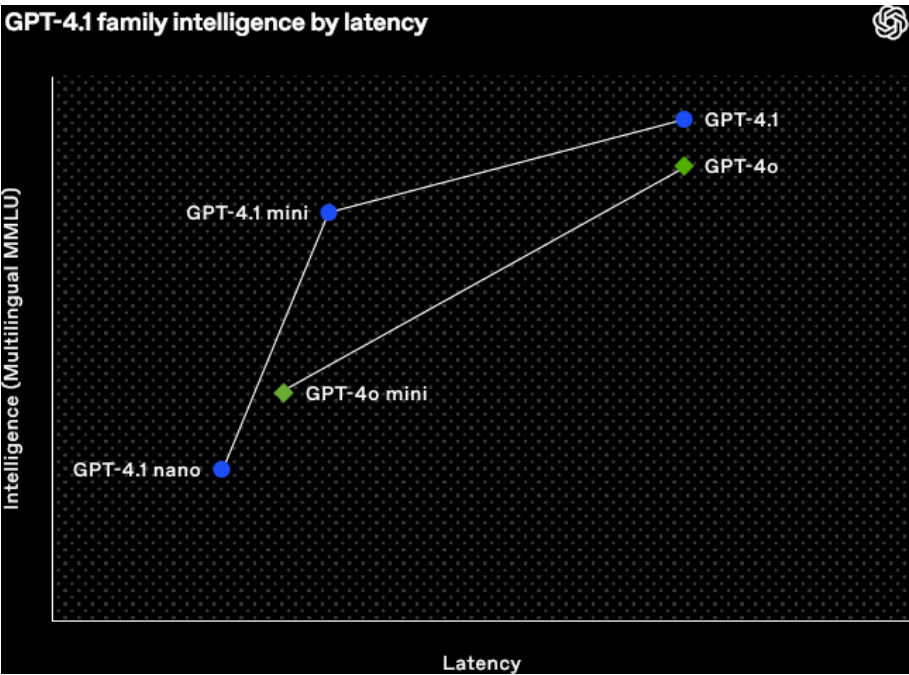


Figura 12: Rendimiento del conjunto de modelos GPT-4.1 [19]

Modelo	Entrada	Entrada en caché	Salida	Blended Avg. Coste
GPT-4.1	2,00 \$ / 1M de fichas	0,50 \$ / 1M de fichas	8,00 \$ / 1M de fichas	\$1.84
GPT-4.1 Mini	0,40 \$ / 1M de fichas	0,10 \$ / 1M de fichas	1,60 \$ / 1M de fichas	\$0.42
GPT-4.1 Nano	0,10 \$ / 1M de fichas	0,025 \$ / 1M de fichas	0,40 \$ / 1M de fichas	\$0.12

Figura 13: Precio de los modelos GPT-4.1 [19]

El modelo GPT-4.1 es el escogido realizar las pruebas, para lo cual necesitaremos conseguir una de las claves API disponibles en la plataforma de OpenAI. Siempre que tengamos créditos en la cuenta podemos acceder al modelo, por lo que hay que tener en cuenta este factor. Con este modelo de pago, ya tenemos todos los modelos de los que queremos evaluar su rendimiento en nuestro proyecto.

2.3. Modelos de Embedding

Los modelos de embedding son la parte esencial en el modelo de recuperación de información de un sistema RAG, ya que permiten transformar datos complejos como

texto, imágenes o audio [20]; en representaciones vectoriales densas y de baja dimensión. Estas representaciones conservan relaciones semánticas relevantes y resultan mucho más manejables para recuperar documentos relevantes dada una consulta de usuario.

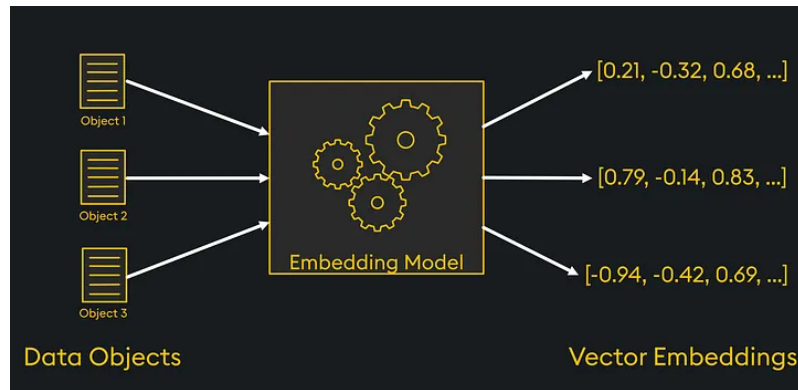


Figura 14: Ejemplo básico de funcionamiento de un modelo de embeddings [20]. Recibe objetos de datos, que pueden ser de todo tipo, y los transforma en una representación vectorial

Por otra parte, los embeddings también son una pieza fundamental en multitud de técnicas de aprendizaje automático. Los vectores generados pueden emplearse en tareas como clasificación, agrupamiento (clustering), recomendación o recuperación de información, etc.

Gracias a su versatilidad, los modelos de embedding se utilizan en una amplia variedad de aplicaciones del mundo real. En sistemas de análisis de sentimientos, permiten identificar matices emocionales en el texto a partir de la representación semántica de las palabras. En sistemas de recomendación, se emplean para medir similitudes entre usuarios y productos, ofreciendo sugerencias personalizadas. En tareas de respuesta automática a preguntas, los embeddings permiten emparejar preguntas y respuestas en función de su similitud semántica, facilitando la recuperación de la respuesta más adecuada. Finalmente, debido a que estos modelos pueden representar palabras y frases en múltiples idiomas dentro de un mismo espacio vectorial, también resultan especialmente útiles en tareas de traducción automática, donde es posible identificar significados equivalentes entre lenguas diferentes [20].

En resumen, los modelos embeddings son una herramienta clave para representar datos de forma compacta y significativa, lo que habilita una gran variedad de aplicaciones en el campo del aprendizaje automático y el procesamiento del lenguaje natural.

En las siguientes sub-secciones hablaremos de los modelos de embeddings utilizados en nuestro sistema RAG.

2.3.1. Nomic Embed (nomic-embed-text)

Nomic Embed [21] es un modelo avanzado de embedding de texto desarrollado por **Nomic**, diseñado para ofrecer una capacidad excepcional en tareas tanto de corto como de largo contexto. Este modelo sobresale frente a alternativas como *Ada-002* o *text-embedding-3-small* de OpenAI (figura 15), gracias a su enfoque en la accesibilidad y auditabilidad completa del modelo.

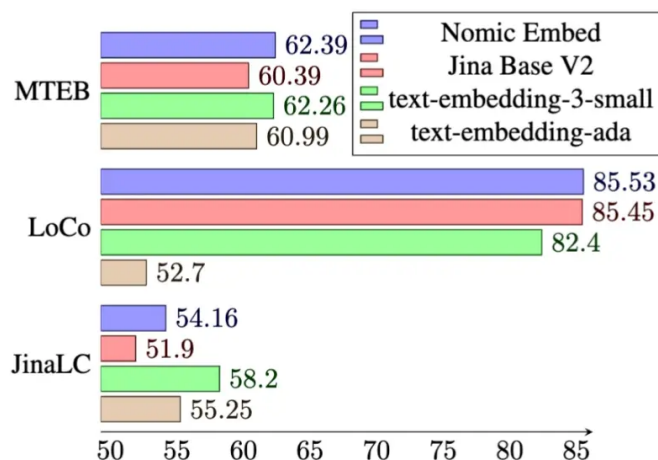


Figura 15: Rendimiento comparativo de Nomic Embed y otros modelos en distintos benchmarks [21]

Las características destacadas de este modelo [21] se pueden resumir en: código abierto y rendimiento. Nomic Embed se distingue por su enfoque abierto completamente abierto, incluyendo datos de entrenamiento, código de entrenamiento y modelos bajo licencia Apache2.

Evaluado en benchmarks como *MTEB* y *LoCo* (figura 15), Nomic Embed supera a modelos líderes en tareas de largo contexto, demostrando consistencia y superioridad en entornos supervisados y no supervisados.

En resumen, nomic Embed se posiciona como una opción muy recomendable para proyectos que buscan integrar modelos de embedding de texto avanzados y completamente auditables.

2.3.2. BGE-M3

BGE-M3 [22] es otro modelo de embedding de texto basado en la arquitectura *XLNet*-*RoBERTa*, diseñado para ser altamente versátil en diversos escenarios de recuperación de información. Su nombre proviene de sus tres capacidades:

- **Multi-funcionalidad:** BGE-M3 es capaz de realizar de manera simultánea recuperación densa, recuperación con multiples vectores y recuperación dispersa, ampliando significativamente su rango de aplicación.
- **Multilingüe:** Este modelo ofrece soporte para más de cien idiomas, lo que lo convierte en una herramienta eficaz en contextos multilingües y de alcance global.
- **Multi-granularidad:** Puede procesar texto con distintos niveles de granularidad, desde frases cortas hasta documentos de hasta 8192 tokens de longitud.

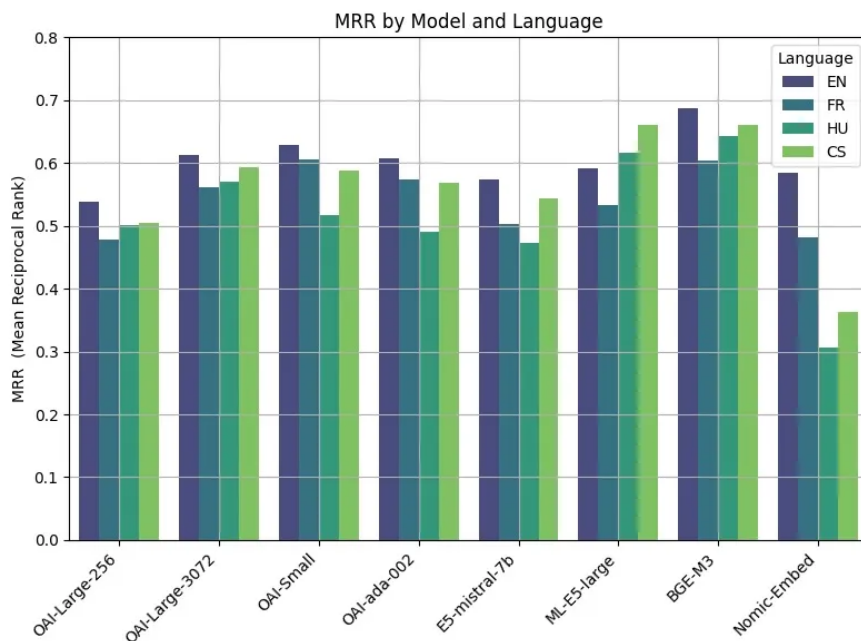


Figura 16: Rendimiento comparativo del modelo bge-m3 con otros modelos [22]

Definitivamente, BGE-M3 resulta ideal para sistemas de búsqueda semántica multilingüe y cualquier entorno donde se requiera una comprensión profunda del texto en múltiples idiomas. Su enfoque multifuncional representa un avance significativo en el desarrollo de modelos de embedding flexibles, escalables y multilingües.

2.3.3. Artic Embed 2.0 (snowflake-arctic-embed2)

El modelo Artic Embed 2.0 [23], desarrollado por Snowflake, representa la nueva iteración en sus modelos de embedding *frontier*. Esta nueva versión incorpora soporte multilingüe sin comprometer su rendimiento o escalabilidad.

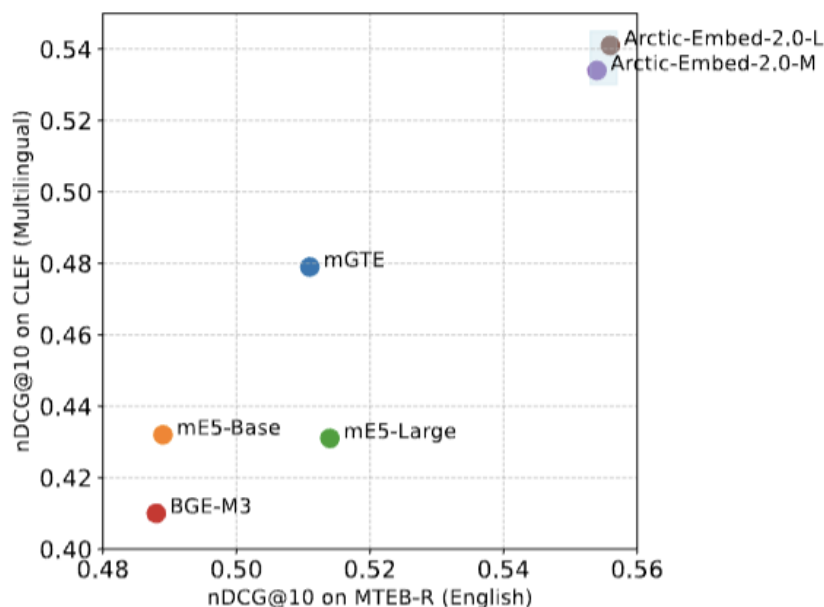


Figura 17: Métricas obtenidas por los modelos de Artic Embed en comparación con otros, usando el benchmark MTEB-R en inglés. [23]

Artic Embed 2.0 logra resultados altos en tareas de recuperación semántica tanto en inglés como en otros idiomas europeos, demostrando una notable capacidad de generalización incluso en lenguas no vistas durante el entrenamiento.

Snowflake ha diseñado este modelo pensando en entornos de alta demanda, ofreciendo latencias de 10ms por consulta. Incluye una compresión eficiente (MRL, Matryoshka Representation Learning) que permite búsquedas rápidas y económicas en grandes conjuntos de datos.

Este modelo se distribuye bajo una licencia de uso Apache 2.0, permitiendo su libre uso.

En resumen, Snowflake ofrece un modelo de embedding con soporte multilingüaje, capacidad de compresión y orientado a entornos empresariales.

2.3.4. Jina Embeddings v2 Base ES (jina-embeddings-v2-base-es)

El modelo de embedding bilingüe español-inglés *jina-embeddings-v2-base-es* [24], desarrollado por Jina AI, está diseñado para aplicaciones que requieren representaciones semánticas robustas en cualquiera de estos dos idiomas.

Basado en una arquitectura BERT (JinaBERT), este modelo está pensado para ofrecer un alto rendimiento en aplicaciones monolingües y bilingües ya que ha sido entrenado específicamente para admitir entradas de español e inglés sin sesgo.

Además, su tamaño se ubica entre los más ligeros entre los embeddings multilingües disponibles, facilitando su implementación en entornos con recursos limitados.

Por último, puede integrarse fácilmente vía Python a través de Ollama, y es apto para sistemas RAG según pruebas recientes con LlamaIndex (ver figura 18).

Embedding	WithoutReranker		bge-reranker-base		bge-reranker-large		Cohere-Reranker	
	Hit Rate	MRR	Hit Rate	MRR	Hit Rate	MRR	Hit Rate	MRR
OpenAI	0.859551	0.697285	0.904494	0.819944	0.910112	0.847846	0.926966	0.850749
bge-large	0.786517	0.629588	0.876404	0.804213	0.876404	0.824157	0.88764	0.821255
llm-embedder	0.786517	0.57191	0.848315	0.780618	0.853933	0.80309	0.859551	0.809363
Cohere-v2	0.764045	0.533801	0.876404	0.7897	0.882022	0.821723	0.876404	0.809925
Cohere-v3	0.825843	0.635206	0.876404	0.79485	0.876404	0.822004	0.876404	0.825562
Voyage	0.842697	0.673596	0.921348	0.820693	0.910112	0.841011	0.904494	0.832959
JinaAI-Small	0.820225	0.625187	0.893258	0.81779	0.904494	0.836517	0.910112	0.844757
JinaAI-Base	0.842697	0.676966	0.932584	0.840918	0.938202	0.868071	0.938202	0.865075

Figura 18: Comparación de varios modelos de embedding con JINA. [24]

2.3.5. Text Embedding 3 (text-embedding-3-small)

El modelo *text-embedding-3-small* es una versión mejorada y eficiente del modelo de embedding anterior *text-embedding-ada-002*, ambos pertenecientes a **OpenAI** [25].

Como se puede observar en la figura 19, la puntuación media en una prueba comparativa de uso común para la recuperación en varias lenguas (MIRACL) aumento de un 31,4 % hasta el 44 %, mientras que la puntuación media para tareas en inglés (MTEB) aumento del 61 % hasta el 62,3 %.

Puntos de referencia de evaluación	ada v2	text-embedding-3-small	text-embedding-3-large
Media en MIRACL	31,4	44,0	54,9
Media en MTEB	61,0	62,3	64,6

Figura 19: Evaluaciones de los modelos de embedding de OpenAI [25]

El precio de uso también se ha reducido al ser un modelo más eficiente que el anterior, casi cinco veces.

Este modelo ha sido entrenado con una técnica que permite a los desarrolladores equilibrar el rendimiento y costes al utilizar las integraciones, podemos acortarlas sin perder sus propiedades de representación de conceptos (ver figura 20).

	ada v2	text-embedding-3-small	text-embedding-3-large
Embedding size	1536	512	1536
Average MTEB score	61.0	61.6	62.3
			62.0
			64.1
			64.6

Figura 20: Evaluaciones de los modelos de embedding de OpenAI con distintos tamaños [25]

De este modo se logra un uso altamente flexible, permitiendo utilizar almacenes de datos

vectoriales más optimizados sin perder rendimiento.

2.4. Alternativa al uso de LLM: RASA Pro

En esta sección abordaremos una alternativa al uso de los LLMs, **Rasa Pro**. En un principio este proyecto iba a ser desarrollado usando la herramienta Rasa Open Source, un framework gratuito de RASA para la creación de asistentes conversacionales basado en el entrenamiento de una red neuronal con un conjunto de “intenciones” de usuario y de respuestas asociadas.

Rasa se consolidó como una de las principales plataformas para el desarrollo de asistentes conversacionales gracias a que ofrecían un marco flexible basado en el procesamiento del lenguaje natural y el aprendizaje automático. Sin embargo, los modelos tradicionales usados por Rasa, limitan su capacidad para comprender consultas complejas o para responder de una forma natural, a parte de la necesaria actualización de la red para cada cambio realizado.

En ese contexto surgió Rasa Pro, la versión empresarial de Rasa. Rasa Pro incorpora funcionalidades avanzadas que aprovechan las capacidades de los LLMs. Uno de los aspectos clave es el uso de CALM (Conversational AI for Language Models [26]), un enfoque que combina la flexibilidad de Rasa para gestionar diálogos con la potencia de un modelo de lenguaje. Esta integración permite obtener respuestas más naturales y contextualizadas, mejorar de entradas de usuario ambiguas y reducir el tiempo de entrenamiento.

A continuación, apreciaremos algunas de las características diferenciadoras con Rasa Open Source, y también, expondremos el porque hemos optado por el uso de LLMs tradicionales en vez de esta herramienta.

2.4.1. Licencia de uso

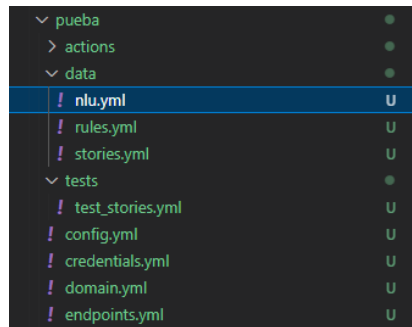
Rasa Pro es un producto que requiere de una licencia de uso, siendo esta uno de sus principales inconvenientes. Aunque es posible obtener una licencia para desarrolladores de manera gratuita, esta presenta ciertas limitaciones:

- Límite de 1000 conversaciones mensuales, 100 para asistentes internos dirigidos a empleados.
- Uso restringido a un sólo usuario.
- Permite su uso en entornos comerciales, pero tiene restricciones especificadas en la licencia.
- Validez de cada clave de 12 meses.

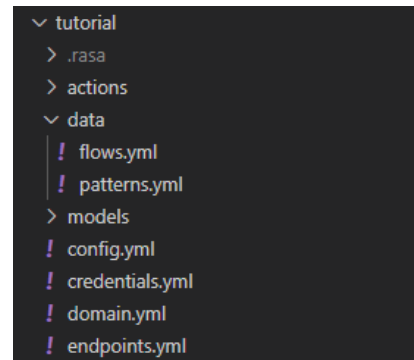
Encontramos dos problemas clave en la lista anterior. En primer lugar, aunque la licencia es gratuita requiere de una renovación anual, lo que puede presentar un problema a largo plazo. El segundo inconveniente, es un límite de conversaciones mensuales demasiado bajo para las necesidades de cualquier proyecto medianamente “importante”, y que presenta una desventaja muy grande en comparación con otras soluciones de código abierto, las cuales no imponen restricciones de uso y son completamente gratuitas.

2.4.2. Facilidad de uso

Un primer vistazo a la estructura y al código generado por Rasa Pro, nos permite observar que, aunque se ha mantenido la misma estructura de proyecto, se han sustituido los ficheros de la carpeta *data* por otros nuevos (ver figura 21).



(a) Estructura de Rasa



(b) Estructura de Rasa Pro

Figura 21: Comparación de las dos estructuras

Estos nuevos archivos engloban los flujos de conversación (ver figura 22) que el asistente debe seguir. Los flujos están diseñados para recoger toda la información necesaria para realizar el proceso que tienen programado.

```
flows:
  transfer_money:
    description: Help users send money to friends and family.
    steps:
      - collect: recipient
      - collect: amount
        description: the number of US dollars to send
      - action: action_check_sufficient_funds
        next:
          - if: not slots.has_sufficient_funds
            then:
              - action: utter_insufficient_funds
              next: END
          - else: final_confirmation
      - collect: final_confirmation
        id: final_confirmation
        next:
          - if: not slots.final_confirmation
            then:
              - action: utter_transfer_cancelled
              next: END
          - else: transfer_successful
      - action: utter_transfer_complete
        id: transfer_successful
```

(a) flows.yml

```
flows:
  pattern_chitchat:
    description: Conversation repair flow for off-topic interactions
      that won't disrupt the main conversation. should not respond
      to greetings or anything else for which there is a flow defined
    name: pattern_chitchat
    steps:
      - action: utter_free_chitchat_response
  pattern_search:
    description: Flow for handling knowledge-based questions
    name: pattern_search
    steps:
      - action: utter_free_chitchat_response
```

(b) patterns.yml

Figura 22: Contenido de los ficheros flows.yml y patterns.yml

Los llamados *Slots* (la memoria del asistente) son leídos en el flujo mediante la etiqueta *collect* y son manejados, al igual que las respuestas, de la misma manera que con Rasa Open Source, en el archivo *domain.yml*. Es posible añadir un prompt en este archivo para ayudar al modelo a manejar las respuestas (ver figura 23)

```
utter_free_chitchat_response:
- text: "placeholder"
  metadata:
    rephrase: True
    rephrase_prompt: |
      The following is a conversation with an AI assistant built with Rasa.
      The assistant can help the user transfer money.
      The assistant is helpful, creative, clever, and very friendly.
      The user is making small talk, and the assistant should respond, keeping things light.
      Context / previous conversation with the user:
      {{history}}
      {{current_input}}
      Suggested AI Response:
```

Figura 23: Ejemplo de una respuesta con prompt integrado

En resumen, los cambios que aporta esta versión hacen que controlar el flujo de conversación sea más sencillo y claro. Ahora las intenciones de usuario, son deducidas gracias al uso de modelos de lenguaje. El código es fácil de adaptar a las necesidades de cualquier proyecto y permite controlar las respuestas dadas por el asistente, debido a que todo este permanece *on premise*.

2.4.3. Rendimiento

El asistente de prueba de Rasa Pro usa por defecto un modelo de lenguaje desarrollado por la misma empresa: **codellama** [27]. Este modelo ya está preparado para usarse en asistentes implementados bajo el CALM, es decir, este modelo traduce los mensajes enviados por un usuario a los comandos que son necesarios para avanzar en la conversación. De acuerdo a la documentación, este modelo no es capaz de generar texto arbitrario o creativos, ni texto problemático o dañino.

A la hora de probar el modelo, Rasa Pro incluye un modo “inspector”, nos permite visualizar en una interfaz de navegador un grafo con el flujo de conversación en tiempo real (ver figura 24).

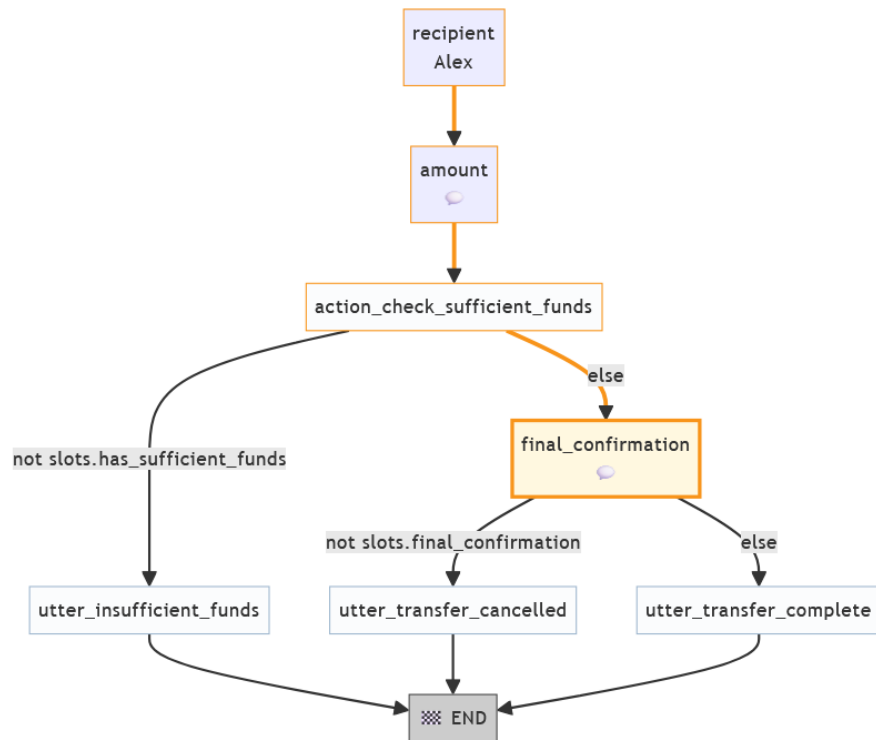


Figura 24: El inspector nos muestra un grafo con el flujo de conversación

Otro aspecto importante es, que en las conversaciones podemos optar por dar los datos uno a uno o todos a la vez (ver figura 25). Esta característica ya estaba presente en la versión anterior de Rasa (Rasa Open Source) y en esta la han mantenido.

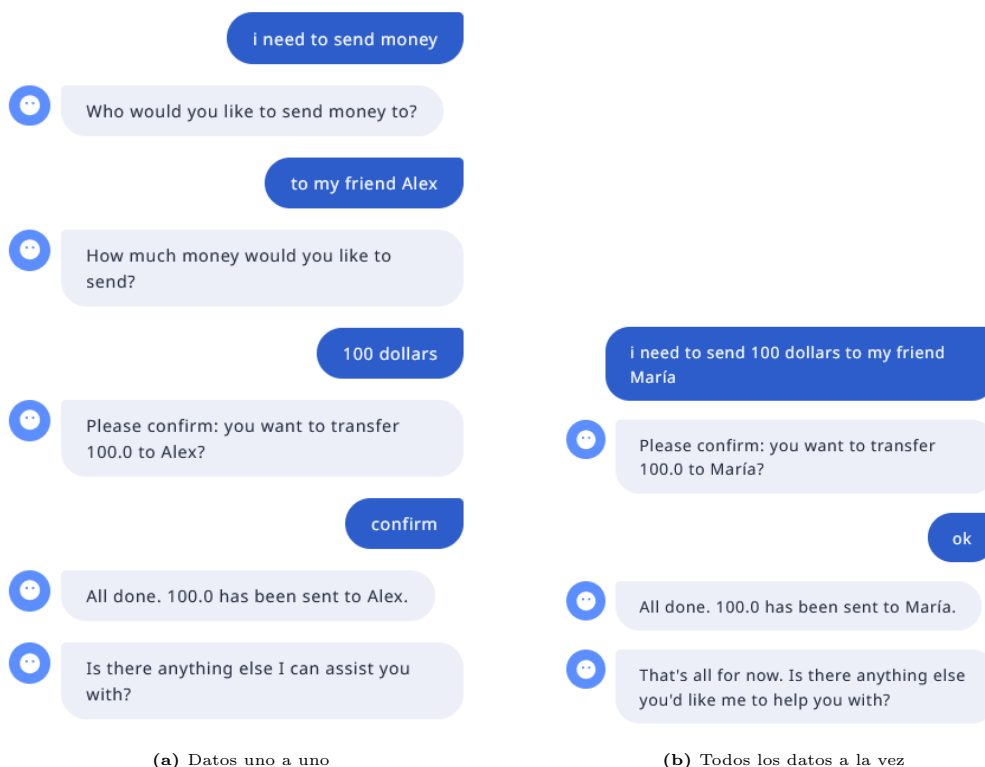


Figura 25: Maneras en las que puedes conversar en una aplicación hecha con Rasa Pro

En cuanto al rendimiento, el tiempo de ejecución del modelo de prueba es casi inmediato. Sin embargo, como hemos mencionado anteriormente, la imposibilidad de este modelo de generar respuestas creativas, lo convierten en una opción no muy recomendable.

Rasa Pro permite modificar el modelo de lenguaje dentro de la configuración del proyecto, ofreciendo compatibilidad con modelos tanto locales como en la nube, incluyendo OpenAI y otras alternativas.

En nuestras pruebas, utilizamos un modelo local *llama3.2* descargado gracias a la plataforma **Ollama** (ver sección 2.5). Los resultados obtenidos no fueron muy buenos, además de presentar tiempos de respuesta significativamente altos en comparación con el modelo anterior, en ocasiones omitía pasos dentro de la conversación. No se realizaron pruebas con modelos de OpenAI debido a la falta de clave de acceso a sus servicios.

2.4.4. Conclusión respecto a Rasa Pro

La integración de base de Rasa Pro con los modelos de lenguaje representa un avance significativo para el desarrollo de asistentes conversacionales en esta plataforma. Con la adopción del enfoque CALM [26], podemos gestionar diálogos de una forma más dinámica eliminando la necesidad de definir las intenciones explícitamente.

Sin embargo y, como hemos mencionado en anteriores apartados, Rasa Pro cuenta con una serie de limitaciones que deben ser consideradas. Las restricciones puestas por su licencia gratuita pueden ser un obstáculo para aplicaciones que requieran una más alta cantidad

de interacciones. El modelo de lenguaje de prueba incorporado, aunque ofrece respuestas en muy poco tiempo, su capacidad de generar respuestas es también limitada y puede no ajustarse a todos los casos de uso.

En nuestras pruebas, el uso de modelos de lenguajes locales no logró resultados satisfactorios debido a unos tiempos de respuesta muy elevados y ciertas inconsistencias en la generación de respuestas.

En definitiva, aunque Rasa Pro es una opción potente y viable para desarrollar asistentes conversacionales, presenta bastantes inconvenientes. Las limitaciones que tenemos tanto por la licencia como por el uso de modelos de lenguaje locales, nos hacen movernos hacia otras opciones de código libre (algunas ya mencionadas), que no requieren de licencia o que no ponen restricciones de uso.

2.5. Ollama

Ollama [28] es una herramienta de código abierto en la que podemos usar LLMs disponibles gratuitamente en nuestra propia máquina local (o servidor). Ollama actúa como un puente entre el modelo de lenguaje y nuestra máquina, no solo permitiéndonos usarlo, si no que también ofreciendo una API que podemos usar en aplicaciones más avanzadas.



Get up and running with large
language models.

Run [DeepSeek-R1](#), [Qwen 3](#), [Llama 3.3](#), [Qwen 2.5-VL](#),
[Gemma 3](#), and other models, locally.

Explore models →

Available for macOS, Linux,
and Windows

Figura 26: Sitio web oficial de Ollama: <https://ollama.com/>

Al acceder localmente a los modelos, podemos mantener un control total sobre los datos y, evitar la inseguridad o los riesgos asociados con los servicios en la nube. Además, las herramientas sin conexión que ofrece Ollama ayudan a reducir latencias y dependencias de servidores externos, haciéndolas más rápidas (dependiendo mucho del equipo en el se

ejecute) y confiables.

En los siguientes apartados, repasaremos algunas de las características más importantes de esta herramienta y mostraremos un ejemplo de uso.

2.5.1. Características principales de Ollama

Privacidad y acceso sin conexión

Es una de las principales características [28] que ofrece Ollama, la cual nos da la opción de ejecutar modelos de forma privada en nuestra máquina y sin conexión a Internet. De esta manera, podemos mantener nuestros datos de forma privada en nuestra equipo local sin que sean enviados a través de la red. Esta es una gran ventaja de Ollama sobre otros servicios en la nube, que podrían usar nuestros datos para otros propósitos.

Manejo de modelos

Añadir modelos a nuestra librería [28] local de Ollama es un proceso muy sencillo (tal y como se describe en la sección 2.5.2). En el momento de escribir esta memoria el repositorio de Ollama cuenta con más de 150 modelos listos para usarse, desde *Deepseek R1* hasta otros más populares como *Llama3*, *Gemma3*, *Qwen2.5* o *Mistral*. La web oficial de Ollama cuenta con la lista de los modelos disponibles.

Integración sencilla

Ollama se puede integrar fácilmente con una amplia variedad de herramientas, librerías y lenguajes de programación, por lo que es sencillo para los programadores incorporar LLMs en su flujo de trabajo [28, 29]. A parte de *Python*, *LangChain* (ver sección 2.7) o *LlamaIndex*, Ollama presenta unas opciones de integración efectivas para el desarrollo de aplicaciones y soluciones avanzadas basadas en la IA.

Personalización y ajuste (Fine-tuning)

Ollama también nos da la posibilidad de ajustar ciertos parámetros de los modelos, como por ejemplo el tamaño de la ventana de contexto que usa el modelo para generar su respuesta o la temperatura/creatividad al generar la misma. Además, Ollama incluye el mecanismo *modelfile*, en el cual podemos editar los parámetros del modelo usando un modelo base (un ejemplo es mostrado en la figura 27).

```
FROM llama3.2
# sets the temperature to 1 [higher is more creative, lower is more coherent]
PARAMETER temperature 1
# sets the context window size to 4096, this controls how many tokens the LLM
PARAMETER num_ctx 4096

# sets a custom system message to specify the behavior of the chat assistant
SYSTEM You are Mario from super mario bros, acting as an assistant.
```

Figura 27: Fichero *modelfile* básico, edita la temperatura y la ventana del contexto del modelo. Tiene de base el LLM *Llama3.2*

2.5.2. Instalación y funcionamiento de Ollama

El archivo de instalación de Ollama se encuentra en su página web oficial, hay disponibles tres versiones para su descarga: *Linux*, *macOS* y *Windows*. La instalación en *Windows* no difiere de cualquier otro software para este sistema operativo, las otras dos versiones requieren de usar la consola para la instalación:

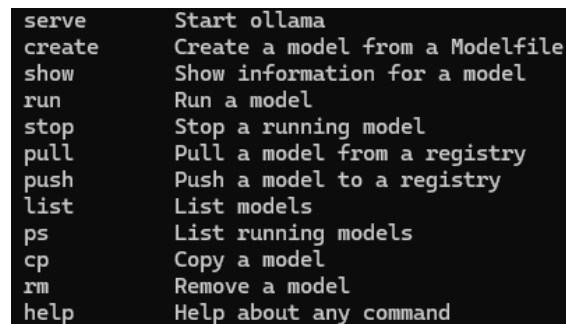
- Para *Linux*:

```
chmod +x ollama_linux.sh
./ollama_linux.sh
```

- Para *macOS*:

```
chmod +x ollama_macos.sh
./ollama_macos.sh
```

Una vez instalado, podemos usar Ollama mediante la consola de comandos de nuestro sistema. La siguiente figura muestra los comandos disponibles:

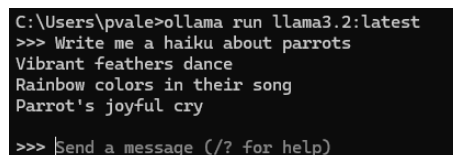
A screenshot of a terminal window showing a list of Ollama commands and their descriptions. The text is as follows:

serve	Start ollama
create	Create a model from a Modelfile
show	Show information for a model
run	Run a model
stop	Stop a running model
pull	Pull a model from a registry
push	Push a model to a registry
list	List models
ps	List running models
cp	Copy a model
rm	Remove a model
help	Help about any command

Figura 28: Lista de comandos disponibles de Ollama

Como podemos observar en la figura 28, para usar un modelo tenemos dos opciones: la primera es descargarlo (*pull*) y después ejecutarlo (*run*), y la segunda es directamente ejecutarlo (*run*), ya que lo descargará automáticamente. El comando *serve* es también muy importante ya que inicia la API de Ollama.

Por último, las figuras 29 y 30, muestran dos ejemplos de uso de los modelos: la primera figura muestra la ejecución desde línea de comandos, mientras que la segunda figura, muestra un pequeño programa escrito en *Python* que además usa la librería *LangChain* (sección 2.7).

A screenshot of a Windows command prompt showing the execution of the Ollama CLI. The text is as follows:

```
C:\Users\pvale>ollama run llama3.2:latest
>>> Write me a haiku about parrots
Vibrant feathers dance
Rainbow colors in their song
Parrot's joyful cry
>>> Send a message (/? for help)
```

Figura 29: Prueba del CLI de Ollama, ejecuta el modelo *Llama3.2* para hacer una consulta



Figura 30: Pequeño programa escrito en *Python*, usando la librería *LangChain*. Pide al modelo *Llama3.2* una consulta

2.5.3. Alternativas: LM Studio

LM Studio [30] es aplicación multi-plataforma (*Windows*, *Linux* y *macOS*) en la que podemos usar grandes modelos de lenguaje de forma local, sin conexión a la nube. Es similar a Ollama salvo que esta no es una opción de código abierto y además, cuenta con una interfaz gráfica sencilla e intuitiva (ver figura 31).

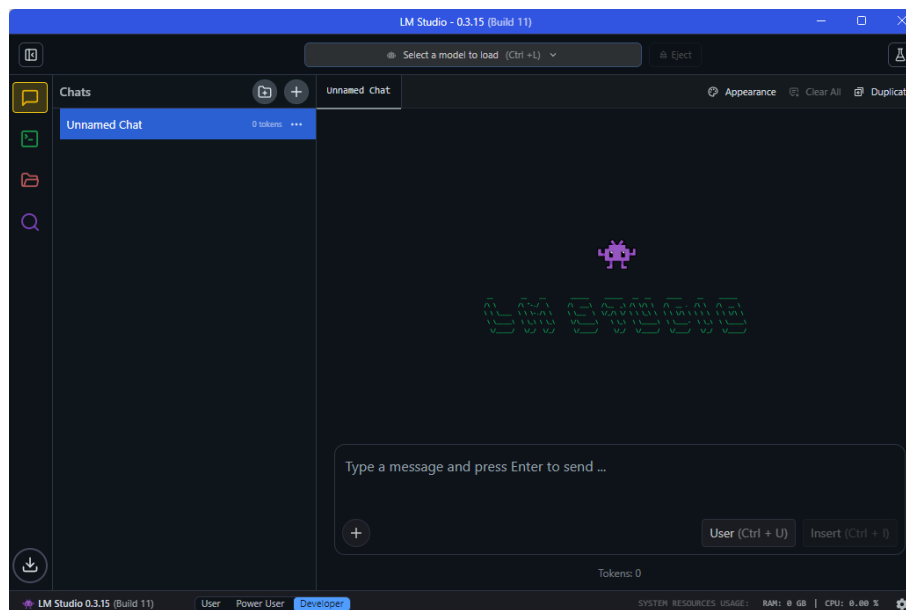


Figura 31: Pantalla principal de LM Studio, la pestaña activa nos permite chatear con un LLM

Fijándonos en la figura 31, la parte de arriba posee una especie de botón usado para seleccionar y cargar el LLM que deseemos usar (ver figura 32).



Figura 32: Selección y carga de una versión de *Llama3.2* en LM Studio

Siguiendo en la figura 31, la barra de la izquierda dispone de otras tres pestañas (de arriba a abajo):

- **Developer:** Nos permite activar o desactivar la API de LM Studio (ver figura 33).
- **My models:** En la cual podemos ver los modelos descargados (figura 34).
- **Discover:** Permite descargar nuevos modelos (figura 35).

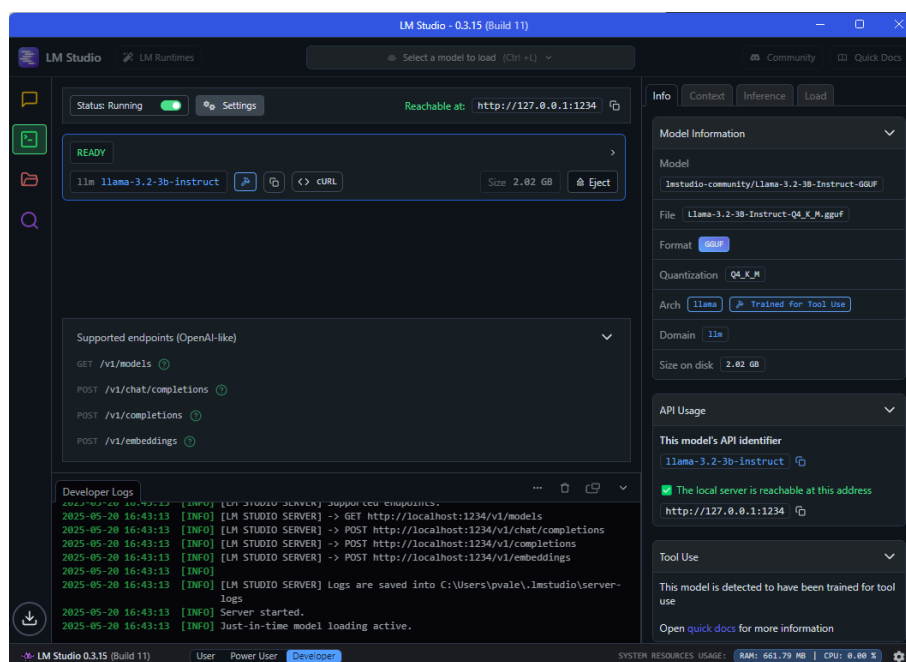


Figura 33: Developer. Pestaña que permite controlar la API de LM Studio, se observa el modelo cargado, la dirección IP de esta y los logs

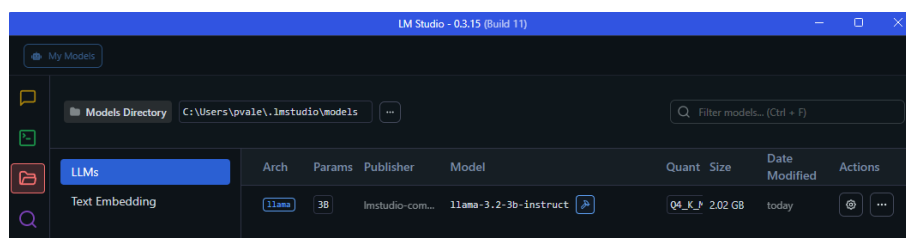


Figura 34: My Models. Pestaña que permite ver los LLMs descargados en LM Studio

Conclusiones: Ollama - LM Studio

Ollama y LM Studio presentan dos herramientas potentes para manejar LLMs de forma local. Aunque LM Studio ofrece una interfaz atractiva y sencilla de usar, son las integraciones de este las que dificultan su uso en aplicaciones más avanzadas.

Ollama esta pensado para esto último, siendo más sencillo de integrar con librerías externas, ya sean para ayudar en la recuperación de información (ChromaDB 2.6) o dedicadas al manejo de LLMs (LangChain 2.7).

Para finalizar, pese a que Ollama no cuenta con una interfaz gráfica, los comandos disponibles para la consola son intuitivos y fáciles de aprender, facilitando su adopción para usuarios menos expertos.

2.6. ChromaDB

ChromaDB [31] es una base de datos vectorial de código abierto, diseñada para almacenar y recuperar eficientemente vectores de embedding (sección 2.3), es decir, las representaciones numéricas de elementos como documentos de texto, imágenes, o vídeos. Su principal función es almacenar estas representaciones vectoriales con una serie de metadatos asociados para su posterior procesamiento por parte de un LLM.

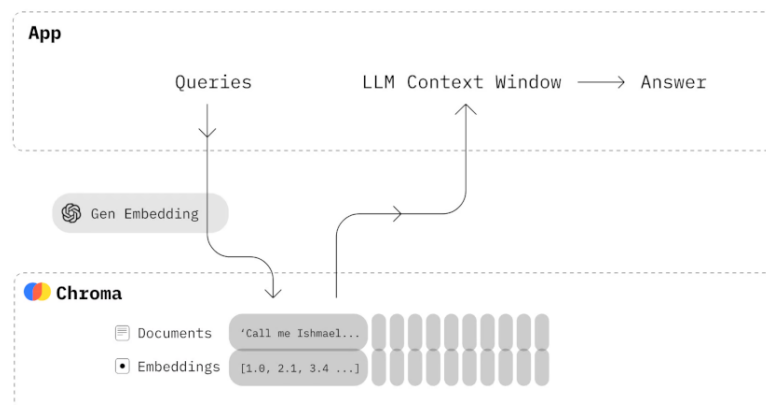


Figura 37: Esquema básico del funcionamiento de ChromaDB

2.6.1. Características de ChromaDB

ChromaDB posee un conjunto de características que hacen el adecuado para ser la base de datos vectorial de cualquier proyecto [32]. La lista siguiente muestra las más importantes:

- **Simple y potente:** Se instala con un simple comando `“pip install chromadb”` y se integra sin problemas mediante la SDK de *Python*, permitiendo una rápida configuración.
- **Multi-función:** Cuenta con una amplia gama de funcionalidades como la búsqueda vectorial, búsqueda de texto, almacenamiento de documentos, filtrado de metadatos, y recuperación multi-modal. Asimismo, es apto para aplicaciones que requieran una alta escalabilidad.

- **Gran soporte de lenguajes:** Ofrece SDKs para los lenguajes de programación más populares, incluyendo *Python*, *JavaScript/TypeScript*, *Ruby*, *PHP* y *Java*.
- **Integración de base:** Cuenta con un enfoque de integración nativo para modelos de embedding de *HuggingFace*, *OpenAI*, *Ollama*, y más. Del mismo modo, es compatible con las librerías *LangChain*, *LlamaIndex*, y muchas más previstas en un futuro.
- **Almacenamiento local:** Las bases de datos pueden ser almacenadas tanto en memoria como en nuestro equipo local, manteniendo nuestra información privada y no dependiente de servicios externos.
- **Código abierto:** Bajo la licencia Apache2.0.

2.6.2. Funcionamiento de ChromaDB

El funcionamiento de esta base de datos es simple y se puede poner en marcha rápidamente utilizando *Python* [33]. En la figura 38 se muestra un método para crear la base de datos, añadir documentos y realizar consultas (líneas 4, 8 y 16 respectivamente).

```
1  import chromadb
2  chroma_client = chromadb.Client()
3
4  collection = chroma_client.get_or_create_collection(
5      name="my_collection"
6  )
7
8  collection.upsert(
9      documents=[
10         "This is a document about pineapple",
11         "This is a document about oranges"
12     ],
13     ids=["id1", "id2"]
14 )
15
16 results = collection.query(
17     query_texts=["This is a query document about florida"],
18     n_results=2
19 )
20
21 print(results)
```

Figura 38: ChromaDB, puesta en marcha mediante *Python*

Existen varias formas de realizar cada uno de estos pasos. Por ejemplo, al crear el cliente Chroma, podemos especificar el modelo de embedding (por defecto se utiliza *all-MiniLM-L6-v2*).

En cuanto a la inserción de documentos, cuando la base de datos recibe un texto, lo convierte en su representación vectorial utilizando el modelo de embedding y asocia los metadatos correspondientes en el proceso.

Finalmente, las consultas también se vectorizan utilizando el mismo modelo (preferiblemente y devuelven los documentos más relevantes mediante una función de búsqueda (por defecto, la similitud coseno).

Los resultados (figura 39) obtenidos al ejecutar el código mostrado en la figura anterior

indican las distancias (línea 10) de los documentos recuperados. Estos resultados representan el output de la búsqueda en la base de datos, donde el documento cuyo ID es “id2” tiene una puntuación más alta (menos es mejor).

```
1  {
2    'ids': [['id2', 'id1']],
3    'embeddings': None,
4    'documents': [['This is a document about oranges',
5                  'This is a document about pineapple']],
6    'uris': None,
7    'included': ['metadata', 'documents', 'distances'],
8    'data': None,
9    'metadata': [[None, None]],
10   'distances': [[1.1462137699127197, 1.3015384674072266]]
11 }
```

Figura 39: ChromaDB. Resultados de la consulta, se observan las distancias de los documentos recuperados

2.6.3. Alternativas: Pinecone

Pinecone [34] es una base de datos vectorial gestionada en la nube [35], diseñada para realizar búsquedas en tiempo real por similitud siendo altamente escalable. Esta tecnología ha ganado en popularidad por su facilidad de uso y por su rendimiento.

Pros y contras de Pinecone

Este modelo de recuperación contiene un conjunto de ventajas similar a ChromaDB con la única excepción del alojamiento web. Veamos algunas de ellas:

- **Búsqueda en tiempo real:** Pinecone ofrece una capacidad de búsqueda asombrosamente rápida, permitiendo a los usuarios recuperar vectores similares en tiempo real.
- **Escalabilidad:** La arquitectura de Pinecone escala ante un crecimiento de la demanda.
- **Indexado automático:** Los vectores son indexados automáticamente por Pinecone, reduciendo la carga de trabajo y simplificando el proceso de despliegue.
- **Soporte Python:** La SDK de *Python* es sencilla de utilizar, haciendo accesible Pinecone para todos los desarrolladores y científicos de datos familiarizados con este ecosistema.

Sin embargo, tiene dos inconvenientes que podrían ser clave para un proyecto. Uno de ellos es el coste, al ser un servicio en la nube, Pinecone ofrece un modelo de negocio basado en tres tipos de categorías: *Starter* gratuito y pensado para pequeñas aplicaciones, *Standard* para aplicaciones de producción a cualquier escala y *Enterprise* para aplicaciones críticas.

La segunda desventaja es la funcionalidad limitada a la hora de realizar consultas avanzadas. A pesar de que Pinecone sobresale en búsquedas por similitud, podría carecer de algunas capacidades de consulta avanzadas que ciertos proyectos requieren.

Funcionamiento de Pinecone

Para utilizar Pinecone antes debemos cumplir ciertos requisitos: tenemos que obtener una clave para su API y debemos instalar la SDK de *Python*. La clave nos permite conectar con el servicio web, permitiéndonos crear índices (ver figuras 40, 41 y 42) y enviar documentos (figuras 43 y 44).

```
1 from pinecone import Pinecone
2
3 pc = Pinecone(api_key="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx")
4 index_name = "developer-quickstart-py"
```

Figura 40: Pinecone. Inicialización a través de su clave API

```
6 ##### create index
7 if not pc.has_index(index_name):
8     pc.create_index_for_model(
9         name=index_name,
10        cloud="aws",
11        region="us-east-1",
12        embed={
13            "model": "llama-text-embed-v2",
14            "field_map": {"text": "chunk_text"}
15        }
16    )
```

Figura 41: Pinecone. Creación de un índice

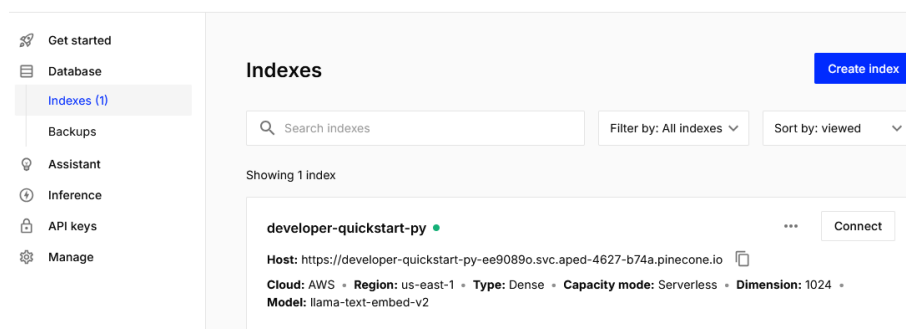


Figura 42: Pinecone. Vista del índice en la web

```
18 ##### Upsert data
19 index = pc.Index(index_name)
20
21 records = [
22     { "id": "rec1", "chunk_text": "The Eiffel Tower was completed in 1889 and stands in Paris, France.", "category":
23       "history" },
24     { "id": "rec2", "chunk_text": "Photosynthesis allows plants to convert sunlight into energy.", "category": "science" },
25     { "id": "rec3", "chunk_text": "Albert Einstein developed the theory of relativity.", "category": "science" },
26     { "id": "rec4", "chunk_text": "The mitochondrion is often called the powerhouse of the cell.", "category": "biology" },
27     { "id": "rec5", "chunk_text": "Shakespeare wrote many famous plays, including Hamlet and Macbeth.", "category":
28       "literature" },
29     { "id": "rec6", "chunk_text": "Water boils at 100°C under standard atmospheric pressure.", "category": "physics" },
30     { "id": "rec7", "chunk_text": "The Great Wall of China was built to protect against invasions.", "category": "history" },
31     { "id": "rec8", "chunk_text": "Honey never spoils due to its low moisture content and acidity.", "category": "food
32       science" },
33     { "id": "rec9", "chunk_text": "The speed of light in a vacuum is approximately 299,792 km/s.", "category": "physics" },
34     { "id": "rec10", "chunk_text": "Newton's laws describe the motion of objects.", "category": "physics" }
35 ]
36
37 index.upsert_records("ns1", records)
```

Figura 43: Pinecone. Agregación de documentos

Showing 10 hits

1	ID: rec10 category: "physics" chunk_text: "Newton's laws describe the motion of objects."	...
SCORE		
0.0613		
2	ID: rec6 category: "physics" chunk_text: "Water boils at 100°C under standard atmospheric pressure."	...
SCORE		
0.0509		
3	ID: rec9 category: "physics" chunk_text: "The speed of light in a vacuum is approximately 299,792 km/s."	...
SCORE		
0.0343		
4	ID: rec3 category: "science" chunk_text: "Albert Einstein developed the theory of relativity."	...
SCORE		
0.0325		

Figura 44: Pinecone. Vista de documentos en la web

Las búsquedas en Pinecone se pueden realizar como se muestra en la siguiente figura: especificando la consulta y los parámetros de recuperación deseados (ver figura 45). Pinecone ofrece también la posibilidad de hacer *rerank* a los resultados (véase la figura 46), es decir, un refinamiento o re-ordenación de los documentos recuperados basado en la relevancia ante una consulta.

```

37 ##### Semantic search
38 index = pc.Index(index_name)
39 query = "Famous historical structures and monuments"
40
41 results = index.search(
42     namespace="ns1",
43     query={
44         "top_k": 5,
45         "inputs": {
46             'text': query
47         }
48     }
49 )
50
51 print(results)

```

(a) Búsqueda semántica

```

{'result': {'hits': [{'_id': 'rec5',
  '_score': 0.18145012855529785,
  'fields': {'category': 'literature',
    'chunk_text': 'Shakespeare wrote many famous '
    'plays, including Hamlet and '
    'Macbeth.'}},
  {'_id': 'rec1',
  '_score': 0.08267835527658463,
  'fields': {'category': 'history',
    'chunk_text': 'The Eiffel Tower was completed '
    'in 1889 and stands in Paris, '
    'France.'}},
  {'_id': 'rec7',
  '_score': 0.0784618929028511,
  'fields': {'category': 'history',
    'chunk_text': 'The Great Wall of China was '
    'built to protect against '
    'invasions.'}},
  {'_id': 'rec3',
  '_score': 0.05581756681203842,
  'fields': {'category': 'science',
    'chunk_text': 'Albert Einstein developed the '
    'theory of relativity.'}},
  {'_id': 'rec4',
  '_score': 0.021570486947894096,
  'fields': {'category': 'biology',
    'chunk_text': 'The mitochondrion is often '
    'called the powerhouse of the '
    'cell.'}}]}}
'usage': {'embed_total_tokens': 8, 'read_units': 6}}

```

(b) Resultados

Figura 45: Pinecone. Búsqueda semántica y resultados

```

54 index = pc.Index(index_name)
55 query = "Famous historical structures and monuments"
56
57 reranked_results = index.search(
58     namespace="ns1",
59     query={
60         "top_k": 5,
61         "inputs": {
62             'text': query
63         }
64     },
65     rerank={
66         "model": "bge-reranker-v2-m3",
67         "top_n": 5,
68         "rank_fields": ["chunk_text"]
69     },
70     fields=["category", "chunk_text"]
71 )
72
73 print(reranked_results)

```

(a) Reranking

```

{'result': {'hits': [{'_id': 'rec1',
  '_score': 0.10650458931922913,
  'fields': {'category': 'history',
    'chunk_text': 'The Eiffel Tower was completed '
    'in 1889 and stands in Paris, '
    'France.'}},
  {'_id': 'rec7',
  '_score': 0.06278928918703079,
  'fields': {'category': 'history',
    'chunk_text': 'The Great Wall of China was '
    'built to protect against '
    'invasions.'}},
  {'_id': 'rec5',
  '_score': 3.21923362287599e-05,
  'fields': {'category': 'literature',
    'chunk_text': 'Shakespeare wrote many famous '
    'plays, including Hamlet and '
    'Macbeth.'}},
  {'_id': 'rec4',
  '_score': 1.6187581422855146e-05,
  'fields': {'category': 'biology',
    'chunk_text': 'The mitochondrion is often '
    'called the powerhouse of the '
    'cell.'}},
  {'_id': 'rec3',
  '_score': 1.5936620911816135e-05,
  'fields': {'category': 'science',
    'chunk_text': 'Albert Einstein developed the '
    'theory of relativity.'}}]}}
'usage': {'embed_total_tokens': 8, 'read_units': 6, 'rerank_units': 1}}

```

(b) Resultados

Figura 46: Pinecone. Reranking y resultados

Se puede observar una clara diferencia entre los resultados de las dos últimas figuras, siendo los obtenidos aplicando *reranking* mucho más relevantes respecto a la consulta hecha.

Conclusión sobre Pinecone

Tras analizar el funcionamiento, características y diferencias entre ChromaDB y Pinecone, se puede concluir que ChromaDB representa una opción superior para muchos escenarios de desarrollo, especialmente aquellos en los que la privacidad, el control total sobre los datos y la independencia de servicios externos son prioritarios.

La principal ventaja de ChromaDB reside en su capacidad para operar de manera local, sin necesidad de conexión a la nube. Esta característica permite que la información sensible se mantenga en el entorno del usuario, eliminando preocupaciones relacionadas con la exposición de datos o la dependencia de terceros. Además, su instalación sencilla, su compatibilidad con múltiples lenguajes y herramientas, y su naturaleza de código abierto lo

convierten en una solución robusta y versátil para sistemas de recuperación de información basados en embeddings.

Si bien Pinecone ofrece una infraestructura escalable y tiempos de respuesta rápidos, su condición de servicio en la nube introduce limitaciones importantes en términos de coste, privacidad y flexibilidad. Por tanto, para proyectos que requieran control local, transparencia y personalización, ChromaDB se posiciona como la alternativa preferente.

2.7. LangChain

LangChain [36] se ha convertido en uno de los proyectos de código abierto con más rápido crecimiento en la historia, debido en gran medida al creciente interés surgido en el ámbito de los LLMs [37]. LangChain es un framework centrado en el desarrollo de aplicaciones impulsadas por los grandes modelos de lenguaje, cubriendo cada etapa de la vida de la aplicación: desde el desarrollo usando componentes de código abierto de LangChain o integraciones de terceros, la producción con la herramienta inspectora *LangSmith*, y el despliegue a través de *LangGraph*.

LangChain implementa una interfaz estandar para los LLMs y tecnologías relacionadas, como son los modelos de embedding y las bases de datos vectoriales, y esta integrado con cientos de proveedores.

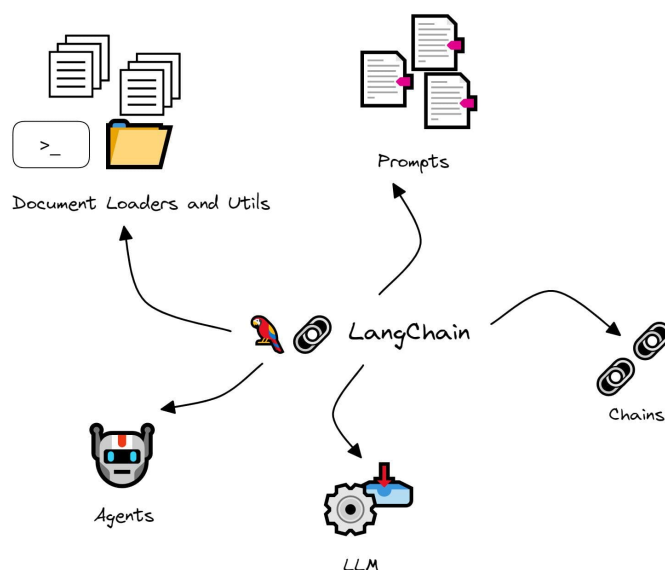


Figura 47: Esquema que muestra las diferentes aplicaciones en las que podemos usar LangChain [38]

La importancia de LangChain radica en un conjunto de factores. El primero de ellos es que simplifica el proceso de desarrollo, reduciendo la complejidad a la hora de construir aplicaciones LLM por medio de plantillas estructuradas. Esto permite a los desarrolladores hacer las aplicaciones de manera modular, haciendo que sean más fáciles de manejar y escalar.

A parte, LangChain permite la integración con múltiples LLMs, permitiéndonos usar el

que más nos apetezca. Por último, el sistema de “*chaining*” (o encadenamiento) ofrece una forma eficiente de llamar a los modelos, lo que incrementa el rendimiento y la precisión de la aplicación [38].

2.7.1. Características e integraciones de LangChain

Aunque ya hemos comentado algunas de las características o integraciones de LangChain, a continuación comentaremos algunas de las más importantes que tiene.

Características

- **Diseño modular:** Divide el proceso de desarrollo en módulos manejables, cada uno, asumiendo un aspecto específico de la aplicación.
- **Manejo del prompt:** Provee de herramientas para crear y manejar prompts de manera eficiente.
- **Chaining:** Permite enlazar múltiples prompts y modelos para crear flujos de trabajo complejos.
- **Manejo del output:** Facilita el post-procesado de la salida de los modelos proporcionando una mejor usabilidad.

Integraciones

- **OpenAI GPT:** Una de las librerías de LLMs más usadas para el procesamiento del lenguaje natural.
- **Mistral AI:** Un LLM desarrollado por la empresa del mismo nombre, diseñada para proporcionar información precisa y asistencia en una variedad de temas.
- **Ollama:** Una biblioteca donde podemos encontrar una gran cantidad de LLMs de libre acceso.
- **Hugging Face Transformers:** Una librería que da acceso a un gran número de modelos pre-entrenados.
- **ChromaDB:** La base de datos vectorial usada en el proyecto.

2.7.2. Tutorial básico de LangChain

Las siguientes figuras muestran una pequeña aplicación hecha usando LangChain y el resultado de su ejecución. En la figura 48 podemos ver el diseño modular y las integraciones antes mencionadas que tiene esta herramienta. Podemos observar también el diseño sencillo y sin complicaciones que ofrece a la hora de manejar prompts e integraciones con LLMs.

```
1 from langchain_mistralai import ChatMistralAI
2 from langchain_core.prompts import ChatPromptTemplate
3
4 MISTRAL_API_KEY = '...'
5
6 llm = ChatMistralAI(
7     model="mistral-large-latest",
8     api_key=MISTRAL_API_KEY
9 )
10
11 prompt = "Translate the following {text} from English into {language}"
12
13 prompt_template = ChatPromptTemplate.from_template(prompt)
14
15 chain = prompt_template | llm
16
17 language = "Italian"
18 text = "Hi, I'm developing a translation app. Can you help me translate this text?"
19
20 for token in chain.stream({"language": language, "text": text}):
21     print(token.content, end="", flush=True)
```

Figura 48: LangChain. Aplicación sencilla que usa mistral-large (LLM desarrollado por MistralAI) para traducir un texto

```
Sure, here is the translation of your text from English into Italian:
"Ciao, sto sviluppando un'app di traduzione. Puoi aiutarmi a tradurre questo testo?"
```

Figura 49: LangChain. Resultado de la traducción del texto de la figura 48

2.7.3. Alternativas: LlamaIndex

LlamaIndex [39] es la principal alternativa que tenemos a LangChain. LlamaIndex es un framework que simplifica la integración de datos públicos y privados en la construcción de aplicaciones usando grandes modelos de lenguaje [40]. Provee de funcionalidades para ingestar datos, indexación de datos, herramientas de búsqueda, etc, por lo tanto, es una solución muy versátil para construir un RAG.

Funcionalidades principales y beneficios de LlamaIndex

El modo de operar de LlamaIndex se puede dividir en tres grandes bloques: ingestión, indexado y consulta [41].

En la ingesta de datos, LlamaIndex simplifica el proceso de conectar con datos externos, desde APIs, PDFs o bases de datos. Esto asegura que datos con o sin estructura puedan ser integrados en aplicaciones LLM.

Una vez los datos son ingestados, LlamaIndex dispone de múltiples modelos de indexación para representar datos de manera eficiente. Algunas de estas herramientas son:

- **List Index:** Organizan los datos secuencialmente, apto para objetos estructurados que cambian en el tiempo.
- **Tree Index:** Estructura los datos en forma de árbol.
- **Vector Store Index:** Almacena los datos provenientes de un modelo de embedding, facilitando al vector hacer búsquedas por similitud.
- **keyword Index:** Asocia los metadatos a los datos almacenados.

Por último, a la hora de consultar, LlamaIndex aprovecha el procesamiento del lenguaje natural que posee para facilitar dichas consultas a través de un prompt. Este enfoque permite a los usuarios interactuar con los datos usando su propio lenguaje natural, simplificando el proceso de recuperación de información

Diferencias en el código: LangChain vs LlamaIndex

A continuación mostraremos algunas figuras comparativas, donde veremos el código de LangChain y LlamaIndex para hacer una misma función.

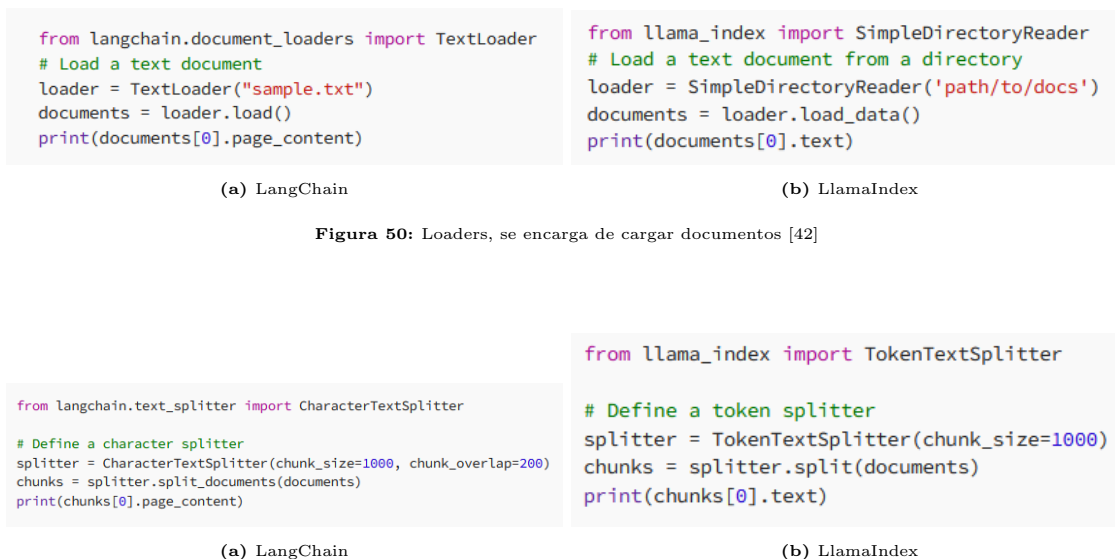


Figura 50: Loaders, se encarga de cargar documentos [42]



Figura 51: Splitter, se encarga de trocear los documentos [42]

Podemos apreciar en las figuras 50 y 51, que la forma que tienen ambas herramientas de cargar y separar documentos es similar, usando sendas funciones con las que obtienen el mismo resultado.

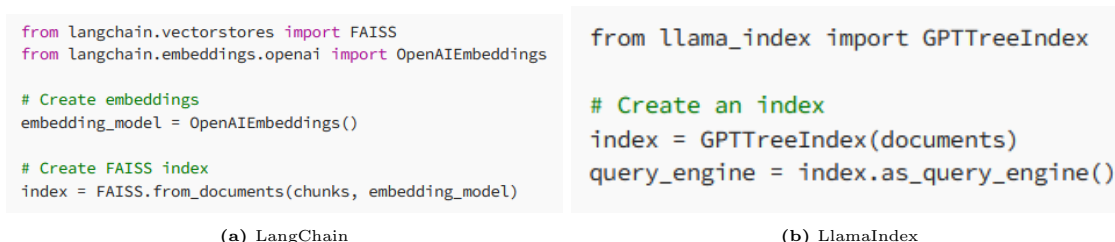
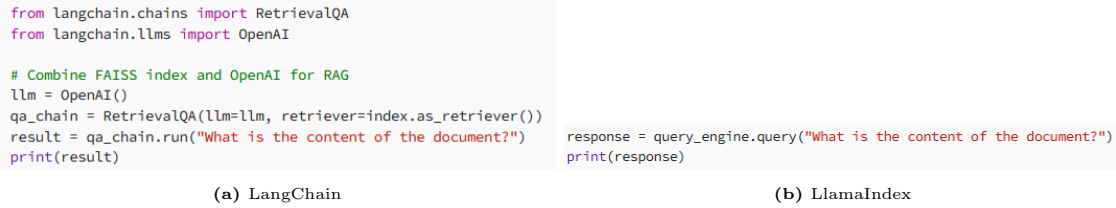


Figura 52: Indexing, se encarga de almacenar los documentos en la base de datos vectorial [42]

En cambio, la figura anterior muestra una diferencia. Por un lado, LangChain muestra su capacidad de integración personalizando los modelos de embedding y la base de datos a usar, mientras que LlamaIndex simplifica este proceso creando una estructura jerárquica y usando valores por defecto.



```
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

# Combine FAISS index and OpenAI for RAG
llm = OpenAI()
qa_chain = RetrievalQA(llm=llm, retriever=index.as_retriever())
result = qa_chain.run("What is the content of the document?")
print(result)
```

(a) LangChain

```
response = query_engine.query("What is the content of the document?")
print(response)
```

(b) LlamaIndex

Figura 53: Chains/Query, se encarga de hacer una consulta a la base de datos vectorial [42]

A la hora de hacer consultas pasa algo similar a lo visto en la figura 52. La figura 53 muestra la forma de hacer una consulta con ambas herramientas, siendo mas simple la manera que ofrece LlamaIndex.

Conclusión: LangChain vs LlamaIndex

Tras evaluar ambas herramientas, se ha determinado que LangChain es la opción más adecuada para el desarrollo de aplicaciones avanzadas basadas en LLMs.

LangChain destaca por su excepcional flexibilidad y capacidad de integración, permitiendo un control granular sobre cada componente del flujo de trabajo. Su capacidad para manejar flujos de trabajo multi-modales y su soporte para una amplia gama de modelos y bases de datos vectoriales lo posiciona como una herramienta poderosa para aplicaciones que demandan un alto nivel de personalización. Además, ofrece una interfaz estandarizada, que facilita la integración con multitud de proveedores y simplifica el proceso de desarrollo.

Aunque LlamaIndex ofrece una solución más sencilla y directa, especialmente adecuada para proyectos que se centran en la recuperación y el procesamiento de documentos estructurados de manera jerárquica, su enfoque simplificado y facilidad de configuración no son suficientes para proyectos que requieren un alto nivel de personalización y control. LangChain, por otro lado, proporciona una infraestructura robusta y versátil que puede adaptarse a una amplia variedad de necesidades y complejidades del proyecto.

Por lo tanto, LangChain se presenta como la opción superior para aquellos que buscan una solución altamente personalizable y con capacidad de manejar múltiples tipos de datos y estrategias de recuperación complejas. Su capacidad para integrarse con una amplia variedad de modelos y sistemas de almacenamiento, junto con su diseño modular y herramientas avanzadas para el manejo de prompts y outputs, lo convierten en la elección ideal para el desarrollo de aplicaciones avanzadas basadas en LLMs.

2.8. Streamlit

Streamlit [43] es un framework de código abierto en Python pensado para desarrollar aplicaciones interactivas basadas en el aprendizaje automático, inteligencia artificial o ciencia de datos. Streamlit permite a los desarrolladores y científicos de datos crear aplicaciones atractivas, informativas o realizar visualizaciones de datos en pocas líneas de código.

Una de las principales aspectos de Streamlit es su simpleza y facilidad de aprendizaje. Además, Streamlit dispone de una amplia gama de herramientas o *widgets* de visualización

de datos: gráficas, histogramas, mapas, etc [44].

Otra ventaja de Streamlit es su flexibilidad, permitiendo desarrollar aplicaciones para un gran rango de casos de uso: desde simples código para visualizar datos, hasta aplicaciones complejas basadas en modelos de lenguaje.

2.8.1. Funcionalidades clave de Streamlit

Ya hemos mencionado algunos de sus aspectos más importantes, en la siguiente lista mostraremos los aspectos clave que hacen de Streamlit una de las herramientas más potentes [45]:

- **Desarrollo centrado en Python:** En contraste con los frameworks web tradicionales, esta herramienta usa Python. Elimina el requerimiento de usar lenguajes orientados a la web.
- **Widgets interactivos:** Como ya hemos comentado, Streamlit dispone de *widgets* propios incluyendo botones, paneles desplegados, etc; que proporcionan más interactividad.
- **Actualizaciones en tiempo real:** Las aplicaciones Streamlit ejecutan el código de Python de arriba hacia abajo, actualizándose automáticamente cada vez que el usuario interactúa con un *widget* o modifica una entrada de datos.
- **Soporte a la visualización de datos:** Streamlit está integrado con las librerías de visualización de datos más comunes, incluyendo *Seaborn* o *Plotly*.
- **Prototipado rápido:** Streamlit posee una sintaxis sencilla, posibilitando un desarrollo de prototipos ágil y perfeccionamiento posterior.
- **De código abierto y extensible:** Al tener este enfoque, Streamlit confiere a los usuarios la posibilidad de extender su funcionalidad a través de plugins o integraciones hechos por la comunidad.
- **Opciones de despliegue:** Las aplicaciones desarrolladas en Streamlit pueden ser desplegadas en distintas plataformas como: Streamlit Community Cloud, Heroku, o AWS.

2.8.2. Ejemplo de aplicación LLM en Streamlit

El siguiente ejemplo muestra una manera sencilla de integrar un LLM en Streamlit, usando varias librerías de LangChain, así como integraciones para los LLMs de Mistral AI y funcionalidades propias de Streamlit.

En el caso de uso escogido para el ejemplo, simularemos una conversación con un agente IA que es un asistente de venta de vehículos. Para ello necesitaremos configurar un prompt donde recogeremos las instrucciones que deberá seguir el LLM en su respuesta. La siguiente figura muestra el prompt usado en la aplicación, y además, cuenta con dos variables, la consulta del usuario y el historial de la conversación. De esta manera podemos lograr que nuestro agente tenga memoria, lo que es muy útil en casos como este.

```
21 # Contexto, tambien agrega el historial de conversacion
22 CHAT_PROMPT_TEMPLATE = """
23     Eres un asistente profesional de venta de coches.
24     Tu meta es ayudar a los clientes a encontrar el coche perfecto basado en sus necesidades y
25     preferencias.
26     Responde a las preguntas teniendo en cuenta el historial de la conversaci3n, sin mencionarlo.
27     -----
28     Chat history: {chat_history}
29     User question: {user_question}
30 """
```

Figura 54: Ejemplo aplicaci3n LLM en Streamlit. Prompt con dos variables

Como hemos mencionado anteriormente, para este ejemplo hemos usado un LLM perteneciente a Mistral AI, en concreto, “mistral-small”, dado a que es bastante r3pido y eficiente. La figura 55 muestra como se hace la llamada al LLM usando las herramientas que nos facilita LangChain, acabando retornando la “chain” o cadena en la que se escribir3 la respuesta.

```
36 # funcion para obtener respuesta
37 def get_response(user_query, chat_history):
38
39     llm = ChatMistralAI(
40         model_name="mistral-small-latest",
41         api_key=" ",
42         temperature=0.7
43     )
44     prompt = ChatPromptTemplate.from_template(CHAT_PROMPT_TEMPLATE)
45     chain = prompt | llm | StrOutputParser()
46
47     return chain.stream([{"chat_history": chat_history, "user_question": user_query}])
48
```

Figura 55: Ejemplo aplicaci3n LLM en Streamlit. Llamada al LLM

El programa principal reflejado en la figura 56, no requiere de m3s de 20 l3neas. Si nos fijamos en la l3nea 58, podemos observar como Streamlit gestiona las sesiones de estado para guardar el historial de conversaci3n. Estas son variables especiales donde podemos guardar el valor de cualquier variable o clase.

La consulta de usuario se guarda f3cilmente mediante el m3todo *chat.input* de Streamlit, que crea en la interfaz un input para la entrada de texto. En las l3neas 70 y 75 de figura 56 podemos ver como se agregan la consulta y la respuesta del modelo a la variable que guarda el historial de conversaci3n, mientras que en la 74, se hace una llamada a la funci3n que retorna la respuesta del modelo. En concreto, el m3todo *write_stream* de Streamlit escribe la respuesta en tiempo real seg3n van llegando los *tokens* del modelo de lenguaje, proporcionando una respuesta inmediata y una mejor experiencia de usuario.

```

54 st.set_page_config(page_title="DriveNet", page_icon="🚗")
55 st.markdown("<h1>¡Bienvenido a DriveNet! 🚗🗣️</h1>", unsafe_allow_html=True)
56 st.markdown("<h2>Encuentra el coche de tus sueños con la ayuda de Carina, tu asistente virtual.</h2>",
57            unsafe_allow_html=True)
58
59 if "chat_history" not in st.session_state:
60     st.session_state.chat_history = [AIMessage(content=random.choice(WELCOME_MESSAGES))]
61
62 for message in st.session_state.chat_history:
63     if isinstance(message, AIMessage):
64         with st.chat_message("AI"):
65             st.write(message.content)
66     elif isinstance(message, HumanMessage):
67         with st.chat_message("Human"):
68             st.write(message.content)
69
70 user_query = st.chat_input("Escribe tu mensaje aquí ...")
71 if user_query:
72     st.session_state.chat_history.append(HumanMessage(content=user_query))
73     with st.chat_message("Human"):
74         st.markdown(user_query)
75     with st.chat_message("AI"):
76         response = st.write_stream(get_response(user_query, st.session_state.chat_history))
77     st.session_state.chat_history.append(AIMessage(content=response))

```

Figura 56: Ejemplo aplicación LLM en Streamlit. Programa principal

En resumen, Streamlit se caracteriza por la sencillez de su código y por la integración que permite con otras herramientas como LangChain, etc. La siguiente subsección, muestra algunas capturas del funcionamiento de este ejemplo.

Funcionamiento del ejemplo



Figura 57: Interfaz del ejemplo aplicación LLM en Streamlit. Bienvenida

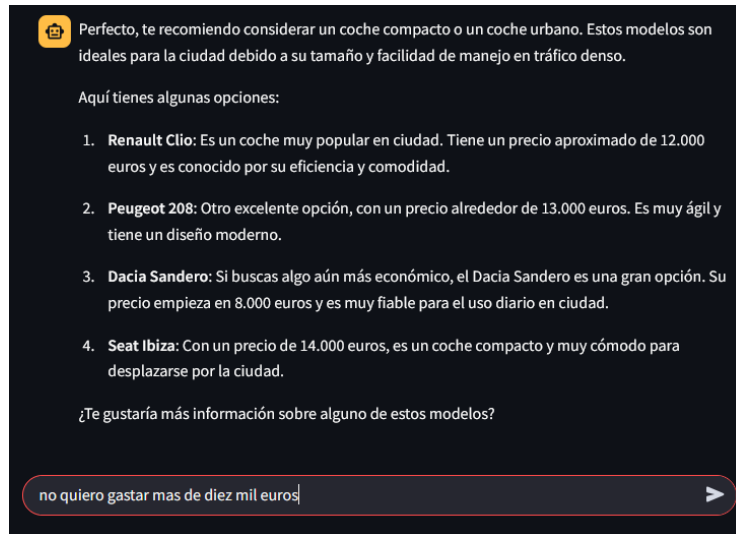


Figura 58: Interfaz del ejemplo aplicación LLM en Streamlit. Opciones

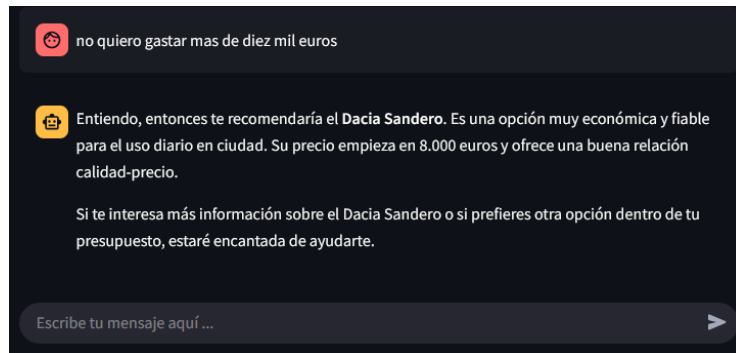


Figura 59: Interfaz del ejemplo aplicación LLM en Streamlit. Respuesta con memoria

2.9. Aplicaciones alternativas al proyecto propuesto

En esta sección hablaremos de aplicaciones o servicios similares a nuestro proyecto, aplicaciones de ayuda a ciudadanos ante una catástrofe natural, como la DANA; o servicios que usen de base un RAG para facilitar el acceso a la información.

2.9.1. Ayuda Dana

Ayuda Dana [46] es una plataforma digital colaborativa creada tras la DANA en Valencia para coordinar de forma centralizada la ayuda a los afectados. Nació el 31 de octubre de 2024 a partir de la unión de iniciativas espontáneas en redes sociales, y que reunió a voluntarios de diferentes perfiles (informáticos, logísticos, atención al cliente, etc.), organizados como si fueran una empresa, pero sin apoyo oficial [47].

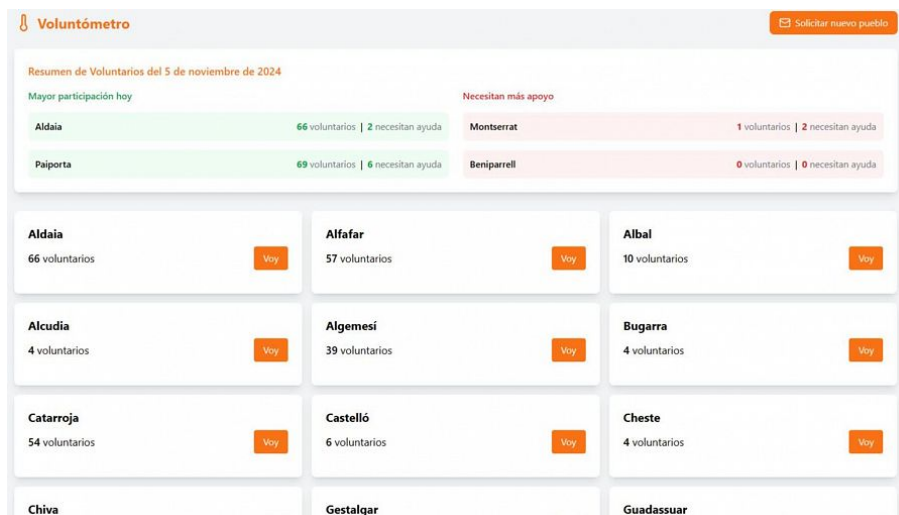


Figura 60: El voluntómetro muestra las solicitudes de ayuda y los voluntarios que hay en cada lugar [47]

La aplicación conecta solicitudes de ayuda con ofertas de voluntariado, gestionando peticiones y ofrecimientos en tareas como limpieza, alojamiento, asistencia médica, apoyo psicológico o transporte. Además, integra un servicio de búsqueda de desaparecidos, con el que han logrado localizar a 20 personas [47].

Entre sus funcionalidades destacan:

- Registro de solicitudes con niveles de urgencia (rojo, naranja, verde).
- Mapa de puntos de recogida y distribución de suministros.
- Teléfono gratuito para personas mayores sin acceso a internet.
- Validación de información para evitar datos falsos.

Ayuda Dana se ha convertido en una herramienta clave de coordinación ciudadana, combinando tecnología, voluntariado y solidaridad para dar respuesta rápida y eficaz a una emergencia colectiva.

2.9.2. TuCocheDana

TuCocheDANA era una plataforma digital creada durante la DANA para ayudar a propietarios a recuperar sus vehículos perdidos tras las riadas. La iniciativa fue desarrollada por Juan Francisco Soler, profesor de informática en el IES Alyanub de Almería, y su exalumno René Molina, estudiante de Ingeniería Mecánica en la UPV [48].

Su funcionamiento era sencillo:

- Si una persona encontraba un coche abandonado, podía registrarlo indicando matrícula, ubicación, marca, color y detalles adicionales.
- Si otra persona buscaba su coche perdido, bastaba introducir la matrícula en el buscador para obtener información y la localización exacta.

La plataforma garantizaba la privacidad no proporcionando un listado abierto de coches encontrados, tan solo mostraba los datos vinculados a la matrícula introducida. Además, a través de un bot de Telegram, avisaba automáticamente a los propietarios cuando su

coche fuese registrado, evitando así que tuviesen que comprobar la web de forma constante [48].

La plataforma funcionaba a través de la web *tucochedana.es*, que actualmente esta cerrada debido al tiempo transcurrido desde la tragedia.

En resumen, TuCocheDANA fué un ejemplo de innovación digital solidaria, que combinó tecnología, colaboración ciudadana y gestión responsable de datos para resolver un problema muy concreto tras la catástrofe.

2.9.3. Justicio

Justicio [49] es una plataforma de código abierto que utiliza un enfoque RAG, combinando la API de *OpenAI* con la base de datos vectorial *Qdrant*. Gracias a esta arquitectura, es capaz de entender consultas en lenguaje natural, recuperar información legislativa relevante y generar respuestas claras y fundamentadas, citando siempre las fuentes originales [50].

Inicialmente concebido como un chatbot jurídico, Justicio se entrenó con legislación estatal, autonómica y europea, permitiendo a cualquier persona —desde juristas hasta ciudadanos sin formación legal— acceder de manera rápida y comprensible a normativas complejas. Su valor diferencial radica en transformar el BOE y otros textos legales densos y poco accesibles en respuestas útiles y contextualizadas [51].

En 2025, la plataforma dio un paso más al lanzar un servicio gratuito que ofrece el acceso íntegro al BOE con una interfaz visual, estructurada y amigable (ver figura 61). Este nuevo sistema incluye:

- Un dashboard inicial con resúmenes y datos clave diarios.
- Secciones reorganizadas de manera comprensible y navegable.
- Herramientas para descargar, exportar y trabajar directamente con los documentos.
- Una doble modalidad de consumo: detallada o simplificada.



Figura 61: Interfaz de Justicio en el BOE del día 3 de septiembre de 2025 [49]

Con esta evolución, Justicio no solo se ha consolidado como un ejemplo de aplicación práctica de RAG en el ámbito jurídico, sino que también ha logrado democratizar el acceso a la legislación española, transformando un recurso indispensable pero históricamente poco usable como el BOE en una herramienta realmente útil para profesionales y ciudadanos.

2.9.4. Conclusión sobre las aplicaciones alternativas

En síntesis, proyectos como *Ajuda Dana* y *TuCocheDana* evidencian la capacidad de la tecnología para ofrecer soluciones útiles en situaciones de emergencia, mientras que *Justicio* demuestra la aplicabilidad del enfoque RAG para hacer más accesible e interpretable la información contenida en el BOE.

En este contexto, el presente trabajo se sitúa en la misma línea, al proponer un sistema

RAG aplicado a la documentación del BOE relativa a la DANA, con el fin de mejorar su comprensión y convertir un recurso jurídico denso en una herramienta práctica para la ciudadanía en momentos de crisis.

2.10. Métodos de evaluación de nuestro RAG

En esta sección se describen las distintas técnicas y configuraciones que utilizaremos para evaluar nuestro sistema RAG. La evaluación se llevará a cabo mediante la combinación de diferentes procesos que afectan tanto a la recuperación de información como a la generación de respuestas. Para finalizar, evaluaremos diversas métricas con las que obtendremos el valor de calidad de los resultados.

El conjunto de datos utilizado se almacena en forma de documentos en una base de datos vectorial **Chroma**, donde cada documento está pre-procesado y segmentado en fragmentos de **512** y **1024** términos respectivamente. Esta decisión se ha tomado para evaluar la granularidad del contenido indexado y la relevancia de los documentos recuperados (enfoque *Naive RAG*) [8], así como la calidad del contexto aportado al modelo generativo.

Paralelamente, se ha evaluado el impacto del **re-ranking** (enfoque *Advanced RAG*) [8] como paso posterior a la recuperación. Este proceso consiste en el reordenamiento de los documentos recuperados en primera instancia a través de un modelo de recuperación adicional. En nuestro caso usaremos el modelo **BAAI/bge-reranker-large** disponible en **HuggingFace** [52], que soporta documentos de más longitud, escritos en más idiomas (a parte del inglés) y a priori, logrará mejores resultados.

En las siguientes sub-secciones, listaremos el resto de técnicas usadas para la evaluación del sistema RAG.

2.10.1. Estrategias de búsqueda usadas

Se han considerado varias estrategias de búsqueda a la hora de recuperar los documentos más relevantes de la base de datos:

- **Similarity**: búsqueda por similitud mediante la distancia coseno en los embeddings.
- **Maximal Marginal Relevance (MMR)**: método usado para recuperar documentos que son a la vez relevante para consulta y diversos entre sí. Este enfoque ayuda a reducir la redundancia e incrementa los aspectos que son cubiertos en los documentos seleccionados [53].
- **TF-IDF y BM25**: métodos clásicos basados en la frecuencia de los términos. TF-IDF se basa en la frecuencia y rareza de los términos del corpus, mientras que BM25 ajusta estos valores dependiendo del tamaño del documento.
- **Grafo**: técnica que combina la búsqueda por similitud con relaciones semánticas obtenidas a través de los metadatos de los documentos [54].

2.10.2. Embeddings usados

Los embeddings utilizados para indexar y recuperar documentos incluyen los siguientes modelos pre-entrenados:

- **nomic-embed-text**: Modelo de código abierto entrenado en inglés. Es útil para clasificación, búsqueda y análisis semántico.
- **bge-m3**: Modelo multilingüe con más de cien idiomas. Optimizado para recuperación densa, escasa y multivectorial.
- **snowflake-artic-embed2**: Evolución del modelo anterior de Snowflake, con rendimiento similar a bge-m3. Es también multilingüe.
- **jina-embeddings-v2-base-es**: Bilingüe español-inglés, ideal para realizar tareas en español: búsqueda, clasificación, etc.
- **text-embedding-3-small**: Versión mejorada del modelo anterior *text-embedding-ada-002*, modelos pertenecientes a **OpenAI**.

Todos estos modelos están disponibles mediante la plataforma **Ollama**, exceptuando el último.

2.10.3. LLMs usados

Los modelos de lenguaje empleados son los siguientes.

- **llama3.2**: Modelo perteneciente a Meta de 3B de parámetros. Proporcionado localmente y de forma gratuita por Ollama.
- **mistral-small**: Versión 3.2 perteneciente a Mistral AI, cuenta con 24B de parámetros. Usado a través de la API gratuita de Mistral AI.
- **gpt-4.1**: Nuevo modelo perteneciente a OpenAI. De pago y disponible a través de clave API de OpenAI.

2.10.4. Métricas de evaluación usadas

Para finalizar, presentaremos las métricas que se encargarán de medir la calidad de las respuestas de nuestro sistema RAG:

- **Accuracy**: Evalúa la precisión de la respuesta generada en comparación a la respuesta esperada.
- **Faithfulness**: Evalúa si la respuesta usa la información de los documentos recuperados.
- **Groundness**: Evalúa la capacidad del modelo de apoyar sus respuestas en las evidencias de los documentos recuperados.
- **Relevance**: Evalúa la relevancia de los documentos recuperados para responder a la pregunta.

Estas métricas evaluarán nuestro RAG a través de un dataset de preguntas generado a partir de los documentos de la fuente de conocimiento.

Capítulo 3 : Desarrollo del Proyecto

En este capítulo se describe el proceso de diseño, implementación y experimentación llevado a cabo para construir un sistema de generación aumentada por recuperación sobre un dominio específico, utilizando como fuente principal documentos del *Boletín Oficial de Estado* (BOE) relacionados con la **DANA** ocurrida a finales del pasado año.

La propuesta principal del proyecto es proveer de información veraz y actualizada a cualquier persona interesada en lo sucedido, por eso es fundamental la creación de un buen corpus que sirva como base para responder a nuestro sistema RAG.

Por último, exploramos como distintas configuraciones del sistema, desde modelos de embeddings hasta las estrategias de búsqueda y generación (LLMs), afectan a la calidad y fiabilidad de las respuestas generadas. Los resultados provenientes de esta experimentación serán mostrados en el capítulo 4.

3.1. Definición de la arquitectura

La arquitectura planteada para nuestro sistema RAG, es una combinación típica que se suele usar en la construcción de estos sistemas. Como se puede observar en la figura 62, el proceso para obtener una salida del RAG pasa por varias etapas.

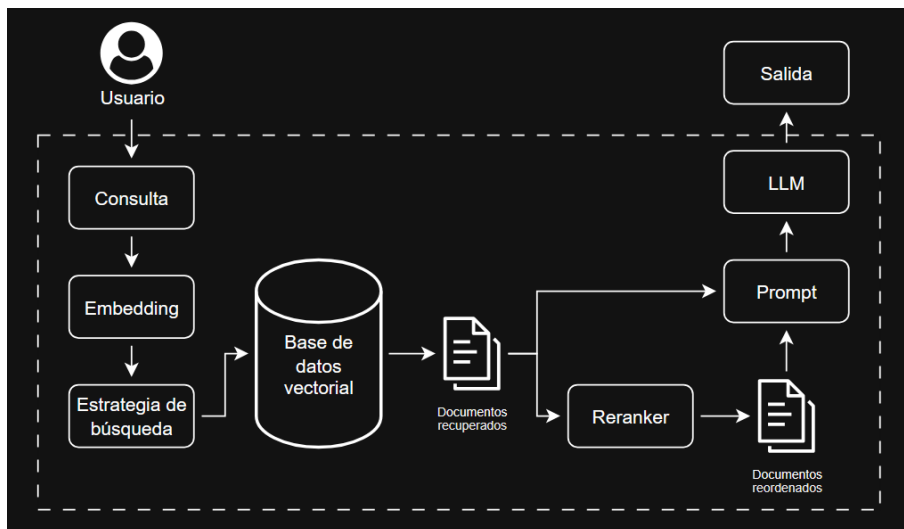


Figura 62: Arquitectura de nuestro sistema RAG

La consulta de usuario es vectorizada con el mismo modelo de embedding que hemos usado para la construcción de la base de datos vectorial para una mejor integridad de la respuesta, después es lanzada a la base de datos vectorial junto con la estrategia de búsqueda deseada.

Los documentos recuperados son agregados al prompt para generar la respuesta usando el LLM que el usuario indique. Otra opción disponible en nuestro sistema, es la de usar un

modelo de *reranking* que filtre los documentos recuperados y seleccione los mejores para proporcionárselos de la misma manera al LLM mediante el prompt.

La salida generada mostrará la información necesaria para responder a la entrada del usuario teniendo en cuenta el contenido relevante aportado por los documentos devueltos por el modelo de recuperación de información.

3.2. Planificación y presupuesto

En esta sección explicaremos la planificación seguida en el proyecto, la metodología usada y todas las fases por las que pasamos durante el proceso de desarrollo de nuestra aplicación. Terminaremos la sección planteando el presupuesto estimado del proyecto, basándonos en los días transcurridos y en los integrantes pertenecientes al equipo de desarrollo.

3.2.1. Metodología seguida: Scrum

El proyecto se planifico desde un principio siguiendo el enfoque de las metodologías ágiles o **Scrum**. Este es un proceso en el que se aplican de manera regular un conjunto de buenas prácticas para trabajar colaborativamente, en equipo, y obtener el mejor resultado posible de un proyecto [55]. Aunque el ámbito de este Trabajo de Fin de Máster es individual, este proceso se sigue en la gran mayoría de empresas del sector de las tecnologías de la información, por lo que es interesante explorar su uso en el desarrollo de este proyecto.

Para llevar a cabo este seguimiento se propuso el uso de la herramienta *Jira*, la cuál ofrece plantillas para crear la planificación de un proyecto siguiendo la metodología Scrum (figura 63). Además, esta herramienta permite la conexión con repositorios de *GitHub* (figura 64) y asignar ramas de nuestro repositorio a las tareas contenidas en los tableros del sprint, de esta manera podemos mantener un cierto seguimiento en el avance del desarrollo en ambas plataformas.

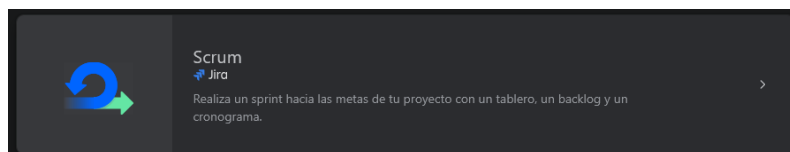


Figura 63: Jira. Plantilla Scrum para el proyecto



Figura 64: Jira. Conexión con la plataforma de control de versiones GitHub

En Scrum, un proyecto se ejecuta en ciclos temporales cortos y de duración fija llamados *sprints*. En nuestro caso, estas iteraciones variaron entre 1, 2 o hasta 4 semanas dependiendo de la carga de trabajo o las tareas correspondientes al sprint. Cada sprint culminaba en una reunión presencial o vía telemática. donde se anunciaban los progresos alcanzados en el sprint, y se planeaban los objetivos para el siguiente.

La figura 65 muestra la pestaña de *Backlog*. Esta pestaña muestra en la parte superior todas las tareas que se tienen que cumplir en el sprint actual; y en la inferior, las tareas pendientes para futuros sprints.

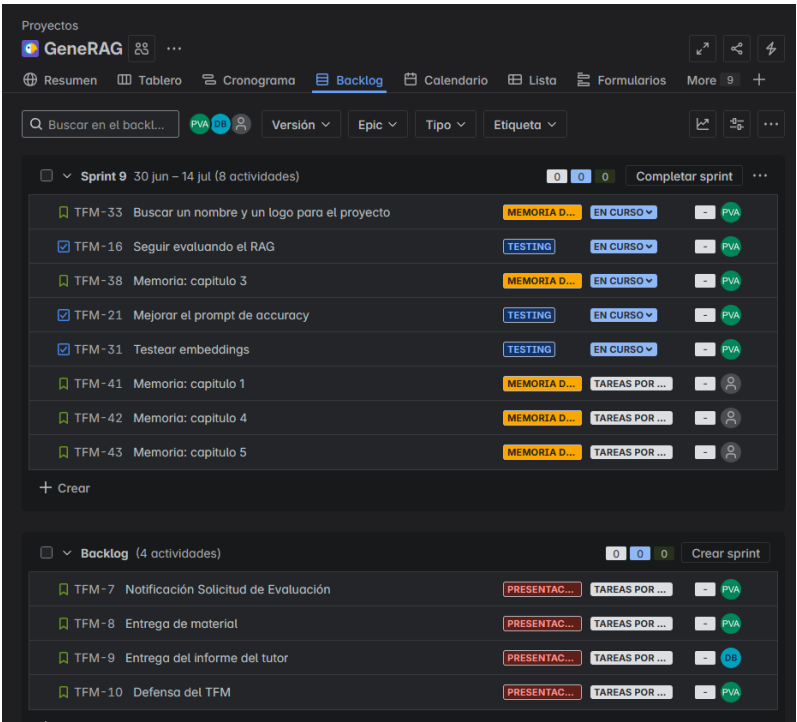


Figura 65: Jira. Pestaña Backlog, tareas del sprint 9 y pendientes [56]

Una vez iniciado el sprint, en la pestaña *Tablero* (figura 66) podemos ver las tareas asignadas anteriormente en el backlog y las columnas “POR HACER”, “EN CURSO”, “EN REVISIÓN”, y “LISTO”, columnas que podemos editar y añadir más. La lógica de estas columnas es mantener un control sobre que tarea se esta haciendo en este momento y quien es el encargado de ella. Cada tarea va avanzando hacia la columna de su derecha donde será evaluada hasta llegar a ”LISTO”, cuando podremos considerar la tarea acabada.

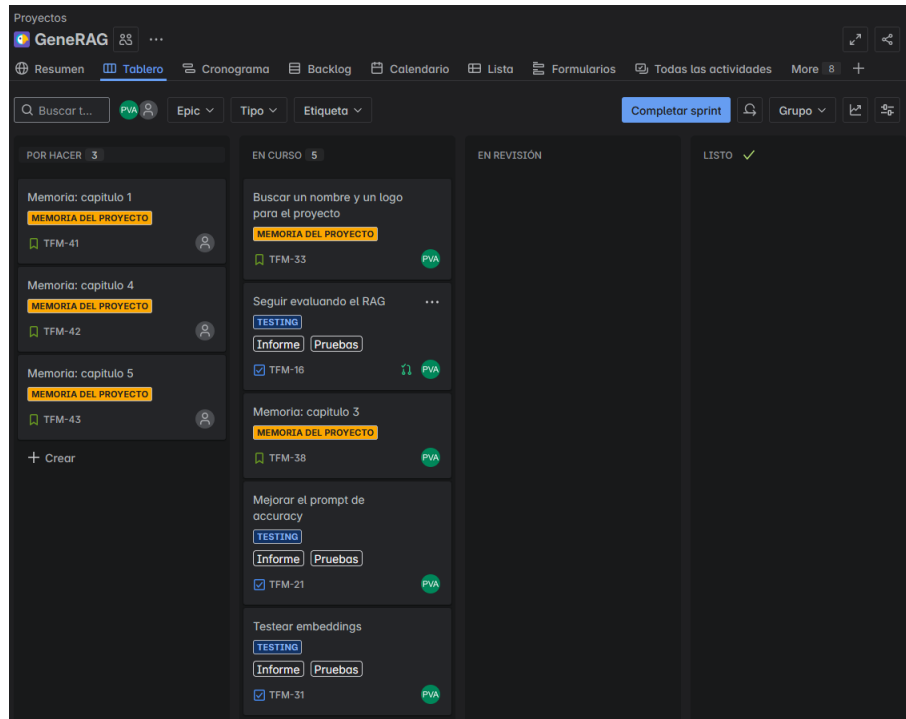


Figura 66: Jira. Tablero con tareas

Por último, podemos ver el progreso en el tiempo de nuestro proyecto en la pestaña *Cronograma* [56], el cual será explicado en la siguiente sección.

3.2.2. Fases del proyecto

Las siguientes subsecciones explican brevemente lo realizado durante las distintas fases del proyecto, desde el inicio de la idea para el proyecto hasta su conclusión y entrega [56]. En la figura 67, observamos las fases por las que pasó el proyecto, su duración y los sprints totales. La duración total del proyecto ha sido de 204 días.

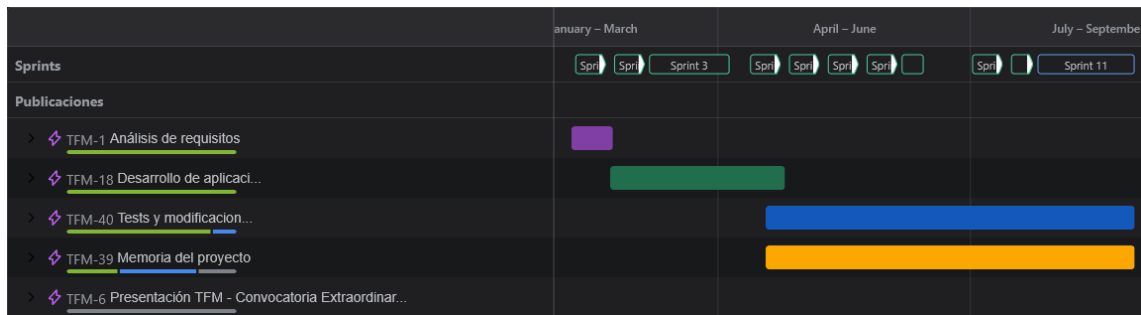


Figura 67: Cronograma del proyecto. Extraído del proyecto en Jira [56]

Fase 1: Análisis de requisitos

En esta fase se presenta la idea del proyecto. Al principio, la idea era seguir experimentando con la plataforma RASA e iniciar un nuevo proyecto dedicado a crear un chatbot dedicado a un dominio en específico.

Pronto, la idea giró en torno a la utilización de los LLMs, dado a que era y es una tecnología en auge que presenta una mejora notable tanto en eficiencia como en rendimiento a lo que ofrece RASA en estos momentos. En este tiempo se realizaron varias demos usando las plataformas RASA y LangChain, decidiéndose por usar esta última en el proyecto.

En la reunión que marcaba el final de sprint y de la fase, se determinó el propósito del chatbot: crear un asistente virtual capaz de responder con precisión y eficacia a cualquier pregunta relacionada con la DANA. Para ello, debíamos crear un RAG con los documentos del BOE relacionados con este tema. Además, nuestro tutor nos propuso la tarea de medir el grado de acierto del sistema RAG mediante el uso de agentes críticos.

Así pues, el proyecto quedó definido: crear un RAG sobre el contexto específico de la DANA, y medir su rendimiento evaluándolo a través de agentes críticos.

Fase 2: Desarrollo de la aplicación

En esta fase intermedia del proyecto, avanzamos en el desarrollo de la aplicación. Los cambios y adicciones al proyecto eran mostrados en las reuniones de finales de sprint, en las cuales se planteaban nuevas soluciones a las dificultades encontradas y se fijaban nuevos objetivos para el siguiente sprint. En esta etapa se completo el sistema RAG desde cero, desarrollo que se describe a continuación.

El primer escollo, fue conseguir la fuente de conocimiento. Gracias a que los documentos del BOE se pueden conseguir vía API, se facilitó este primer problema. Fueron diseñados varios scripts para conseguir de forma automática estos documentos, y con la ayuda de *GitHub Actions*, fueron programados para ejecutarse una vez al día salvo los domingos. De esta forma, obtuvimos la fuente de conocimiento que necesitaba nuestro RAG.

La creación del RAG no tuvo mucha complicación debido a la gran cantidad de tutoriales y la versatilidad que nos permite LangChain en este contexto. La primera versión sin interfaz, era totalmente funcional en apenas unos días.

En la interfaz empleamos más tiempo, dado a que la librería usada (Streamlit) ofrece

bastantes elementos de personalización. La versión final permite cambiar el modelo de lenguaje usado para generar la respuesta, y también, elegir la estrategia de búsqueda empleada por el modelo de recuperación. En la parte de creación y actualización de la base de conocimientos, usamos dos alternativas para subir documentos desde la propia interfaz: desde carpeta o desde ficheros individuales.

Completada esta fase, ya teníamos un sistema RAG totalmente funcional, con su fuente de conocimiento y una interfaz para responder consultas o añadir más documentos.

Fase 3: Test y modificaciones

Esta es la etapa de más duración del proyecto, y en la cual se realizaron todos los test para completar el segundo objetivo del trabajo.

En primera instancia, desarrollamos un pequeño script diseñado para generar un conjunto de preguntas extraídas de nuestra fuente de conocimiento. Para su obtención, usamos el modelo de lenguaje Mistral Small 3.2, en el cual configuramos un prompt para este cometido. El conjunto de preguntas conseguido fue el usado para elaborar los test.

Seguidamente, se preparó otro script en el que se configuró todo lo necesario para evaluar el RAG. Se definieron cuatro agentes críticos destinados a evaluar las métricas de *accuracy*, *faithfulness*, *groundedness* y *relevancia*, es decir, el grado de similitud de la respuesta generada con la esperada, el uso de las evidencias proporcionadas por el modelo de recuperación en la respuesta generadas, y como de relevantes son los documentos recuperados para responder a la consulta.

Las pruebas tomaron bastante tiempo debido al número realizado. Cada prueba de Mistral Small 3.2 empleaba alrededor de 3 horas en realizarse cuando el modelo estaba disponible, en ocasiones el servicio era más lento o fallaba. Llama 3.2 gastaba más o menos el mismo tiempo y GPT-4.1 era el más rápido (40 minutos), aunque para estos usamos conjuntos de pruebas más pequeños. Los resultados obtenidos fueron guardados y expuestos en tablas y gráficas, observables en el siguiente capítulo de esta memoria.

Fase 4: Documentación

Esta fase se realizó en conjunto con la anterior (fase 3), por eso tiene más o menos la misma duración que esta.

En la estructura seguimos como guía la memoria usada en mi TFG, añadiendo y quitando las secciones que creímos necesarias.

3.2.3. Presupuesto

En este apartado se describen los distintos recursos hardware y software empleados en el desarrollo del proyecto, explicando su uso y el coste de estos.

Recursos hardware

La tabla 3.1 muestra los recursos hardware usados. Para calcular el coste total de la tabla 3.2, hemos dividido el precio entre la vida útil de cada dispositivo en días, para después multiplicarlo por el tiempo de realización del proyecto (204 días).

Dispositivo	Modelo	Uso
Ordenador sobremesa	Ryzen5 3600 + 32GB RAM + 6700XT 12GB VRAM	Desarrollo y prueba de la aplicación
Ordenador portátil	Ryzen7 6800HS + 16GB RAM	Desarrollo y prueba de la aplicación

Tabla 3.1: Recursos hardware utilizados

Modelo	Precio	Vida útil	Total
Ryzen5 3600 + 32GB RAM + 6700XT 12GB VRAM	1500€	8 años	104.8€
Ryzen7 6800HS + 16GB RAM	1000€	6 años	94.15€
Total			198.95€

Tabla 3.2: Presupuesto de los recursos hardware utilizados

Recursos software

En este apartado se describen los recursos software utilizados, tanto su uso en la tabla 3.3, como su precio en la tabla 3.4. Los precios de estos recursos van ligados al coste de su licencia o uso.

Herramienta	Uso
Windows 11	Sistema Operativo
Visual Studio Code	Entorno de desarrollo
Ollama	Gestor de LLM y embeddings
LangChain	Librería control LLM
Streamlit	Librería interfaz
GitHub & GitHub Actions	Plataforma donde esta el repositorio del proyecto, y plataforma para crear automatizaciones
Mistral API	Uso de LLMs de Mistral
OpenAI API	Uso de LLMs y embeddings de OpenAI
TexStudio	Procesador de látex usado para elaborar la memoria

Tabla 3.3: Recursos software utilizados

Herramienta	Uso
Windows 11	145€
Visual Studio Code	Gratuito
Ollama	Gratuito
LangChain	Gratuito
Streamlit	Gratuito
GitHub & GitHub Actions	Gratuito
Mistral API	Gratuito
OpenAI API	12,79€ (15\$)
TexStudio	Gratuito
Total	157.79€

Tabla 3.4: Presupuesto de los recursos software utilizados

Recursos personales

En este apartado describiremos los miembros presentes en la realización del proyecto. En la tabla 3.5 tendremos en cuenta el puesto, tiempo invertido y coste de ese tiempo.

Las horas realizadas por los tutores se han calculado teniendo en cuenta una media de una hora por reunión. De un total de 15 sesiones de reunión, disponemos de un total de 900 minutos o 15 horas.

El coste de su salario, se ha estimado sabiendo el salario bruto aproximado de un profesor de universidad (31500€ anuales, 24500€ netos según el IRPF de 2024). Por lo que, en un horario normal de 40 horas semanales, habiendo 4.34 semanas/mes , asciende a 11.76€ por hora.

Para el cómputo de mis horas, conociendo la duración del proyecto y en una media de 3 horas de trabajo al día, tenemos un total de 612 horas aproximadamente. El salario medio de un programador junior ronda los 1500€, así que podemos estimar un salario medio por hora de 8.64€, y un total de 5.287,68€.

Nombre	Puesto	Tiempo	Coste	Total
David Griol Barres	Profesor de Universidad	15h	11.76€/h	176.4€
Zoraida Callejas Carrión	Profesor de Universidad	15h	11.76€/h	176.4€
Pablo Valenzuela Álvarez	Programador Junior	612h	8.64€	5.287,68€
Total				5.640,48€

Tabla 3.5: Presupuesto de los recursos personales utilizados

Coste total

Sumando todos los costes parciales del proyecto, tenemos un subtotal de 5.997,22€.

Tipo	Coste
Coste del hardware	198.95€
Coste del software	157.79€
Coste del personal	5.640,48€
Total	5.997,22€

Tabla 3.6: Coste total del proyecto

3.3. Construcción del corpus de documentos

La plataforma elegida para el control de versiones del proyecto fue *GitHub*, ya que es la plataforma con la que llevamos años trabajando en proyectos anteriores y es la más implantada en el mercado.

A parte de dar un acceso rápido y seguro al código del repositorio, *GitHub* ofrece una herramienta para controlar flujos de trabajo automáticos llamada **GitHub Actions**. Flujos que hemos usado para automatizar las descargas de los documentos usados para crear el corpus de información con la que hemos nutrido el modelo de recuperación de información de nuestro sistema RAG. Este es un concepto conocido como integración continua y entrega continua (*CI/CD*).

Como se ha mencionado en anteriores apartados, hemos usado documentos del BOE (Boletín Oficial del Estado) relacionados con la DANA para construir nuestro corpus de conocimiento. Desde la propia web del BOE, podemos acceder a los sumarios de cada día mediante descarga gracias a una API.

GET /datosabiertos/api/boe/sumario/{fecha}

Descripción
Sumario para un día determinado.

Formato de la respuesta
application/xml  [Descargar XSD](#)

Parámetros

Parámetro	Valor	Descripción
fecha		Fecha (aaaammdd)

Figura 68: BOE. API web desde la que descargar sumarios por día [57]

cURL

```
curl -L -X GET -H "Accept: application/xml" "https://www.boe.es/datosabiertos/api/boe/sumario/20250703"
```

URL

```
https://www.boe.es/datosabiertos/api/boe/sumario/20250703
```

Cuerpo de la respuesta

```
<?xml version="1.0" encoding="utf-8"?>
<response>
  <status>
    <code>200</code>
    <text>ok</text>
  </status>
  <data>
    <sumario>
      <metadatos>
        <publicacion>BOE</publicacion>
        <fecha_publicacion>20250703</fecha_publicacion>
      </metadatos>
      <diario numero="159">
        <sumario_diario>
          <identificador>BOE-S-2025-159</identificador>
          <url_pdf szBytes="428309" szKBytes="418">https://www.boe.es/boe/dias/2025/07/03/pdfs/BOE-S-2025-159.pdf</url_pdf>
        </sumario_diario>
        <seccion codigo="1" nombre="I. Disposiciones generales">
          <departamento codigo="9562" nombre="MINISTERIO DE ASUNTOS EXTERIORES, UNIÓN EUROPEA Y COOPERACIÓN">
            <epigrafe nombre="Tratados internacionales">
              <item>
                <identificador>BOE-A-2025-13557</identificador>
                <control>2025/10194</control>
                <título>Enmiendas de 2019 al Convenio internacional para la seguridad de la vida humana en el mar, 1974, adoptadas en Londres el 13 de junio de 2019 mediante la Resolución<
                <url_pdf szBytes="208477" szKBytes="204" pagina_inicial="87709" pagina_final="87710">https://www.boe.es/boe/dias/2025/07/03/pdfs/BOE-A-2025-13557.pdf</url_pdf>
                <url_html>https://www.boe.es/diario_boe/txt.php?id=BOE-A-2025-13557</url_html>
                <url_xml>https://www.boe.es/diario_boe/xml.php?id=BOE-A-2025-13557</url_xml>
              </item>
            </epigrafe>
          </departamento>
        </seccion>
      </diario>
    </sumario>
  </data>
</response>
```

Figura 69: BOE. Resultados de la búsqueda de un día [57]

Esta API nos ofrece dos opciones de descarga para el sumario: json y xml. En la figura 69 podemos ver la versión xml de la descarga.

El requerimiento para la obtención del sumario es la fecha redactada en un formato **aaaammdd**, es decir, si queremos obtener el sumario del día 30 de mayo de 2025, tenemos que redactarlo de la siguiente manera: 20250530.

3.3.1. Script de descarga de sumarios

La fecha del sistema es fácil de obtener mediante código, por dicha razón se implementaron dos scripts: uno para la descarga automática del sumario del día actual y otro para la descarga de sumarios de días anteriores, necesarios para el paso siguiente (ver sección 3.3.2).

```
scripts > get_sumario_hoy.sh
1  #!/bin/bash
2
3  # Obtener la fecha actual en formato YYYYMMDD
4  fecha=$(date +"%Y%m%d")
5
6  # Definir nombres del sumario
7  sumario="../data/sumarios/$fecha.xml"
8
9  echo " 📄 Iniciando descarga del sumario del día $(date)."
10
11 # Descarga
12 curl -s -L -X GET -H "Accept: application/xml" "https://www.boe.es/datosabiertos/api/boe/sumario/$fecha" -o "$sumario"
13 echo " ✅ XML descargado: $sumario"
14
15 echo " 📄 Descarga completada."
16
```

Figura 70: Script para descargar el sumario del día

Como se observa en la figura 70, se obtiene la fecha del sistema automáticamente y se usa tanto para la llamada a la API como para guardar el documento descargado. Este script es el se usó posteriormente para crear hacer una automatización por medio de GitHub Actions (ver sección 3.3.3).

3.3.2. Script para la descarga de documentos

Los sumarios en su estructura xml contienen varias etiquetas que podemos usar para buscar y descargar los documentos que nos interesen. Como podemos observar en la figura 69, la etiqueta *item* engloba tanto las etiquetas del título como varias etiquetas de descargas. Ayudándonos de estas etiquetas, se optó por crear un script en Python que automatizase el proceso de descarga de los documentos relacionados con la DANA.

El funcionamiento de este script es el siguiente:

- Recorremos en bucle todas las etiquetas “*item*”
- Para cada etiqueta “*item*” se guardan en variables: título y enlace de descarga pdf.
- Se comprueba si el título contiene la palabra “DANA”.
- Se descarga el documento gracias a la etiqueta del enlace pdf.

En la figura 71 se puede observar este procedimiento.

```
# Buscar y procesar los items que contienen "DANA"
for item in root.findall("./item"):
    identificador = item.find("identificador").text if item.find("identificador") is not None else "N/A"
    titulo = item.find("titulo").text if item.find("titulo") is not None else "N/A"
    url_pdf = item.find("url_pdf").text if item.find("url_pdf") is not None else "N/A"

    # Filtrar solo los que contienen "DANA"
    if re.search(r'\bDANA\b', titulo, re.IGNORECASE):
        print(f"🔍 Identificador: {identificador}")
        print(f"📄 Título: {titulo}")
        print(f"📄 Descargando PDF: {url_pdf}")

        # Descargar XML y PDF, y guardar el enlace HTML
        archivo_pdf = descargar_pdf(url_pdf, identificador)

        print(f"✅ PDF guardado en: {archivo_pdf if archivo_pdf else '❌ Error en descarga'}")
        print("-" * 50)
```

Figura 71: Fragmento del script encargado de obtener documentos relacionados con la DANA

Como sucedió en el apartado anterior, se hizo necesaria la implementación de otro script para la descarga de los documentos de días anteriores. El procedimiento es el mismo que el visto en la figura 71, añadiendo un bucle extra en el cual recorre todos los sumarios descargados.

Tanto estos como los scripts del apartado anterior, están disponibles en la carpeta **scripts** del repositorio [58] en *GitHub*.

3.3.3. Automatización de procesos de descarga

Una vez implementados los scripts para la descarga de sumarios y de documentos, llega la hora de crear la automatización en *GitHub*. Para ello usaremos la herramienta para crear acciones: *GitHub Actions*.

GitHub Actions es una plataforma de integración y despliegue continuos (*CI/CD*) que permite la automatización de flujos de trabajo [59].

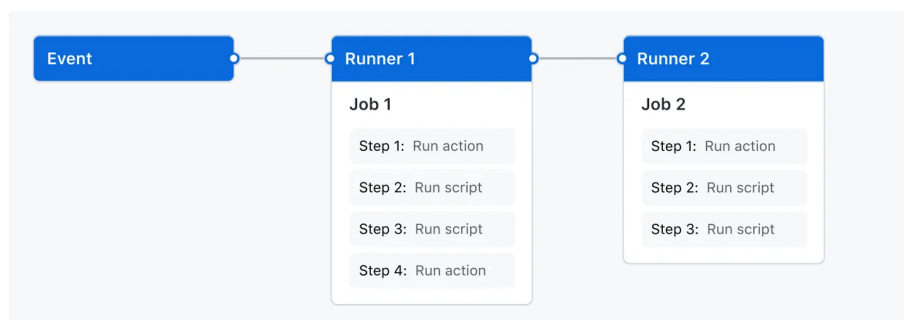


Figura 72: GitHub Actions. Ejemplo de funcionamiento de un flujo de trabajo [59]

Como se observa en la figura 72, un flujo de trabajo (*workflow*) se activa cuando sucede un evento (*event*) en el repositorio: *push*, *commit*, etc. Cada trabajo (*job*) ejecuta uno o varios scripts en una máquina virtual creada automáticamente que se elimina al completar los procesos del flujo de trabajo.

Los flujos de trabajo son procesos automatizados y configurables definidos por un fichero *YAML* ubicado en el directorio *.github/workflow* de nuestro repositorio [58]. Nuestro objetivo es que el flujo de trabajo se ejecute una vez al día, exceptuando los domingos (no

se publica el BOE los domingos); y que ejecute ambos scripts vistos en las dos secciones anteriores.

En este sentido, tenemos que configurar un evento que haga que se ejecute todos los días exceptuando el domingo. Esto se logra usando el comando *cron* como se muestra en la figura 73. En dicha figura, se programa para ejecutarse de lunes a sábado a las 9:00 de la mañana, sin importar día o mes.

```
on:
  schedule:
    - cron: "0 9 * * 1-6"
```

Figura 73: GitHub Actions. Fragmento del flujo de trabajo que muestra el uso del comando cron

Tras programar el evento que activa el flujo de trabajo, solo queda indicar las tareas que se ejecutarán en el flujo. En la figura 74 se pueden observar estas tareas bajo la etiqueta *steps*, estos son iniciadas secuencialmente como indica la figura. En las primeras líneas, indica sobre que sistema se esta ejecutando: una máquina virtual Ubuntu.

```
jobs:
  tareas_diarias:
    name: Ejecución de tareas diarias
    runs-on: ubuntu-latest

    steps:
      - name: Checkout del repositorio
        uses: actions/checkout@v4.2.2

      - name: Configuración de Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Descarga del resumen del día
        run: |
          cd scripts
          chmod +x ./get_sumario_hoy.sh
          ./get_sumario_hoy.sh

      - name: Descarga de elementos del resumen del día
        run: |
          cd scripts
          pip install requests
          chmod +x ./get_items_hoy.py
          python ./get_items_hoy.py

      - name: Commit y push al repositorio
        uses: stefanzweifel/git-auto-commit-action@v5
        with:
          commit_message: Update data
```

Figura 74: GitHub Actions. Fragmento del flujo de trabajo que muestran los trabajos que son ejecutados

La lista siguiente contiene una breve descripción de cada una de las tareas del flujo por sus nombres:

1. **Checkout del repositorio:** Es una acción creada por la comunidad que clona el repositorio en la máquina virtual.

- 2. **Configuración de Python:** Utiliza otra acción creada por la comunidad, se encarga de instalar un entorno *Python* en la máquina virtual.
- 3. **Descarga del sumario del día:** Lanza el script de descarga del sumario del día (sección 3.3.1).
- 4. **Descarga de elementos del sumario del día:** Lanza el script de descarga de documentos del sumario del día relacionados con la DANA (sección 3.3.2).
- 5. **Commit y push al repositorio:** Utiliza la acción *auto-commit*, creada por la comunidad. Como su nombre indica, se encarga de actualizar los sumarios y documentos descargados, si hay nuevos.

Para finalizar la subsección, queda comprobar si la acción se esta lanzando correctamente. Podemos obtener esta información en la pestaña “Acciones” de la página web de *GitHub* de nuestro repositorio [58]. En las figuras siguientes podemos observar el funcionamiento de las acciones en el repositorio del proyecto [58].

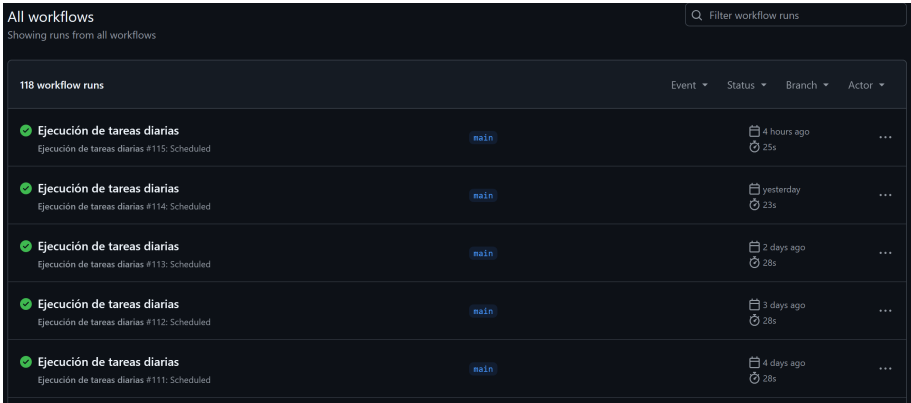


Figura 75: Pestaña de acciones del proyecto. Se muestran una parte de las acciones automatizadas

Podemos comprobar en la figura 75 como, efectivamente, las acciones de lanzan una vez al día.

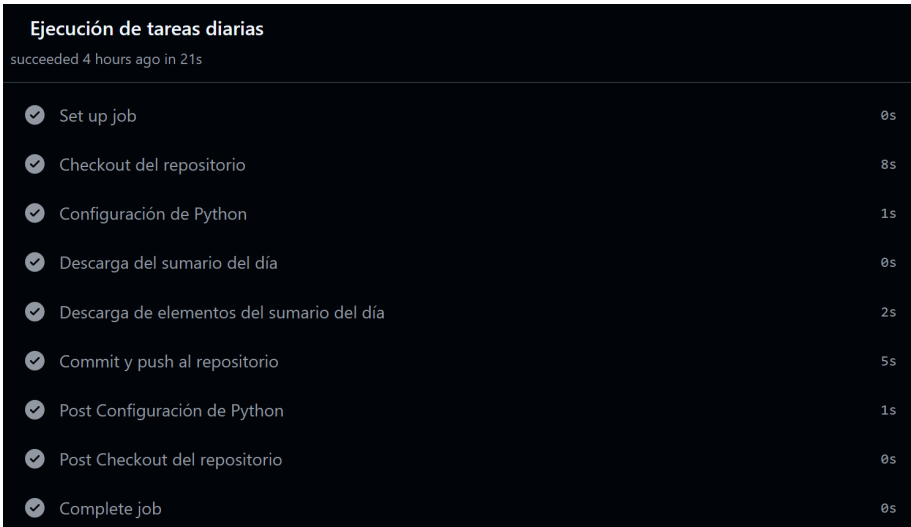


Figura 76: Vista en detalle de una acción. Se observa el tiempo empleado en realizar cada paso

En la vista en detalle aportada en la figura 76, podemos entender como funciona internamente una acción, siguiendo los pasos indicados en el fichero *YAML*. Los procesos *post* se encargan de deshacer los procesos de *checkout* y configuración de *Python*, para terminar liberando la máquina virtual en el último paso.

De esta manera, hemos logrado crear una base de conocimiento con documentos del BOE relacionados con la DANA, y que es actualizada diariamente.

3.4. Desarrollo del chatbot

Esta sección contará los pasos que hemos seguido en el desarrollo de la aplicación chatbot que tiene de base nuestro sistema RAG. Encontraremos apartados dedicados tanto al desarrollo de la interfaz web como al de los diferentes componentes que configuran nuestro sistema RAG, incluyendo imágenes con fragmentos de código y de la interfaz final.

El objetivo es crear una interfaz en la cuál: podamos añadir nuevos documentos a nuestro sistema RAG y, evidentemente, un chatbot con el que podamos comunicarnos, y que utilice documentos recuperados del RAG para construir su respuesta. Para dicho cometido se implemento una clase *chatbot* (ubicada dentro de la carpeta *classes* del repositorio [58]), que contiene los métodos necesarios para hacer un funcionar el sistema RAG completo.

En las siguientes subsecciones mostraremos la implementación de estos procesos: la configuración, carga y agregación de nuevos documentos a la base de datos vectorial, y la recuperación y generación de información; todo ejemplificándolo con fragmentos de código representativos y muestras del resultado en la interfaz web.

3.4.1. Clase Chatbot: constructor

El constructor de la clase *Chatbot*, como se observa en la figura 77, contiene una gran cantidad de parámetros. Con ello podemos configurar desde la inicialización todos los aspectos necesarios del sistema RAG propuesto.

```
44 class Chatbot:
45
46     ### Constructor ###
47     def __init__(self,
48                 language_model: str = "mistral-small-latest", num_ctx: int = 4096,
49                 chunk_size: int = 1024, chunk_overlap: int = 100,
50                 embedding_model: str = EMBEDDING.NOMIC.value,
51                 search_type: str = "similarity", k: int = 5,
52                 chroma_directory: str = "chroma",
53                 docs_directory: str = "my_data",
54                 reranker_model: str = "BAAI/bge-reranker-v2-m3",
55                 top_n: int = 3,
56                 prompt_template: str = default_prompt
57                 ):
```

Figura 77: Clase Chatbot. Parámetros del constructor

En la siguiente lista, vamos a explicar cada uno de los parámetros:

- **language_model:** Indica el modelo de lenguaje (LLM) que usaremos para generar la respuesta.
- **num_ctx:** Ventana de contexto que usa el LLM, es decir, la cantidad de información aportada al LLM mediante el prompt.
- **chunk_size y chunk_overlap:** Parámetros usados para separar y solapar documentos a la hora de añadirlos al RAG.
- **embedding_model:** Modelo de embedding usado por el RAG, también es usado en las consultas de usuario.
- **search_type y k:** Estrategia de búsqueda usada al recuperar información. La cantidad de documentos recuperados la indica k.
- **chroma_directory:** Localización del modelo de recuperación (base de datos vectorial).
- **docs_directory:** Localización del corpus usado, que nutre la base de datos vectorial.
- **reranker_model y top_n:** Modelo utilizado para hacer reranking sobre los documentos recuperados. La cantidad de documentos filtrados por el reranker la indica top_n.
- **prompt_template:** Plantilla de texto utilizada para generar la respuesta.

Como se observa en la imagen anterior cada parámetro usa valores por defecto, de esta manera se facilita el uso de esta clase. Cada uno de estos parámetros son editables en todo momento, puesto que se han incluido métodos *GET* y *SET* para estos cometidos.

A continuación veremos en las figuras retazos de la configuración de alguno de ellos. Como por ejemplo la figura 78, donde se muestra la configuración del modelo de lenguaje según sea el seleccionado. Hasta ahora contamos con el modelo *mistral-small* correspondiente a MistralAI, *GPT-4.1* de OpenAI y Llama3.2 perteneciente a Ollama, por lo que necesitamos diferentes clases para que funcionen: ChatMistralAI, ChatOpenAI y ChatOllama. En los modelos proporcionados por Mistral AI y OpenAI necesitamos facilitar la clave de su API.

```
if language_model == LLM.mistral_small.value:
    self.language_model = ChatMistralAI(
        model=language_model,
        mistral_api_key=MISTRAL_API_KEY,
        temperature=0, random_seed=12345)
elif language_model == LLM.gpt_41.value:
    self.language_model = ChatOpenAI(
        model=language_model,
        openai_api_key=OPEN_API_KEY, temperature=0)
else:
    self.language_model = ChatOllama(
        model=language_model, num_ctx=num_ctx,
        temperature=0, seed=12345)
```

Figura 78: Clase Chatbot. Configuración del LLM

En cuanto a la configuración del modelo de recuperación, la figura 79 muestra su implementación. La separación de documentos es efectuada mediante el uso de la clase *RecursiveTextSplitting* [60], que se encarga de separar los documentos dado un determinado tamaño del chunk (*chunk_size*). El solapamiento se controla con *chunk_overlap*, este ayuda a mantener una relación semántica entre documentos similares o consecutivos.

La figura 79 también muestra como se configura el modelo de embedding y la base de datos vectorial (*vector_store*); utilizando sendas clases *OllamaEmbeddings* y *Chroma* para realizar estas acciones. En el caso de usar un modelo de embedding proveniente de OpenAI, tenemos que usar la clase *OpenAIEmbeddings* que ofrece LangChain.

```
self.text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap,
    length_function=len)

if embedding_model == EMBEDDING.TE3S.value:
    self.embedding_service = OpenAIEmbeddings(
        model=embedding_model,
        openai_api_key=OPEN_API_KEY)
else:
    self.embedding_service = OllamaEmbeddings(model=embedding_model)

self.vector_store = Chroma(
    persist_directory=chroma_directory,
    embedding_function=self.embedding_service)
```

Figura 79: Clase Chatbot. Configuración de varios parámetros: text_splitter, modelo de embedding y modelo de recuperación

La configuración de la estrategia de búsqueda no es tan simple, en la figura 80 podemos ver las opciones de configuración disponibles. Mientras que métodos como *Similarity* o *MMR* están incluidas por defecto por Chroma, necesitamos usar clases específicas para las demás.

Las estrategias *TF-IDF* y *BM25* utilizan una metodología distinta, por lo que necesitan el uso de las clases *TFIDFRetriever* [61] y *BM25Retriever* [62] para funcionar correctamente.

La estrategia de recuperación en grafo usa la clase *GraphRetriever* [63], en la que usamos algunos metadatos obtenidos de los documentos para filtrar y recuperar documentos con metadatos similares.

```
self.retriever = search_type
if search_type == "similarity" or search_type == "mmr":
    self.retriever = self.vector_store.as_retriever(search_type=search_type, search_kwargs={"k": k})
elif search_type == "tfidf":
    docs = self.__get_documents_from_chroma()
    self.retriever = TFIDFRetriever.from_documents(docs, k=k)
elif search_type == "bm25":
    docs = self.__get_documents_from_chroma()
    self.retriever = BM25Retriever.from_documents(docs, k=k)
elif search_type == "grafo":
    self.retriever = GraphRetriever(
        store=self.vector_store,
        edges=[("keywords", "keywords"), ("author", "author"), ("subject", "subject"), ("title", "title")],
        strategy=Eager(k=k, start_k=1, max_depth=2)
    )
```

Figura 80: Clase Chatbot. Configuración de la estrategia de búsqueda

La especificación del prompt es sencilla y se puede implementar en una sola línea (figura

81. La clase *PromptTemplate* ofrece multitud de métodos para este fin. El contenido de la variable *prompt_template* se puede ver al final de la sección 3.4.3 de esta memoria.

```
self.prompt_template = PromptTemplate.from_template(prompt_template)
```

Figura 81: Clase Chatbot. Configuración del prompt

La última de las configuraciones corresponde a la del modelo para hacer **reranking** (figura 82). Para ello necesitamos usar varias clases, y como parámetros: el modelo de reranking y el número de documentos que filtrará (*top_n*).

La variable *compressor_retriever* guarda la configuración del modelo de reranking junto con el modelo de recuperación escogido para esta iteración del sistema RAG. De modo que al llamarla, se producirá un “doble” filtrado de los documentos: primero se producirá la recuperación de documentos normal, y seguido, el filtrado de los mejores documentos recuperados por parte del modelo reranker.

```
self.hf_cross_encoder = HuggingFaceCrossEncoder(model_name=reranker_model)
self.compressor = CrossEncoderReranker(model=self.hf_cross_encoder, top_n=top_n)
self.compression_retriever = ContextualCompressionRetriever(
    base_compressor=self.compressor,
    base_retriever=self.retriever
)
```

Figura 82: Clase Chatbot. Configuración del reranker

Hemos podido ver en las figuras anteriores, la cantidad de integraciones que hemos usado para la configuración de los diferentes elementos de la clase *Chatbot*. Como ya comentamos en la sección 2.7, esta es la gran ventaja que ofrece *LangChain* respecto a otros frameworks, y que tenemos que aprovechar para facilitar el desarrollo de cualquier proyecto.

3.4.2. Interfaz de carga y actualización de la base de datos

En este apartado veremos como cargar y actualizar la base de datos vectorial del sistema RAG. Las siguientes imágenes, muestran fragmentos del código utilizado para su implementación.

En la siguiente figura podemos ver como se implementa parte de la interfaz usando *Streamlit*. Hemos marcado tres bloques en la figura que pasaremos a explicar a continuación.

```
def update_page():
    if "chatbot" not in st.session_state:
        st.session_state["chatbot"] = Chatbot(chroma_directory="chroma/snow_1024")

    st.set_page_config(page_title="Actualizar ", page_icon="images/icon_blue.png")
    st.logo("images/horizontal_blue.png", icon_image="images/icon_blue.png")

    st.header("Actualizar la base de datos")

    get_existing_docs()

    col1, col2 = st.columns([3, 1], vertical_alignment="bottom")
    with col1:
        st.text_input(
            label="Carga desde directorio",
            value = st.session_state.get("folder_path", None),
            disabled=True
        )
    with col2:
        st.button("Seleccionar y Cargar", on_click=select_folder)

    st.file_uploader(
        "Carga desde fichero/s",
        key="file_uploader",
        type="pdf",
        accept_multiple_files=True,
        on_change=upload_files
    )
```

Figura 83: Interfaz. Código de la página de carga y actualización de la base de datos

El primer bloque contiene la inicialización del *Chatbot* junto con la configuración de logos y cabeceras de la página. *Streamlit* provee de cantidad de recursos como se observa en el bloque, por ejemplo, cuenta con la variable *session_state* a la cual podemos asignar cualquier tipo variable sin temer perder los datos al actualizar la página.

En los bloques dos y tres, añadimos a la interfaz estructuras que permiten subir documentos, ya sea desde directorio con la función *select_folder* (bloque dos) o simplemente cargando ficheros individuales y la función *upload_files* (bloque 3). Los ficheros usados para añadir documentos a la base de datos son automáticamente copiados a un directorio asignado en el constructor (*docs_directory*, figura 78), creando así una base de referencias a las consultas de usuario.

El resultado final de la interfaz se muestra en la siguiente figura. El código completo de esta página esta disponible en el repositorio [58], carpeta *pages* en el archivo **update_db.py**.



Figura 84: Interfaz. Diseño de la página de carga y actualización de la base de datos

En cuanto al código necesario para actualizar la base de datos, desarrollamos en la clase *Chatbot* varias alternativas para incluir nuevos documentos. En la parte a) de la figura 85 podemos ver la versión para cargar todos los documentos de un directorio, ya que usa la clase *PyPDFDirectoryLoader* (la versión para cargar ficheros individuales es exactamente igual, salvo por la carga individual con *PyPDFLoader*). También podemos observar como separa los documentos (*text_splitter*), genera diferentes IDs (*__calculate_chunk_ids*) y los incluye en la base de datos (*vector_store*).

La parte b) de la misma figura muestra como se generan los IDs para los documentos, formándolos con partes de sus metadatos: nombre de archivo, número de página y número del fragmento en la página (depende del tamaño del chunk asignado, una página puede contener de 2 a 5 trozos). De esta forma obtenemos la referencia completa de donde ha obtenido la información el RAG ante una consulta de usuario (ver siguiente sección, figura 105).

```
def populate_db(self, directory_path: str):
    start = timer()
    documents = PyPDFDirectoryLoader(directory_path).load()
    chunks = self.text_splitter.split_documents(documents)
    chunks_with_ids = self.__calculate_chunk_ids(chunks)

    existing_items = self.vector_store.get(include=[])
    existing_ids = set(existing_items["ids"])

    new_chunks = []
    for chunk in chunks_with_ids:
        if chunk.metadata["id"] not in existing_ids:
            new_chunks.append(chunk)

    if len(new_chunks):
        st.write(f"🔥 Añadiendo nuevos documentos: {(len(new_chunks))}")
        new_chunk_ids = [chunk.metadata["id"] for chunk in new_chunks]
        self.vector_store.add_documents(new_chunks, ids=new_chunk_ids)
    else:
        st.write(f"✅ No hay nuevos documentos para añadir")

    self._update_tfidf_bm25_retrievers()
    st.write(f"⌚ Tiempo empleado: {(round(timer() - start, 2))} segundos**")
```

(a) Método populate_db

```
def __calculate_chunk_ids(self, chunks):
    """
    Genera ids para los chunks
    """

    last_page_id = None
    current_chunk_index = 0

    for chunk in chunks:
        source = chunk.metadata.get("source")
        page = chunk.metadata.get("page")
        current_page_id = f"{source}:{page}"

        if current_page_id == last_page_id:
            current_chunk_index += 1
        else:
            current_chunk_index = 0

        chunk_id = f"{current_page_id}:{current_chunk_index}"
        last_page_id = current_page_id

        chunk.metadata["id"] = chunk_id

    return chunks
```

(b) Método __calculate_chunk_id

Figura 85: Clase Chatbot. Fragmentos de código de algunos de los métodos usados para actualizar la base de datos

El código de esta clase esta disponible en la carpeta *classes* del repositorio [58], en el archivo **chatbot.py**.

Finalizaremos este apartado mostrando algunas figuras de los cuadros de diálogo diseñados al añadir archivos a nuestra base de datos.

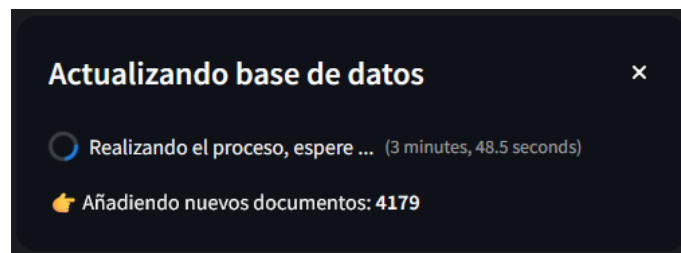


Figura 86: Interfaz. Cuadro de dialogo de tiempo de espera



Figura 87: Interfaz. Cuadros de dialogo. Tiempos para bases de datos con documentos de tamaño 512



Figura 88: Interfaz. Cuadros de diálogo. Tiempos para bases de datos con documentos de tamaño 1024

Si nos fijamos en las figuras 87 y 88, destaca la diferencia de tiempo existente entre el par *nomic-jina* y *bge-snowflake*, siendo estos últimos los que más tiempo consumen al incluir nuevos documentos en la base de datos vectorial. El motivo para esta diferencia puede residir en una metodología más compleja para vectorizar los documentos de este último par, puesto que son los modelos más pesados en comparación.

En contraste con los modelos locales, el tiempo de embedding del modelo *text-embedding-3-small* (OpenAI) es extremadamente rápido. Como se puede observar en la figura 89, el modelo vectoriza todos los documentos en solo 40 segundos.



Figura 89: Interfaz. Tiempo de embedding de *text-embedding-3-small*

3.4.3. Interfaz del chatbot: Recuperación y generación de información

En este apartado explicaremos el funcionamiento de la interfaz del chatbot del sistema RAG, mostrando los apartados correspondientes a la recuperación y generación de información especialmente. Ayudándonos de *Streamlit*, se ha implementado página principal (figura 90) con dos elementos personalizables: modelo de lenguaje y estrategia de búsqueda; y el chat.

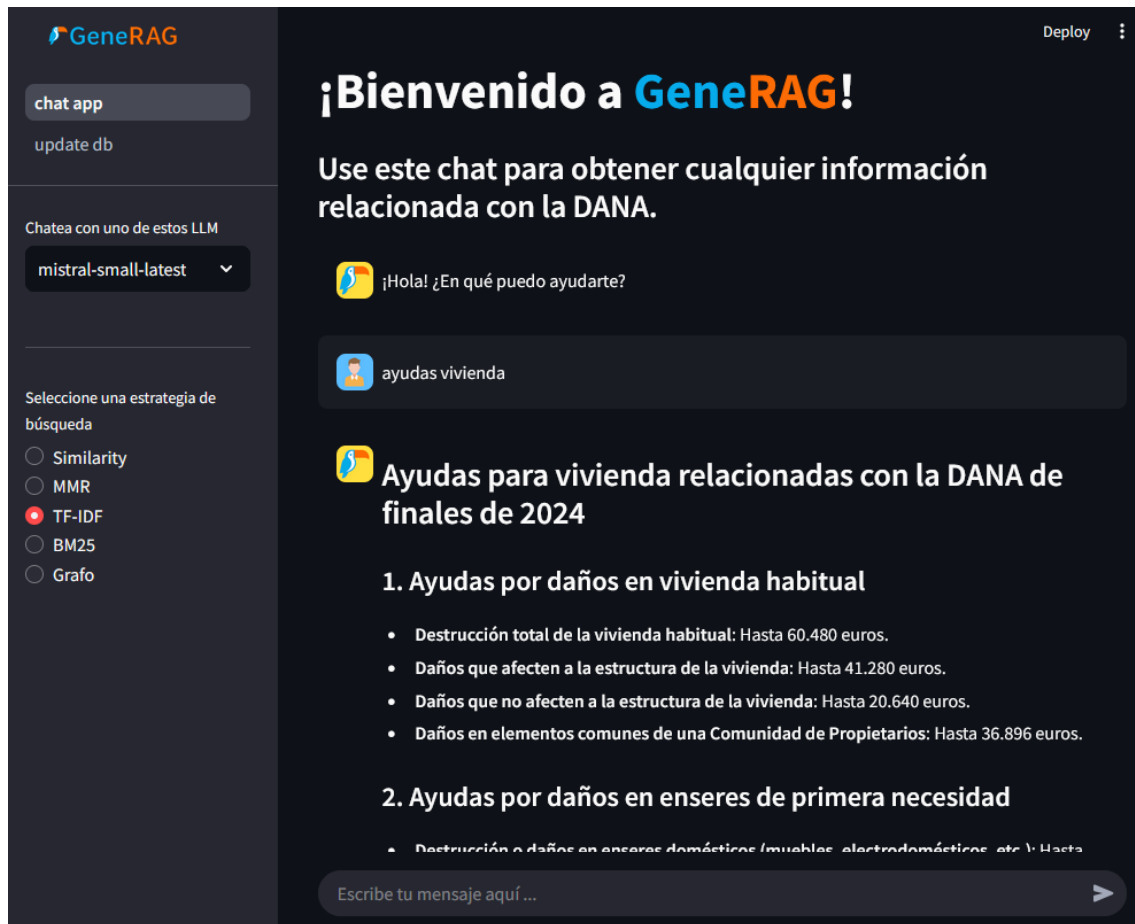


Figura 90: Interfaz. Página principal

En la figura 90 se pueden observar los elementos mencionados en el párrafo anterior, situados en la barra lateral izquierda; además del chat con una conversación entre el usuario y el sistema RAG. A continuación, pasaremos a explicar el funcionamiento interno de estos elementos proporcionados por *Streamlit* y también, las partes de recuperación y generación de información en la parte del chat.

Configuración inicial del chatbot

Al igual que en la página de actualización de la base de datos (figura 83), también necesitamos configurar el chatbot en la página principal.

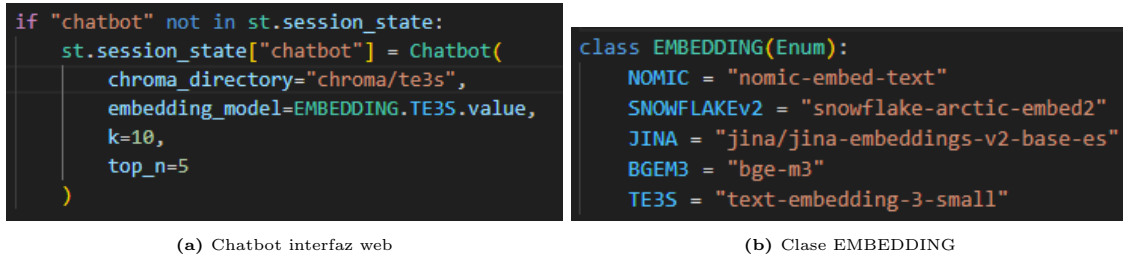


Figura 91: Fragmentos de código. Declaración de la variable Chatbot en la interfaz principal y clase EMBEDDING

En la parte a de la figura anterior, se observan los detalles aportados para dicha configuración. Indicamos el modelo de embedding (del cual podemos elegir entre las opciones disponibles vistas en la parte b), los documentos recuperados (**k**) y los documentos filtrados por el reranker (**top_n**). Como se observa en la figura, este ejemplo usa el modelo de embedding *text-embedding-3-small* junto un modelo de reranking, que se encargará de filtrar los cinco mejores documentos de los diez recuperados por el modelo de recuperación.

La clase EMBEDDING esta situada en la carpeta *classes*, en el fichero **LLM.py**. Los modelos de embedding disponibles deben ser descargados con antelación mediante Ollama (sección 2.5) a excepción del modelo *text-embedding-3-small*, del cual necesitaremos aportar nuestra clave de OpenAI.

Elementos de selección: selectbox y radio

Streamlit ofrece en su documentación oficial multitud de componentes que podemos usar en la interfaz web. Como se observó en la figura 90, hemos usado una caja de selección (*selectbox*) para elegir el modelo de lenguaje con el que chatear, y botones de radio (*radio*) para escoger la estrategia de búsqueda del modelo de recuperación.

El código mostrado en la parte a de la figura 92, muestra la implementación de la caja de selección. La etiqueta *sidebar* indica a Streamlit, que debe estar situado en la barra lateral, normalmente situada a la izquierda.

De entre los parámetros, destacan:

- **options:** Lista de elementos que muestra la caja de selección (parte b, figura 92).
- **key:** Variable de *session_state* que tomará el valor determinado en la caja de selección.
- **on_change:** Función que se activará cuando cambie el valor de la caja de selección.

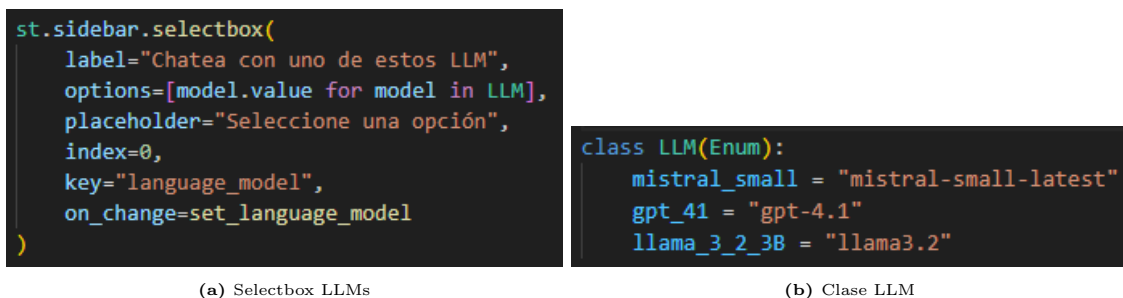


Figura 92: Fragmentos de código. Caja de selección de los modelos de lenguaje y clase LLM.

Los modelos disponibles vistos en la parte b de la figura 92, han de ser descargados usando *Ollama*; exceptuando los modelos de *Mistral AI* y de *OpenAI*, del cual necesitamos un código de su API.

La función *set_language_model* (figura 93) usa un método disponible en la clase *Chatbot* que cambia el modelo de lenguaje a usar por el RAG (figura 94). Dicho método actúa diferente según el modelo elegido, necesitando la clave API para los modelos de *Mistral AI* y *OpenAI*. El método es similar al aportado en el constructor, figura 78.

```
def set_language_model():
    chatbot = st.session_state["chatbot"]
    chatbot.set_language_model(st.session_state["language_model"])
```

Figura 93: Función de la interfaz. Modificar el modelo de lenguaje

```
def set_language_model(self, language_model: str, num_ctx: int = 4096):
    if language_model == LLM.mistral_small.value:
        self.language_model = ChatMistralAI(
            model=language_model,
            mistral_api_key=MISTRAL_API_KEY,
            temperature=0, random_seed=12345)
    elif language_model == LLM.gpt_41.value:
        self.language_model = ChatOpenAI(
            model=language_model,
            openai_api_key=OPEN_API_KEY,
            temperature=0)
    else:
        self.language_model = ChatOllama(
            model=language_model,
            num_ctx=num_ctx,
            temperature=0, seed=12345)
```

Figura 94: Clase *Chatbot*. Modificar el modelo de lenguaje

La selección en radio, se implementa de forma similar (parte a de la figura 95). Los valores entre los que seleccionar, en este caso han de ser aportados entre corchetes. Entre los demás parámetros destacan:

- **key:** Variable de *session_state* que tomará el valor determinado en la selección de radio.
- **on_change:** Función que se activa al cambiar el valor de la selección de radio.

La función de la interfaz vista en la parte b de la figura 95, activa el método de clase *Chatbot* para cambiar la estrategia de búsqueda.

```
st.sidebar.radio(
    "Seleccione una estrategia de búsqueda",
    ["Similarity", "MMR", "TF-IDF", "BM25", "Grafo"],
    key="search_type",
    index=2,
    on_change=set_search_type
)
```

(a) Selección de radio, estrategias de búsqueda

```
def set_search_type():
    chatbot = st.session_state["chatbot"]
    chatbot.set_search_type(st.session_state["search_type"])
```

(b) Función de la interfaz

Figura 95: Fragmentos de código. Selección de radio para las estrategias de búsqueda y función de la interfaz para cambiarlas

Como se observa en la figura siguiente, cada estrategia cambia el método de recuperación (*retriever*). Las estrategias *similarity* y *mmr* son traídas por defecto en la base de datos Chroma, mientras que para las demás tenemos que usar clases externas: *TFIDFRetriever*, *BM25Retriever* y *GraphRetriever*. (El método es similar al aportado en el constructor, figura 80).

```
def set_search_type(self, search_type: str, k: int = 5):
    if search_type == "similarity" or search_type == "mmr":
        self.retriever = self.vector_store.as_retriever(search_type=search_type, search_kwargs={"k": k})
    elif search_type == "tfidf":
        docs = self._get_documents_from_chroma()
        self.retriever = TFIDFRetriever.from_documents(docs, k=k)
    elif search_type == "bm25":
        docs = self._get_documents_from_chroma()
        self.retriever = BM25Retriever.from_documents(docs, k=k)
    elif search_type == "grafo":
        self.retriever = GraphRetriever(
            store=self.vector_store,
            edges=[("keywords", "keywords"), ("author", "author"), ("subject", "subject"), ("title", "title")],
            strategy=Eager(k=k, start_k=1, max_depth=2)
        )
```

Figura 96: Clase Chatbot. Modificar la estrategia de búsqueda

Recuperación y generación de información

La parte dedicada a la interfaz del chatbot consta de varios bloques diferenciados, como se puede observar en la figura 97.

En las primeras dos líneas (85-86) se crea la variable *chat_history* si no se encuentra disponible en la variable de sesión *session_state*, y le agrega un mensaje del sistema dando la bienvenida al chat. En dicha variable iremos guardando todo el historial de conversación entre el usuario y el sistema.

El bucle for de las líneas 88 a 94 recorre cada mensaje disponible en la variable de sesión *chat_history*. Cada mensaje es diferenciado entre sistema y usuario (*AIMessage* y *HumanMessage*), y son impresos con distintos avatares según corresponda.

La variable *user_query* de la línea 96, guarda la consulta hecha por el usuario. Esta consulta es retornada gracias al elemento de *Streamlit chat_input*, que genera una entrada de texto flotante en la interfaz y en la que se puede escribir.

A continuación, dicha entrada de texto se agrega al historial de conversación marcada como mensaje de usuario (*HumanMessage*). Genera la respuesta del sistema llamando a la función *query* (que explicaremos seguidamente), y agrega la respuesta al historial de conversación como *AIMessage*.

```

85     if "chat_history" not in st.session_state:
86         st.session_state.chat_history = [AIMessage(content="¡Hola! ¿En qué puedo ayudarte?")]
87
88     for message in st.session_state.chat_history:
89         if isinstance(message, AIMessage):
90             with st.chat_message("AI", avatar="images/avatar_ai.png"):
91                 st.write(message.content)
92         elif isinstance(message, HumanMessage):
93             with st.chat_message("Human", avatar="images/avatar_user.png"):
94                 st.write(message.content)
95
96     user_query = st.chat_input("Escribe tu mensaje aquí ...")
97     if user_query:
98         st.session_state.chat_history.append(HumanMessage(content=user_query))
99         with st.chat_message("Human", avatar="images/avatar_user.png"):
100             st.markdown(user_query)
101         with st.chat_message("AI", avatar="images/avatar_ai.png"):
102             with st.spinner("Pensando ...", show_time=True):
103                 response = query(user_query)
104             st.session_state.chat_history.append(AIMessage(content=response))

```

Figura 97: Fragmento de código correspondiente a la interfaz del chatbot

En la función *query* vista en la figura 98, suceden ambos procesos de recuperación y generación de información. La recuperación se da entre las líneas 20 a la 23, y la generación en la 42; pero entre ellas se producen otros procesos que dan más detalle y riqueza a la respuesta, y que pasaremos a explicar con detenimiento.

En las líneas 20 y 21 se consigue el modelo de recuperación (*retriever*) usado por el *chatbot*. En el ejemplo se usa un modelo de recuperación con un reranker, el cuál efectuará un doble filtrado de los documentos recuperados por el modelo principal.

El método para iniciar la recuperación se ve en la línea 23. Mediante *invoke*, el *retriever* seleccionará los documentos más relevantes a la consulta proporcionada por el usuario a través del parámetro *question*.

A continuación, guardamos el contexto recuperado por el modelo (línea 84), es decir, el contenido del texto de los documentos recuperados. También guardamos ciertos metadatos (línea 25-39) de los documentos. En el ejemplo, usamos el ID del documento para mostrarlo junto con la respuesta del sistema, ofreciendo así un extra de información y referencias al usuario de donde ha sido obtenida la información.

Para finalizar con la figura 98, queda explicar la generación de información. Se encuentra en la línea 42, realizandose mediante la llamada al método de la clase *Chatbot answer_query* y aportando sólo dos parámetros: la pregunta (*question*) y el contexto (*context*). La respuesta se guarda en la variable *response_text* mediante la función *write_stream* provista por Streamlit, que genera una escritura en tiempo real (*token a token*).

Las variables *response_text* y *sources* son retornadas por esta función, que terminarán siendo agregadas al historial de conversación.

```
def query(question: str):
    chatbot = st.session_state["chatbot"]

    # retriever = chatbot.get_retriever() # normal
    retriever = chatbot.get_compression_retriever() # reranker

    results = retriever.invoke(question)
    context = "\n\n---\n\n".join([doc.page_content for doc in results])
    metadata = [
        {
            "author": doc.metadata.get("author", None),
            "creator": doc.metadata.get("creator", None),
            "id": doc.metadata.get("id", None),
            "keywords": doc.metadata.get("keywords", None),
            "source": doc.metadata.get("source", None),
            "subject": doc.metadata.get("subject", None),
            "title": doc.metadata.get("title", None)
        }
        for doc in results
    ]

    sources_set = {item["id"] for item in metadata if item.get("id")}
    sources = "---\n\n**Referencias**:\n\n" + "\n".join(f"\t{src}" for src in sources_set)

    response_text = st.write_stream(
        chatbot.answer_query(question, context)
    )
    st.write(sources)

    return response_text + "\n\n" + sources
```

Figura 98: Función *query* de la interfaz del chatbot. Hace los procesos de recuperación y generación de información del sistema RAG

La figura 99 muestra el funcionamiento del método *answer_query* de la clase *Chatbot*. En la figura podemos observar el funcionamiento de la cadena (*query_chain*) que nos brinda *LangChain* para trabajar con modelos de lenguaje. En esta cadena podemos combinar el modelo de recuperación, los parámetros necesarios para el *prompt*, la propia plantilla del *prompt* y el modelo de lenguaje. Generando así, la respuesta de forma más simple y eficiente con el comando *stream*, que al igual que antes, genera la respuesta en tiempo real (*token a token*).

```
def answer_query(self, query: str, context: str):
    query_chain = (
        {"context": self.retriever, "question": RunnablePassthrough(), "context": RunnablePassthrough()}
        | self.prompt_template
        | self.language_model
        | StrOutputParser()
    )
    return query_chain.stream({"question": query, "context": context})
```

Figura 99: Método *answer_query* de la clase *Chatbot*, retorna la respuesta del sistema RAG

Para finalizar este apartado, solo queda mostrar el *prompt* usado por defecto por el *chatbot*. La figura 100 muestra la plantilla aplicada, en ella se hace énfasis en el detalle y organización que deben presentar las respuestas. También se observan en las últimas dos líneas, como han de aportarse los argumentos con los que debe generar la respuesta: la pregunta y el contexto.

El planteamiento, instrucciones u órdenes escritas en el *prompt*, pueden generar distintas

respuestas por el modelo de lenguaje. Por lo tanto, hay que buscar la forma adecuada para que el modelo de lenguaje obtenga mejores resultados.

```
default_prompt = """
Eres un asistente que responde preguntas usando SOLO y ÚNICAMENTE el contexto proporcionado.
Si la respuesta NO está en contexto, simplemente di que no lo sabes y no respondas o te inventes la respuesta.
Tus respuestas DEBEN ser detalladas y estar bien estructuradas, organizando la información en párrafos y listas si es necesario.
Tus respuestas DEBEN de ser respondidas con las partes de información sacadas del contexto.
Tus respuestas NO DEBEN mencionar expresiones como "de acuerdo a la información" o "según el contexto".

Si el usuario pide información sobre ayudas, estas también pueden ser subvenciones.
Si el usuario pide información sobre subvenciones, estas también pueden ser ayudas.
Todas las preguntas realizadas por el usuario están relacionadas con la DANA (Depresión Aislada en Niveles Altos) sucedida a finales de 2024.

Aquí tienes la pregunta y el contexto:

Pregunta: {question}\n
Contexto: {context}\n
"""
```

Figura 100: Prompt por defecto usado en el proyecto

Capturas del chatbot

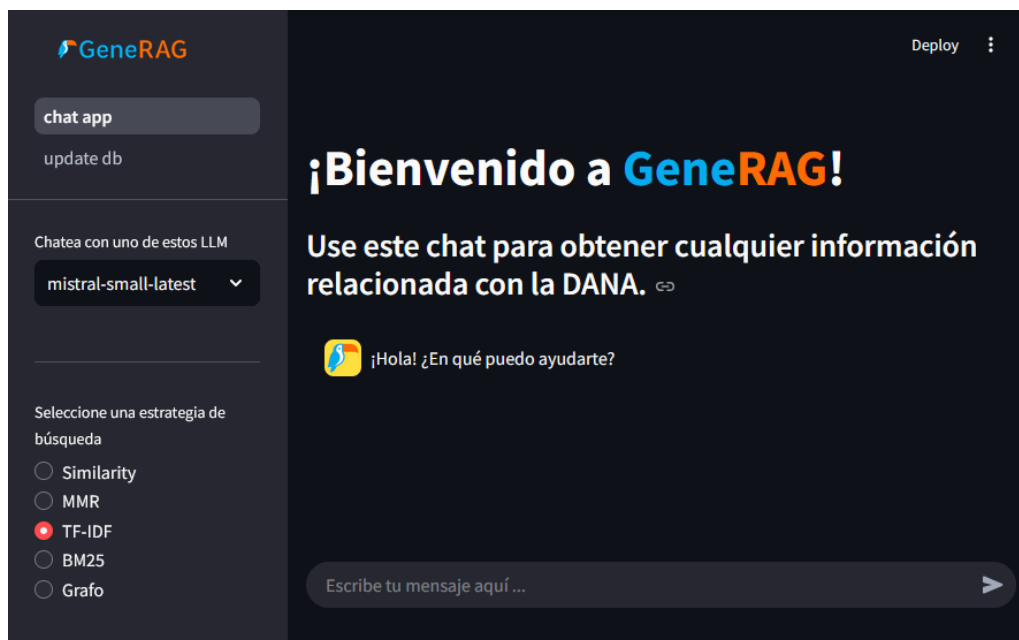


Figura 101: Interfaz web del chatbot



Figura 102: Interfaz web con la barra lateral izquierda plegada

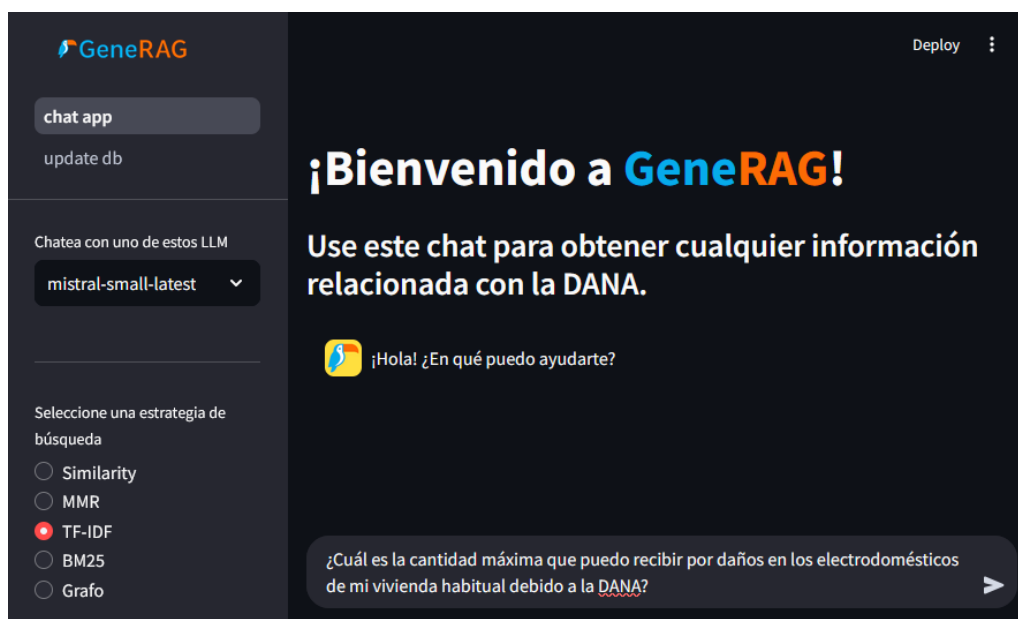


Figura 103: Interfaz web con pregunta

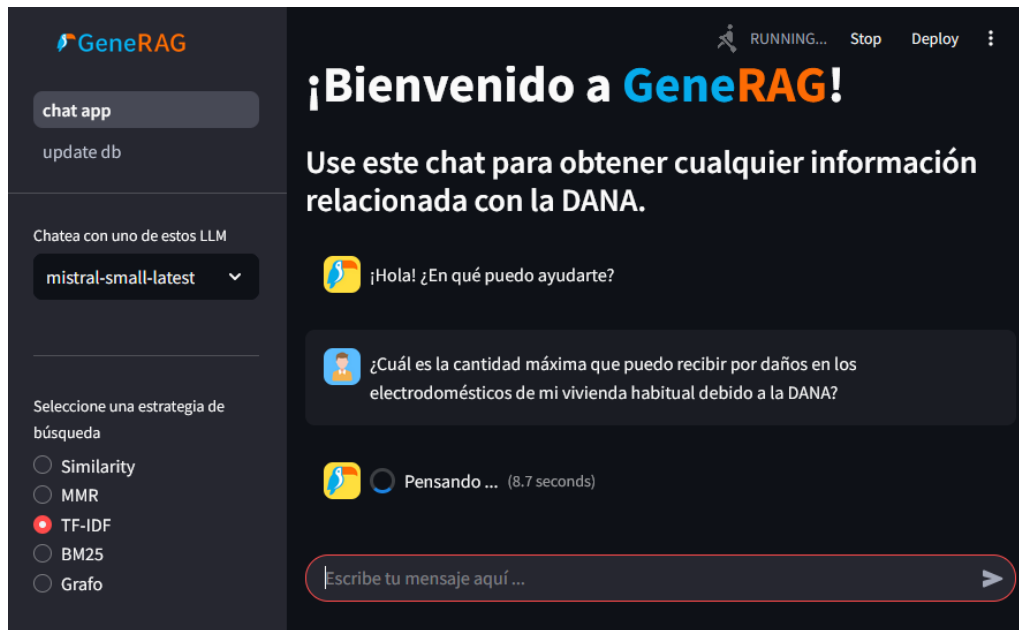


Figura 104: Interfaz web generando respuesta

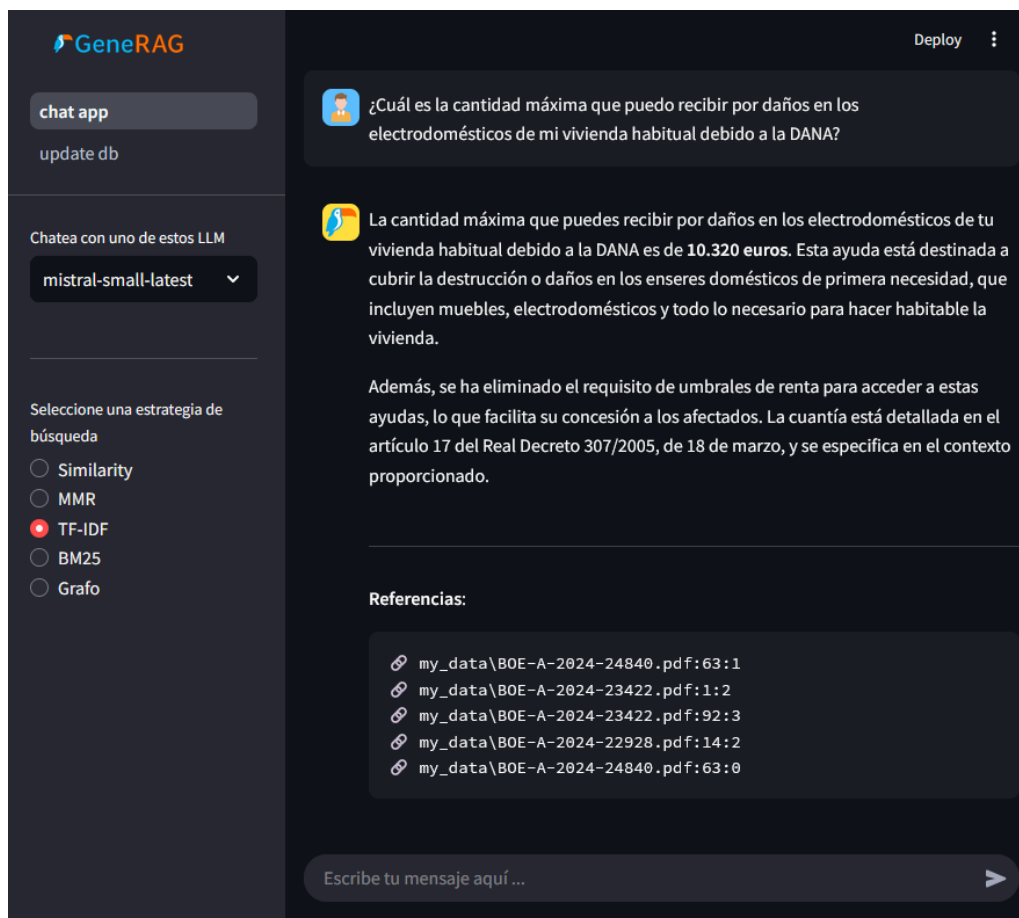


Figura 105: Interfaz web con respuesta generada y referencias

3.5. Conclusiones sobre el desarrollo de la aplicación

El desarrollo de este proyecto ha puesto de manifiesto las claras ventajas de adoptar un enfoque basado en **RAG** frente a la solución planteada inicialmente: RASA. Como hemos comentado, RASA depende de un sistema de *intents* y respuestas predefinidas, que exige un mantenimiento continuo y una anticipación de escenarios de interacción; el modelo con RAG ofrece una mayor versatilidad. Gracias a la integración continua con *GitHub Actions*, la base de conocimiento se actualiza automáticamente con los documentos del BOE, lo que garantiza sincronización con la fuente original y reduce de forma significativa las tareas de mantenimiento.

La incorporación de *LangChain* resultó clave en la integración de los distintos componentes, facilitando las tareas de recuperación y generación de información. El proceso de desarrollo estuvo además favorecido por la abundante documentación y los numerosos tutoriales disponibles, lo que permitió una rápida implementación y conexión con diferentes plataformas de modelos de lenguaje.

Por su parte, la interfaz desarrollada con *Streamlit* ofreció una experiencia moderna y práctica. Su capacidad de generar aplicaciones web inyectando código *Python* simplificó el trabajo, al permitir el uso de elementos interactivos habituales sin necesidad de programación adicional desde el *backend*. De esta manera, se aceleró la construcción de la interfaz, la personalización y la adaptación del sistema a las necesidades específicas del proyecto.

En conjunto, el nuevo enfoque proporciona un sistema más preciso, rápido y escalable. Así, se superan las limitaciones del marco anterior y se establece una solución sólida, eficiente y preparada para su evolución futura.

Capítulo 4 : Evaluación del Proyecto

En este capítulo se describen los procesos realizados para evaluar nuestro sistema RAG. Para ello se ha generado un conjunto de preguntas junto a varios agentes críticos, sobre los que se evalúa la generación de respuesta de los modelos de lenguaje elegidos en el proyecto (ver sección 2.2.1). El objetivo de realizar estas pruebas es mostrar el rendimiento de diferentes modelos de lenguaje sobre un mismo dataset de preguntas.

En las siguientes secciones explicamos como se ha generado el conjunto de preguntas y como se han definido los agentes críticos para evaluar el RAG. También mostramos los resultados obtenidos, razonando el porqué de ellos.

Y para finalizar el capítulo, se elaboran posibles casos de uso para nuestro sistema RAG y respondemos algunas preguntas relacionadas con el impacto causado al cambiar ciertos parámetros de un sistema RAG.

4.1. Generación del conjunto de preguntas

En la creación del conjunto de preguntas, hemos optado por usar el modelo de lenguaje *mistral-small* para generar preguntas basadas en nuestra fuente de conocimiento (documentos PDF del Boletín Oficial del Estado relacionados con la DANA).

En la siguiente figura, vemos la configuración del modelo de lenguaje *mistral-small* que hemos usado. Al ser un modelo externo, necesitamos aportar su clave de acceso. Si nos fijamos en la figura 106, le hemos indicado que sea un poco creativo en la generación del par pregunta-respuesta aumentando el parámetro de *temperature*.

```
llm = ChatMistralAI(  
    model="mistral-small-latest",  
    mistral_api_key=MISTRAL_API_KEY,  
    temperature=0.1,  
    random_seed=12345  
)
```

Figura 106: Configuración LLM de generación del conjunto de preguntas

El siguiente paso en la generación del conjunto de preguntas es facilitar al modelo de lenguaje las instrucciones necesarias mediante un *prompt* para generar el conjunto de preguntas.

Fijándose en la figura 107, el prompt hace especial hincapié en la necesidad del usuario de generar una única pregunta con su respuesta, siempre estando relacionado con la DANA y dado un determinado contexto. La pregunta debe mantener un formato muy específico y la salida debe presentarse separado por comas (para su posterior procesado en formato csv).

El contexto aportado al final del prompt se refiere al contenido en texto de cada página de cada documento presente en la fuente de conocimiento, por lo que es necesario ejecutar el prompt varias veces en bucle. El número de ficheros que integraban dicha fuente en el

momento de la generación de este conjunto de preguntas y respuestas era de alrededor de 80 con un total de más de 560 páginas.

```
QA_generation_prompt = """
Tu tarea es generar una ÚNICA pregunta con su respectiva respuesta basándote únicamente en el contexto aportado.
Tus preguntas deben estar relacionadas con la DANA (Depresión Aislada en Niveles Altos) ocurrida a finales de 2024.
Las preguntas generadas deben tener un formato de preguntas frecuentes (FAQ).
Debes formular tus preguntas imitando el estilo que usaría una persona afectada por la catástrofe buscando información.
Tus preguntas deben poder ser respondidas usando una pieza específica del contexto.
No debes mencionar cosas como "de acuerdo a la información" o "según el contexto".
Escribe en ESPAÑOL.
---
Realiza tu respuestas como sigue, manteniendo un formato separado por comas (csv):

"tu pregunta","tu respuesta"

---
Contexto: {context}\n
Output::"""
```

Figura 107: Prompt usado en la generación del conjunto de preguntas

Para añadir los documentos hemos vuelto a usar la clase *PyPDFDirectoryLoader*, pasando como argumento la carpeta donde hemos descargado con anterioridad los ficheros de nuestra fuente de conocimiento. La figura 108 muestra este proceso y el de configuración inicial de la cadena *chain*, siendo similar a lo mostrado con anterioridad en otros apartados.

```
documents = PyPDFDirectoryLoader("../my_data").load()
prompt = ChatPromptTemplate.from_template(QA_generation_prompt)
chain = prompt | llm | StrOutputParser()
```

Figura 108: Carga de datos y preparación del modelo de lenguaje

El bucle antes mencionado, en el cual se deben generar todas las preguntas se puede apreciar en la figura 109. En dicha figura, se crea un fichero para albergar el conjunto entero de preguntas-respuestas, donde cada línea del fichero contendrá:

- **Pregunta:** La pregunta generada por el modelo de lenguaje.
- **Respuesta:** La respuesta a la pregunta generada por el modelo de lenguaje.
- **Contexto:** El contexto del cual se ha generado la pregunta.
- **Recurso:** El fichero del cual se ha conseguido el contexto.

```
with open("eval/preguntas_mistral_plus2.csv", "w", encoding="utf-8") as f:
    f.write("Pregunta,Respuesta,Contexto,Recurso\n")
    for doc in tqdm(documents, desc="Generando hoja de preguntas", unit="páginas"):
        page_content = doc.page_content.replace('\n', ' ').replace(' ', '')
        response = chain.invoke({"context":page_content})
        source = doc.metadata["source"]
        line = f'{response},{page_content},{source}'
        f.write(line + "\n")
        time.sleep(2)
```

Generando hoja de preguntas: 100% | 569/569 [1:09:40<00:00, 7.35s/páginas]

Figura 109: Bucle para la generación del conjunto de preguntas

La generación de este conjunto de preguntas ha tenido una duración en tiempo de una hora

y diez minutos aproximadamente. La inclusión en el bucle de la parada (*time.sleep(2)*) de dos segundos fue necesario para evitar el error por sobrecarga del modelo (recordemos que es un modelo externo ofrecido de forma gratuita por *Mistral AI*). Sin esta parada, la duración habría sido de alrededor de unos cincuenta minutos.

El resultado final de la ejecución esta disponible en el recurso [64] de la bibliografía. El número final de preguntas se redujo a 300 debido a que existía bastante repetición entre las preguntas generadas. Una muestra de las quince primeras líneas de este fichero pueden observarse en la siguiente figura.

	A	B	C	D
1	Pregunta	Respuesta	Contexto	Recurso
2	¿Cuáles fueron las comunidades autónomas más afectadas por la DANA?	Las comunidades autónomas más afectadas fueron la Comunitat Valenciana, Castilla-La Mancha, Andalucía, Cataluña y, en menor medida, Illes Balears y Aragón.	I. DISPOSICIONES GENERALES JEFATURA DEL ESTADO 22928 Real De	./_my_data/BOE-A-2024-22928.pdf
3	¿Cuántos efectivos militares y policías han sido desplegados para ayudar en la DANA?	Hasta la fecha, se han desplegado más de 7 500 efectivos militares y 10 000 policías nacionales y guardias civiles para asistir en la DANA de finales de 2024.	El Estado ha desplegado hasta la fecha más de 7 500 efectivos militares y 1	./_my_data/BOE-A-2024-22928.pdf
4	¿Qué comunidades autónomas han sido declaradas zona afectada gravemente por la DANA?	Las comunidades autónomas declaradas zona afectada gravemente por la DANA de 2024 son Comunitat Valenciana, Castilla-La Mancha, Andalucía, Illes Balears y Aragón.	de 2024 ha acordado declarar zona afectada gravemente por una emergenc	./_my_data/BOE-A-2024-22928.pdf
5	¿Puedo solicitar una moratoria para mis préstamos hipotecarios si mi vivienda fue afectada por la DANA?	Si, puedes solicitar una moratoria para tus préstamos hipotecarios si tu vivienda está situada en la zona declarada como afectada gravemente por la DANA y ha sufrido daños o has tenido que ser realojado. Esta moratoria cubre tanto a personas físicas como a trabajadores por cuenta propia y empresas con un volumen de facturación inferior a 6 millones de euros en el último ejercicio cerrado.	A su vez, se faculta a las empresas y trabajadores por cuenta propia a solici	./_my_data/BOE-A-2024-22928.pdf
6	¿Qué medidas se han establecido para los contratos de arrendamiento financiero o «leasing» debido a la DANA?	La moratoria se extiende a los contratos de arrendamiento financiero, conocidos como «leasing» o «leasing financiero», para facilitar la financiación de la actividad económica de las personas trabajadoras por cuenta propia afectadas por la DANA.	por una emergencia de protección civil y para las personas trabajadoras por	./_my_data/BOE-A-2024-22928.pdf
7	¿Qué medidas específicas contempla el real decreto-ley aprobado en noviembre de 2024 para enfrentar los efectos de la DANA?	La norma contempla medidas perentorias en materia tributaria, de seguridad social, o económica, con el fin de subvenir y mitigar los severos impactos que esta catástrofe natural ha tenido en el ámbito socioeconómico. Además, se regulan aspectos indispensables para hacer frente a los efectos devastadores que la DANA ha provocado en la vida de miles de personas, incluyendo daños personales, en viviendas, enseres o establecimientos.	libertades de los ciudadanos regulados en el Título I de la Constitución, al ré	./_my_data/BOE-A-2024-22928.pdf
8	¿Cuál es la cantidad máxima de ayuda que puedo recibir si mi vivienda habitual fue destruida por completo?	La cuantía máxima de ayuda por la destrucción total de la vivienda habitual es de 60 400 euros.	CAPÍTULO II Medidas de apoyo en materia de daños personales, vivienda,	./_my_data/BOE-A-2024-22928.pdf
9	¿Cuál es el plazo para presentar las solicitudes de ayudas por daños personales y a las de personas físicas o jurídicas que hayan efectuado prestaciones personales o de bienes debido a la DANA?	Las solicitudes para la concesión de estas ayudas se presentarán en el plazo de dos meses, contados a partir de la publicación en el «Boletín oficial del Estado» de este real decreto-ley.	aportar entonces las certificaciones acreditativas del cumplimiento de los rec	./_my_data/BOE-A-2024-22928.pdf
10	¿Cómo se financiarán las ayudas para la DANA?	Las ayudas se financiarán con cargo a los créditos de los conceptos 461 y 761 de la aplicación presupuestaria 16.01.9290 «Contingencias asociadas a la Depresión Alada en Niveles Altos (DANA) de 2024», dotados con carácter de ampliables en el vigente presupuesto del Ministerio del Interior.	3. Las ayudas que se concedan en aplicación de lo previsto en este artículo	./_my_data/BOE-A-2024-22928.pdf
11	¿Hasta cuándo se extienden los plazos de presentación o ingreso de declaraciones y autoliquidaciones tributarias afectadas por la DANA?	Los plazos de presentación e ingreso de las declaraciones y autoliquidaciones tributarias cuyo vencimiento se encuentre entre el 28 de octubre y el 31 de diciembre de 2024, se extenderán hasta el 30 de enero de 2025.	Artículo 7. Tramitación por el Consorcio de Compensación de Seguros de	./_my_data/BOE-A-2024-22928.pdf
12	¿Hasta cuándo se extenderán los plazos para la presentación e ingreso de declaraciones-liquidaciones y autoliquidaciones?	Los plazos para la presentación e ingreso de declaraciones-liquidaciones y autoliquidaciones afectadas por la DANA de 2024 se extenderán hasta el 30 de enero de 2025.	extenderán hasta el 30 de enero de 2025, salvo que el plazo otorgado por la	./_my_data/BOE-A-2024-22928.pdf
13	¿Hasta cuándo se extenderán los plazos para interponer recursos de reposición o reclamaciones económico-administrativas debido a la DANA?	Los plazos para interponer recursos de reposición o reclamaciones económico-administrativas contra actuaciones de la Agencia Estatal de Administración Tributaria o referidas a tributos o derechos aduanares gestionados por ésta, relativas a los obligados tributarios que cumplan los requisitos del párrafo primero del apartado 1, se extenderán hasta el 30 de enero de 2025, salvo en estos casos que el otorgado por la norma general sea mayor, en cuyo caso éste resultará de aplicación.	atendiera al requerimiento o solicitud de información con trascendencia tribu	./_my_data/BOE-A-2024-22928.pdf
14	¿Hasta cuándo se extienden los plazos para presentar declaraciones catastrales?	Los plazos de presentación de declaraciones catastrales cuyo vencimiento se produzca entre el 28 de octubre y el 31 de diciembre de 2024, se extenderán hasta el 30 de enero de 2025.	Artículo 10. Medidas en materia catastral en el ámbito de competencias de	./_my_data/BOE-A-2024-22928.pdf
15	¿Hasta cuándo puedo presentar el formulario para recibir la ayuda por la DANA?	El formulario podrá presentarse desde el 19 de noviembre al 31 de diciembre de 2024.	ponga a disposición la Agencia Estatal de Administración Tributaria y en el q	./_my_data/BOE-A-2024-22928.pdf

Figura 110: Muestra del conjunto de preguntas [64]

Nota: El fichero de generación de preguntas esta disponible en la raíz del proyecto [58], bajo el nombre **qa_generator.ipynb** (formato Jupyter Notebook).

4.2. Configuración de las alternativas de evaluación

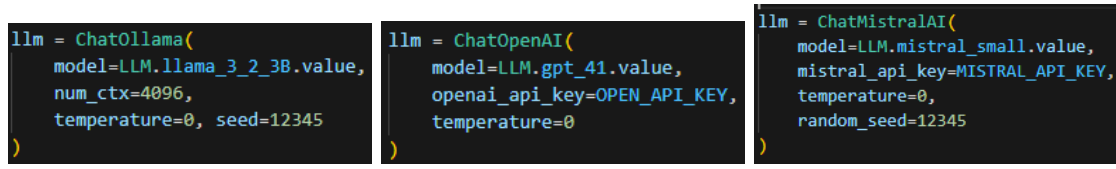
En este apartado veremos la parte que precede al segundo objetivo de este Trabajo de Fin de Máster, donde mostraremos la configuración de las opciones utilizadas para las pruebas del sistema RAG. Empezaremos hablando de los modelos usados para dichas pruebas, los cuales se encargarán tanto de generar las respuestas como de evaluarlas.

Nota: El código de las siguientes figuras se puede encontrar en el fichero **rag_eval.ipynb** del repositorio [58].

4.2.1. Modelos evaluadores y configuración inicial

Como ya hemos comentado en la última sección del capítulo 2.10, se harán estas pruebas en tres modelos: el modelo gratuito y local *llama3.2*, el modelo gratuito pero

externo *mistral-small* y el modelo externo y de pago *gpt-4.1*. La siguiente figura muestra tres imágenes de la configuración de cada uno de estos modelos usando *LangChain*. Como es común para los modelos externos, debemos proporcionar su clave API para poder usarlos.



(a) Llama 3.2

```
llm = ChatOllama(
    model=LLM.llama_3_2_3B.value,
    num_ctx=4096,
    temperature=0, seed=12345
)
```

(b) GPT-4.1

```
llm = ChatOpenAI(
    model=LLM.gpt_41.value,
    openai_api_key=OPEN_API_KEY,
    temperature=0
)
```

(c) Mistral Small 3.2

```
llm = ChatMistralAI(
    model=LLM.mistral_small.value,
    mistral_api_key=MISTRAL_API_KEY,
    temperature=0,
    random_seed=12345
)
```

Figura 111: Configuración de los modelos de lenguaje de pruebas

La variable *model* común para todas las configuraciones de la figura anterior, contiene elementos de la clase *LLM*. Estos elementos se corresponden con los nombres reales de los modelos que son usados para su acceso en la API y ya han sido mostrados en la figura 92.

El siguiente paso es configurar las demás variables de pruebas, la siguiente figura muestra alguna de ellas. En la primera línea podemos observar las estrategias de búsqueda que se probarán como mencionábamos en uno de los apartados de la sección 2.10. También observamos en la figura un *prompt* que usaremos para medir la influencia en los resultados de un cambio en las instrucciones, en este caso más simples.

```
search_type = "tfidf" # ["similarity", "mmr", "tfidf", "bm25", "grafa"]

llm_eval = "gpt-4.1"
chroma_eval = "te3s"

prompt_test = """
Eres un asistente que responde preguntas frecuente (FAQ) usando SOLO y ÚNICAMENTE el contexto proporcionado.
Si la respuesta NO está en contexto, simplemente dí que no lo sabes y no respondas o te inventes la respuesta.
Tus respuestas deben ser claras, concisas y directas, sin rodeos ni explicaciones innecesarias.

Aquí tienes la pregunta y el contexto:

Pregunta: {question}\n

Contexto: {context}\n

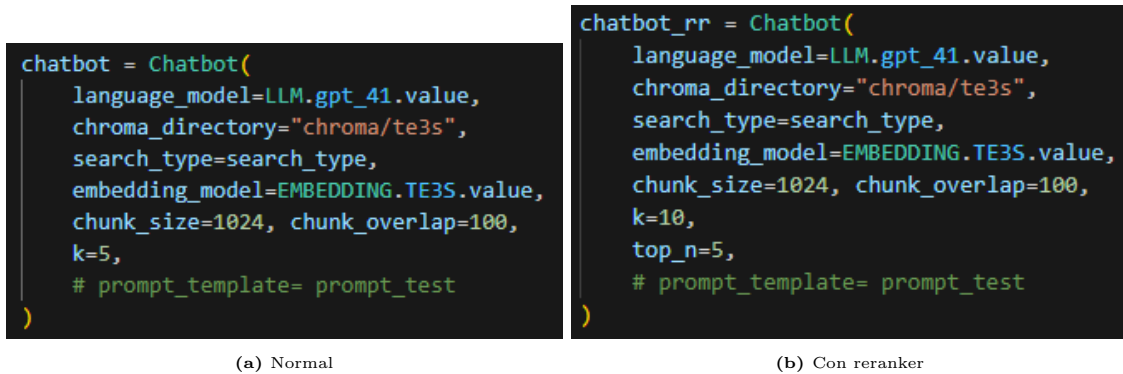
"""
```

Figura 112: Variables de pruebas

Las pruebas las realizaremos con dos *chatbots*, la figura 113 muestra la configuración de ambos. Los dos son exactamente iguales siendo el único cambio, el *reranker* usado en el segundo de ellos (parte b). El caso de esta figura muestra una configuración de pruebas para modelos de *OpenAI*, como se puede observar tanto por el modelo de lenguaje usado (*gpt-4.1*) como por el de embedding (*text-embedding-3-small*).

Los modelos de embedding que probaremos están incluidos en la clase *EMBEDDING* (ya mostrada en la figura 91). El objetivo de la prueba de distintos modelos de embedding es dilucidar cual proporciona mejores resultados en la recuperación en conjunto con la generación de los modelos de lenguaje.

Nota: Las clases *LLM* y *EMBEDDING* se encuentran en la carpeta *classes* del repositorio [58].



```

chatbot = Chatbot(
    language_model=LLM.gpt_41.value,
    chroma_directory="chroma/te3s",
    search_type=search_type,
    embedding_model=EMBEDDING.TE3S.value,
    chunk_size=1024, chunk_overlap=100,
    k=5,
    # prompt_template= prompt_test
)

```

(a) Normal

```

chatbot_rr = Chatbot(
    language_model=LLM.gpt_41.value,
    chroma_directory="chroma/te3s",
    search_type=search_type,
    embedding_model=EMBEDDING.TE3S.value,
    chunk_size=1024, chunk_overlap=100,
    k=10,
    top_n=5,
    # prompt_template= prompt_test
)

```

(b) Con reranker

Figura 113: Configuración de los chatbots de pruebas

4.2.2. Agentes críticos

Los agentes críticos son los encargados de realizar una evaluación de las respuestas generadas por los modelos de lenguaje. Las siguientes figuras muestran los *prompts* con las instrucciones de los cuatro agentes críticos seleccionados para nuestras evaluaciones, y que están definidos en la sección 2.10.

El primero de ellos es el *accuracy* (figura 114), que es el encargado de evaluar la precisión de la respuesta generada por el modelo comparándola con la respuesta original. La figura muestra el *prompt* con las instrucciones necesarias para que el modelo haga de evaluador comparando dos argumentos, e incluso, añadimos los criterios que tiene que seguir para asignar una nota.

Todos los agentes evalúan de cero a cinco, siendo cero la peor nota y cinco la mejor.

```

accuracy_prompt = """
Eres un agente crítico experto en evaluación de precisión de respuestas.
Evalúa la precisión de la respuesta generada en comparación con la respuesta esperada.

Criterios de evaluación:
- 5: La respuesta generada incluye toda la información esencial de la respuesta esperada, incluso si es más extensa o utiliza otras palabras.
- 4: La respuesta generada incluye casi toda la información importante, con leves omisiones o diferencias menores que no afectan el significado general.
- 3: La respuesta generada cubre parte significativa del contenido, pero falta información importante o hay errores leves.
- 2: La respuesta generada solo refleja una parte limitada del contenido de la respuesta esperada y tiene omisiones o distorsiones relevantes.
- 1: La respuesta generada tiene muy poca relación con la respuesta esperada, presenta errores graves u omisiones importantes.
- 0: La respuesta generada no contiene información relevante de la respuesta esperada o está completamente incorrecta.

Recuerda: una redacción diferente o más extensa no debe penalizar si toda la información está presente.

respuesta esperada: {respuesta_esperada}

respuesta generada: {respuesta_generada}

Responde solo con tu evaluación en el siguiente formato:

Respuesta::
Rating: (tu evaluación del 0 al 5)
Respuesta::
"""

```

Figura 114: Agente crítico del accuracy

El siguiente agente mide el *faithfulness* (figura 115). Esta medida evalúa el grado en que una respuesta esta basada en la información contenida en los documentos recuperados, por eso recibe estos dos argumentos. Gracias a esta medición obtenemos la fidelidad de la respuesta con los documentos de nuestra base de conocimiento.

```

faithfulness_prompt = """
Tu tarea es evaluar si la respuesta generada usa EXCLUSIVAMENTE información contenida en los documentos proporcionados, sin añadir datos externos, suposiciones infundadas ni invenciones.

Criterios de evaluación:
5 - Totalmente basada en los documentos, sin añadir nada externo.
4 - Principalmente basada, con pequeñas generalizaciones o frases neutras.
3 - Incluye inferencias plausibles pero no explícitas en los documentos.
2 - Mezcla partes basadas en los documentos con invenciones o suposiciones.
1 - Mayormente inventada, aunque con algún detalle correcto.
0 - Completamente inventada, contradictoria o irrelevante.

Aquí tienes los documentos recuperados y la respuesta generada por el sistema RAG.

Documentos recuperados: {documentos}

Respuesta generada: {respuesta}

Responde solo con tu evaluación en el siguiente formato:

Respuesta::
Rating: (tu evaluación del 0 al 5)
Respuesta::
"""

```

Figura 115: Agente crítico del faithfulness

El *groundedness* es una medida similar a la anterior (figura 116), esta valora el uso de las evidencias aportadas por los documentos recuperados en la respuesta generada por el modelo. Es importante para mostrar lo bien (o mal) que usan la información disponible en los documentos recuperados a la hora de generar la respuesta.

```

groundedness_prompt = """
Tu tarea es evaluar en qué medida la respuesta está respaldada por evidencia específica de los documentos proporcionados.

Criterios de evaluación:
5 - Usa varias evidencias clave con precisión y contexto adecuado.
4 - Usa al menos una evidencia clave de forma clara y correcta.
3 - Se refiere a los documentos de manera vaga o genérica.
2 - Uso superficial, indirecto o débilmente relevante de los documentos.
1 - Menciona información incorrecta o mal interpretada del documento.
0 - No hace uso de los documentos; ignora la evidencia disponible.

Aquí tienes los documentos recuperados y la respuesta generada por el sistema RAG.

Documentos recuperados: {documentos}

Respuesta generada: {respuesta}

Responde solo con tu evaluación en el siguiente formato:

Respuesta::
Rating: (tu evaluación del 0 al 5)
Respuesta::
"""

```

Figura 116: Agente crítico del groundedness

El último agente se encarga de la relevancia (*relevance*, figura 117). La relevancia considera lo buenos que son los documentos recuperados para responder a la pregunta hecha y si se puede responder a ella de forma correcta, por eso recibe esos tres argumentos: pregunta, respuesta esperada y contexto (los documentos recuperados). Con esta medida podemos hacernos una idea de lo bien que funciona el modelo de recuperación de información (modelo de embedding) a la hora de responder preguntas, aportando documentos de calidad.


```

relevance_prompt = """
Tu tarea es evaluar qué tan relevantes son los documentos para responder la pregunta.

Criterios de evaluación:
5 - Abordan directamente el núcleo de la pregunta
4 - Cubren aspectos importantes pero no todos
3 - Proporcionan contexto general útil
2 - Tienen conexión tangencial con el tema
1 - Mínima relación con la pregunta
0 - Completamente irrelevantes

Aquí tienes la pregunta y los documentos recuperados.

Pregunta: {pregunta}

Respuesta esperada: {respuesta}

Documentos recuperados: {documentos}

Responde solo con tu evaluación en el siguiente formato:

Respuesta::
Rating: (tu evaluación del 0 al 5)
Respuesta::
"""

```

Figura 117: Agente crítico de la relevancia

4.2.3. Generación de evaluaciones

En este apartado abordaremos los pasos que hemos seguido para generar las evaluaciones de nuestro sistema RAG. Tal como sucedió en el apartado en el que generamos las preguntas de test (sección 4.1), necesitamos ejecutar un bucle en el que sean lanzadas todas las evaluaciones para cada pregunta. Esto incluye los siguientes pasos:

1. Generar la respuesta normal.
2. Lanzar las evaluaciones a las respuestas normales.
3. Generar la respuesta con reranking.
4. Lanzar las evaluaciones a las respuestas con reranking.
5. Guardar ambos resultados en formato csv.

Usaremos como dataset de pruebas el conjunto de preguntas generado en la sección anterior. En la figura 118 cargamos dicho conjunto en una variable ayudándonos de la clase *pandas*, que permite una fácil lectura de ficheros csv.

```

df = pd.read_csv("eval/preguntas-mistral.csv", on_bad_lines="skip")
# df_shuffle = df.sample(frac=1, random_state=12345)
# test_sample = df_shuffle.head(30) #[30, 75, 150]
test_sample = df

```

Figura 118: Carga de dataset de preguntas

Antes de empezar el bucle de evaluaciones, necesitamos crear las variables que guardarán los resultados. La siguiente figura define dos variables para este propósito: *eval_rag* para las evaluaciones normales y *eval_rag_rr* para las evaluaciones con *reranking*. También se observa un *array* de columnas, que será usado más tarde a la hora de guardar los resultados en ficheros.

```
columns = [
    "pregunta", "respuesta esperada", "respuesta generada",
    "accuracy", "faithfulness",
    "groundedness", "relevance"
]

eval_rag = []
eval_rag_rr = []
```

Figura 119: Columnas para el fichero de la evaluación y variables que contendrán los resultados de las evaluaciones

Las funciones vistas en la figura 121 son usadas dentro del bucle de preguntas para obtener la respuesta generada por el modelo de lenguaje y el contexto recuperado por el modelo de embedding. La primera, *get_response*, es usada para obtener los datos del modelo normal; y la segunda, obtiene los del modelo con *reranking*. Ambas siguen una estructura similar pero muy simplificada de las usadas en la interfaz del *chatbot*.

```
def get_response(question: str, chatbot: Chatbot):
    results = chatbot.get_retriever().invoke(question)
    context = "\n\n---\n\n".join([doc.page_content for doc in results])

    response = chatbot.answer_query2(question, context)

    return response, context

def get_response_reranker(question: str, chatbot: Chatbot):
    results = chatbot.get_compression_retriever().invoke(question)
    context = "\n\n---\n\n".join([doc.page_content for doc in results])

    response = chatbot.answer_query2(question, context)

    return response, context
```

Figura 120: Función *get_response* y *get_response_rr*, ambas usadas para generar respuestas en las pruebas

El primer fragmento del bucle se puede apreciar en la siguiente figura. La imagen muestra el proceso que se sigue para cada pregunta, extrayendo y guardando los valores *pregunta*, *respuesta* y *contexto* (este último no es usado). También genera una respuesta para cada pregunta y obtiene los documentos usados para construir la respuesta, ya que son necesarios para algunas evaluaciones de los agentes, que son formateados a continuación.

```
for index, row in tqdm(test_sample.iterrows(), desc="Evaluando el RAG", total=len(test_sample), unit="pregunta"):
    pregunta = row["Pregunta"]
    respuesta = row["Respuesta"]
    contexto = row["Contexto"]
    ### NORMAL ###

    time.sleep(sleep) # Espera para evitar sobrecargar el servidor
    respuesta_generada, docs_recuperados = get_response(pregunta, chatbot)

    evaluations = {
        "accuracy": accuracy_prompt.format(respuesta_esperada=respuesta, respuesta_generada=respuesta_generada),
        "faithfulness": faithfulness_prompt.format(documentos=docs_recuperados, respuesta=respuesta_generada),
        "groundedness": groundedness_prompt.format(documentos=docs_recuperados, respuesta=respuesta_generada),
        "relevance": relevance_prompt.format(pregunta=pregunta, respuesta=respuesta, documentos=docs_recuperados)
    }
```

Figura 121: Fragmento 1 del bucle de evaluación de preguntas: se guardan datos del dataset de preguntas, se obtienen la respuesta generada y el contexto, y se formatean los agentes críticos

El proceso para generar las respuestas y evaluaciones del modelo con *reranking* es exactamente igual, en el que son usadas las mismas variables pero usando la terminación *_rr*. Por

ejemplo, las evaluaciones de los agentes en la figura 121 son guardadas en la variable *evaluations*, así pues, en las del modelo con *reranking* la variable se denomina *evaluations_rr*.

Siguiendo con el proceso de generación de evaluaciones, queda lanzar los agentes. El proceso se puede observar en la figura 122. La imagen comienza creando una variable *eval_row* que contiene en principio la pregunta, respuesta original y respuesta generada, para en el siguiente bucle ir asignando y guardando uno a uno los resultados de las evaluaciones de los agentes. El bucle termina almacenando la variable *eval_row* en los resultados finales.

Tal como ocurrió antes, el proceso de evaluación de los modelos con *reranking* es exactamente igual, y las variables donde se almacenan sus resultados tienen la terminación *_rr*.

```
try:
    eval_row = {
        "pregunta": pregunta,
        "respuesta esperada": respuesta,
        "respuesta generada": respuesta_generada
    }
    for criterion, evaluation in evaluations.items():
        time.sleep(sleep)
        chain = llm | StrOutputParser()
        response = chain.invoke(evaluation)
        match = re.search(r'Rating:\s*(\d+)', response)
        score = int(match.group(1))
        eval_row[criterion] = score
    eval_rag.append(eval_row)
```

Figura 122: Fragmento 2 del bucle de evaluación de preguntas. Se van generando una a una las evaluaciones de los agentes

Los resultados finales de ambos *chatbots* son guardados individualmente y son impresos en formato csv, tal y como se indica en la siguiente figura. Hemos usado las variables de la figura 112 para diferenciar las evaluaciones.

```
eval_rag_df = pd.DataFrame(eval_rag, columns=columns)
eval_rag_df.to_csv(f"eval/{llm_eval}/{chroma_eval}/csv/eval_{search_type}.csv", index=False)
eval_rag_df_rr = pd.DataFrame(eval_rag_rr, columns=columns)
eval_rag_df_rr.to_csv(f"eval/{llm_eval}/{chroma_eval}/csv/eval_{search_type}_rr.csv", index=False)
```

Figura 123: Impresión de los resultados de los modelos en formato csv

Al realizar las evaluaciones, obtendremos varios ficheros con los resultados de cada una de ellas. Una muestra de lo que puede ocurrir se observa en la figura siguiente. Donde hemos realizado todas las evaluaciones disponibles para el conjunto *GPT-4.1* y *text-embedding-3-small*, resultando en diez ficheros.

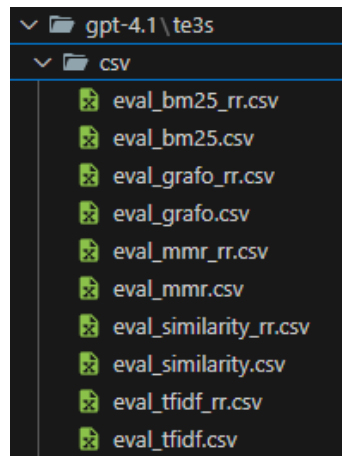


Figura 124: Contenido de la carpeta de evaluaciones *GPT-4.1* en *text-embedding-3-small*

Cada fichero contiene la misma estructura, en la figura 125 podemos ver el contenido del fichero correspondiente a la evaluación de búsqueda TF-IDF, sobre el par *GPT-4.1/text-embedding-3-small*. En este fragmento se puede apreciar tres preguntas con sus respectivas respuestas originales y generadas, y también, los resultados numéricos aportados por los agentes críticos.

```

1 pregunta,respuesta esperada,respuesta generada,accuracy,faithfulness,groundedness,relevance
2 ¿Cómo se concederá la subvención de intereses para los préstamos asociados a la DANA?, "La subvención de intereses se concederá en el mismo acto de concesión del
Economía, Comercio y Empresa.", "La subvención de intereses para los préstamos asociados a la DANA se concederá de la siguiente manera:
3
4 1. **Condiciones y criterios de concesión**
5 - Las condiciones específicas de la subvención, incluyendo los requisitos para los beneficiarios, el valor máximo por beneficiario y los criterios de concesión
6 - La concesión de la subvención será directa, no en régimen de concurrencia competitiva. Esto significa que se otorgará a los beneficiarios que cumplan los requisitos
7 - La adjudicación se realizará atendiendo estrictamente a criterios temporales, según la fecha de la solicitud o de la formalización del préstamo, en los términos
8
9 2. **Procedimiento de concesión**
10 - En el mismo acto de concesión del préstamo por parte de la entidad de crédito, conforme a la regulación establecida, se entenderá concedida la subvención de
11 - No será necesaria una resolución de concesión específica por parte del Ministerio de Economía, Comercio y Empresa.
12
13 3. **Verificación de requisitos**
14 - La verificación del cumplimiento de los requisitos por parte de los beneficiarios de la subvención se realizará posteriormente ("ex post") por el Ministerio de
15
16 En resumen, la subvención de intereses se concede automáticamente en el momento de la formalización del préstamo, sin necesidad de una resolución adicional, y se
17 disponibles. La comprobación de que los beneficiarios cumplen los requisitos se realiza después de la concesión.",5,5,5,5
18 ¿Hasta cuándo se extiende la medida de flexibilización de los contratos de suministro de gas natural debido a la DANA?, "La medida de flexibilización de los contratos
contratos de suministro de gas natural debido a la DANA se extiende hasta el 31 de diciembre de 2025. Durante este periodo, los titulares de puntos de suministro
modificación del caudal diario contratado, la suspensión temporal o la resolución de sus contratos de suministro, sin incurrir en costes adicionales. Esta flexi
los procesos productivos a la situación excepcional generada por la DANA, minimizando el impacto económico de las limitaciones normativas habituales.",5,5,5,5
19 ¿Hasta cuándo puedo solicitar la modificación de las condiciones de mi proyecto subvencionado por la DANA en los municipios afectados?, "Puedes solicitar la modi
actividad, siempre que no dañe derechos de tercero.", "Las entidades beneficiarias de ayudas o subvenciones para proyectos relacionados con la DANA en los municipi
20
21 - Se permite solicitar, con carácter excepcional, la modificación del contenido de la orden de concesión cuando concurren circunstancias sobrevenidas que alteren
de concesión.
22 - El periodo de ejecución de las actuaciones subvencionadas comprende desde el 28 de octubre de 2024 hasta el 31 de diciembre de 2025. Por tanto, la solicitud de
23 - En caso de que surjan circunstancias debidamente justificadas que alteren las condiciones recogidas en el otorgamiento de la financiación, el beneficiario pod
24
25 En resumen, puedes solicitar la modificación de las condiciones de tu proyecto subvencionado por la DANA en los municipios afectados durante el periodo de ejecu
circunstancias sobrevenidas debidamente justificadas y la modificación no suponga cambios esenciales en la orden de concesión.",4,5,5,4

```

Figura 125: Muestra del fichero de evaluaciones correspondiente al modelo de lenguaje *GPT-4.1* y modelo de embedding *text-embedding-3-small*, con la estrategia de búsqueda TF-IDF

Una vez realizadas todas las pruebas que creamos oportunas, podemos generar fácilmente gráficas o tablas con los resultados de todas estas evaluaciones. La sección 4.4 tratará este tema más en profundidad.

4.3. Casos de uso de la aplicación

En esta sección veremos los ejemplos de uso de la aplicación que hemos desarrollado. Algunos ya han sido mostrados en el capítulo anterior, donde explicábamos los pasos seguidos en el diseño e implementación de la interfaz, apoyándonos con ejemplos de esta y código.

Los casos de uso que hemos elegido, representan el funcionamiento básico que obtendrá un usuario al interactuar con la interfaz web. Estos serán representados en una tabla informativa que mostrará: el nombre del caso de uso, los actores que interactúan en él, y las pre-condiciones y post-condiciones de dicho caso de uso. La plantilla usada se puede ver a continuación.

Casos de uso	« Nombre del CU »	« ID »
Actores	« Listado de actores participantes en el CU »	
Pre-condiciones	« Condiciones que se tienen que dar para la realización del CU »	
Post-condiciones	« Efectos tras la realización del CU »	

Tabla 4.1: Plantilla usada en los casos de uso

El primer caso de uso 4.2 representa el uso para el cual fue diseñada una interfaz, la interacción entre usuario y RAG mediante una consulta.

Casos de uso	Pregunta: “¿Cuál es la cantidad máxima que puedo recibir por daños en los electrodomésticos de mi vivienda actual debido a la DANA?”	CU_01
Actores	Usuario/Interfaz	
Pre-condiciones	Sistema RAG completo: sistema de recuperación y generación de información	
Post-condiciones	Respuesta informada con aportación de referencias	

Tabla 4.2: Caso de uso 01, consulta de usuario

La pre-condición de este caso de uso, implica que en la interacción entre usuario e interfaz deben estar todos los componentes del sistema RAG funcionando al 100 %, es decir: debe existir una base de conocimiento de la cual el modelo de recuperación de información extraerá la información/documentos pertinentes, y estos serán la base para generar una respuesta por el modelo de lenguaje que se use.

Un ejemplo de empleo de este caso de uso fue mostrado en las ilustraciones 104 y 105 de esta memoria.

En cuanto a los dos siguientes casos de uso (4.3 y 4.4), representan algunos elementos de personalización en la respuesta. Podremos obtener respuestas usando un modelo de lenguaje distinto, o una recuperación distinta aplicando otra técnica de búsqueda.

La interfaz muestra varias opciones como se pueden ver en las figuras 126 y 127, y como ya se ha mencionado en secciones anteriores y en las pre-condiciones de los casos de uso, los modelos deben estar presentes en la máquina local o ser accesibles mediante API.

Casos de uso	Cambio de modelo de lenguaje en la consulta	CU_02
Actores	Usuario/Interfaz	
Pre-condiciones	Descarga previa en el caso de modelos locales, o acceso vía API a los modelos externos	
Post-condiciones	Generación de respuesta usando el modelo elegido por el usuario	

Tabla 4.3: Caso de uso 02, cambio de LLM

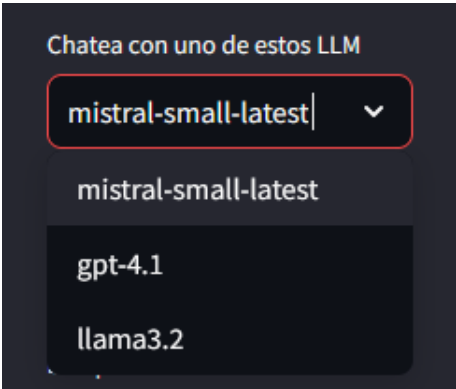


Figura 126: Modelos de lenguaje disponibles en la interfaz web

Casos de uso	Cambio en la estrategia de búsqueda del modelo de recuperación	CU_03
Actores	Usuario/Interfaz	
Pre-condiciones	Modelo de embedding disponible para realizar la recuperación	
Post-condiciones	Recuperación de información usando la técnica elegida por el usuario	

Tabla 4.4: Caso de uso 03, cambio de estrategia de búsqueda

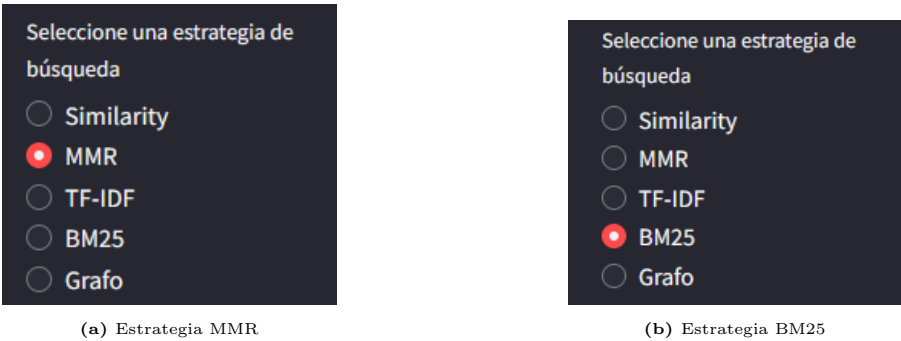


Figura 127: Ejemplos de cambio de estrategia de búsqueda desde la interfaz

Los tres siguiente casos de uso se refieren tanto a la creación de la base de conocimiento del RAG, como a la actualización de esta. Es la base de un sistema RAG y lo que lo diferencia de un modelo de lenguaje genérico, el acceso a datos externos o privados.

Nuestra interfaz cuenta con una página desde la que podemos cumplir el cometido de estos casos de uso: crear una base desde cero (4.5), añadir nuevo conocimiento desde una carpeta (4.6) o fichero (4.7).

Casos de uso	Creación de base de conocimiento para el sistema RAG	CU_04
Actores	Usuario/Interfaz	
Pre-condiciones	No existencia de base de datos vectorial, acceso a modelo de embedding local o vía API	
Post-condiciones	Creación de base de datos vectorial con los documentos aportados	

Tabla 4.5: Caso de uso 04, creación de base vectorial

Casos de uso	Actualizar de base de conocimiento desde carpeta	CU_05
Actores	Usuario/Interfaz	
Pre-condiciones	Existencia de base de datos vectorial, acceso a modelo de embedding local o vía API y carpeta con documentos PDF	
Post-condiciones	Nuevo conocimiento aportado a la base de datos vectorial desde la carpeta seleccionada	

Tabla 4.6: Caso de uso 05, actualización desde carpeta

Casos de uso	Actualizar de base de conocimiento desde fichero	CU_06
Actores	Usuario/Interfaz	
Pre-condiciones	Existencia de base de datos vectorial, acceso a modelo de embedding local o vía API y fichero PDF	
Post-condiciones	Nuevo conocimiento aportado a la base de datos vectorial desde el fichero seleccionado	

Tabla 4.7: Caso de uso 06, actualización desde fichero

Ejemplos de estos casos de uso ya fueron mostrados en el capítulo anterior, el contenido de la página visto en la figura 84, y el resultado de la ejecución de los casos de uso 4.6 y 4.7 en las figuras 86, 87, 88 y 89.

4.4. Resultados de las evaluaciones del RAG

En esta sección presentaremos los resultados de las evaluaciones realizadas (disponibles en el anexo [65] de esta memoria) sobre el sistema RAG. En dichas evaluaciones se tomarán los resultados asignados por los cuatro agentes críticos definidos en el fichero de pruebas:

- **Accuracy:** Evalúa la precisión de la respuesta generada en comparación a la respuesta esperada.
- **Faithfulness:** Evalúa si la respuesta usa la información de los documentos recuperados.
- **Groundness:** Evalúa la capacidad del modelo de apoyar sus respuestas en las evidencias de los documentos recuperados.

- **Relevance:** Evalúa la relevancia de los documentos recuperados para responder a la pregunta.

El conjunto de evaluación creado para la ocasión (ver sección 4.1) cuenta con un total de trescientas preguntas con sus respectivas respuestas y referencias. Los agentes críticos realizarán sus evaluaciones pertinentes y guardarán los resultados en formato csv (como se indica en la sección 4.2.3). Gracias al formato csv, podemos representar los resultados obtenidos por los agentes críticos en las diferentes configuraciones del sistema RAG de forma sencilla en forma de gráfica o de tabla.

Antes de comenzar la sección de resultados, se han dispuesto las tablas 4.8, 4.9 y 4.10. Estas muestran la respuesta generada por los LLMs usados a una misma pregunta, además de mostrar la respuesta original (base) y el accuracy asignado por el susodicho agente crítico.

Como se observa en todas las tablas, las respuestas generadas por los modelos de lenguaje son diferentes. El accuracy mostrado, obviamente, es afectado por la calidad de estas respuestas, diferenciando claramente al modelo Llama 3.2 (menos potente) de los otros dos. En las tablas 4.9 y 4.10 se observa claramente este factor: Llama 3.2 no puede o genera una respuesta incorrecta, lo que hace al agente crítico asignar la peor puntuación; mientras que los modelos de Mistral y OpenAI siempre generan respuestas buenas con puntuaciones altas (recordemos que las puntuaciones asignadas oscilan entre 1 para respuestas malas y 5 para respuestas buenas).

Nota: Las muestras de las tablas se han extraído de las siguientes evaluaciones: modelo de embedding *Jina* con tamaño de documento 1024 para *Llama 3.2* y *Mistral Small 3.2*, y modelo de embedding *text-embedding3-small* con tamaño de documento 1024 para *GPT-4.1*. La estrategia de búsqueda usada para todos ha sido *similarity* con *reranking*.

Pregunta	¿Cuánto dinero se ha movilizado en ayudas urgentes para hogares, empresas, autónomos e instituciones locales afectados por la DANA en la Comunitat Valenciana?	
Respuesta base	10.609 millones de euros	
Modelo	Respuesta	Accuracy
Llama 3.2	La respuesta es: Se han movilizado 10.609 millones de euros en ayudas urgentes para hogares, empresas, autónomos e instituciones locales afectados por la DANA en la Comunitat Valenciana. Esta información se encuentra en el contexto proporcionado, específicamente en el párrafo que describe las medidas adoptadas por el Gobierno para responder a los daños causados por la Depresión Aislada en Niveles Altos (DANA) en diferentes municipios de la Comunitat Valenciana.	4
Mistral Small 3.2	El contexto proporcionado indica que, a través del **Real Decreto-ley 6/2024, de 5 de noviembre** , se han movilizado **10.609 millones de euros** en ayudas urgentes para hogares, empresas, autónomos e instituciones locales afectados por la DANA en la Comunitat Valenciana. Estas ayudas están destinadas a paliar daños personales, materiales en viviendas y enseres, así como en establecimientos industriales, mercantiles y de servicios. Además, el texto menciona que estas ayudas incluyen beneficios fiscales y de Seguridad Social, así como otras medidas complementarias. La movilización de estos fondos forma parte del **Plan de Respuesta Inmediata, de Reconstrucción y de Relanzamiento de la Comunitat Valenciana** , que busca abordar los efectos devastadores de la DANA en la región. No se especifica en el contexto si estos 10.609 millones de euros son exclusivos para la Comunitat Valenciana o si se distribuyen entre otras regiones afectadas. Sin embargo, el texto sí destaca que la DANA afectó a múltiples comunidades autónomas, incluyendo a la Comunitat Valenciana, donde se registraron daños en 75 términos municipales	4
GPT-4.1	Se han movilizado 10.609 millones de euros en ayudas urgentes para hogares, empresas, autónomos e instituciones locales afectados por la DANA en la Comunitat Valenciana. Estas ayudas forman parte de un primer conjunto de medidas implementadas el 5 de noviembre de 2024, destinadas a paliar daños personales, materiales en viviendas y enseres, así como en establecimientos industriales, mercantiles y de servicios. Además, se incluyen beneficios fiscales y de Seguridad Social, junto con otras medidas de protección social para los colectivos afectados en los municipios incluidos en el anexo del Real Decreto-ley 6/2024, de 5 de noviembre. Dentro de estas ayudas, se contemplan también subvenciones específicas, como: - 5.000.000 euros a la Plataforma del Tercer Sector para actuaciones en servicios sociales en los municipios afectados. - 700.000 euros al Consejo General de Trabajo Social para el refuerzo y despliegue del Grupo Estatal de Intervención en Emergencias Sociales (GEIES). Estas ayudas son compatibles con otras subvenciones, ayudas, ingresos o recursos que puedan recibir los beneficiarios. Además, se han previsto convocatorias específicas para entidades locales especialmente afectadas, y se han detallado importes concretos para municipios como València-Pedanías Sur, Vilamarxant, Yátova y Benicull de Xúquer, entre otros. En resumen, el total movilizado en ayudas urgentes asciende a 10.609 millones de euros, incluyendo tanto ayudas directas como subvenciones y otras medidas de apoyo económico y social.	4

Tabla 4.8: Comparativa de respuestas a pregunta 1

Pregunta	¿Dónde puedo adquirir los vehículos subvencionados por el PLAN REINICIA AUTO+?	
Respuesta base	Los vehículos objeto de las subvenciones solo podrán adquirirse a través de los puntos de venta acreditados en el programa PLAN REINICIA AUTO+.	
Modelo	Respuesta	Accuracy
Llama 3.2	No puedo proporcionar una respuesta detallada a la pregunta. ¿Puedo ayudarte con algo más?	1
Mistral Small 3.2	Los vehículos subvencionados por el **PLAN REINICIA AUTO+** solo pueden adquirirse a través de los **puntos de venta acreditados** en dicho programa. Esta adquisición debe realizarse siguiendo el **procedimiento establecido en el artículo 9** de la convocatoria. Además, el ámbito geográfico de estas operaciones de compraventa abarca **todo el territorio nacional** , lo que significa que los puntos de venta acreditados pueden estar ubicados en cualquier parte de España. En resumen: - **Lugar de adquisición** : Puntos de venta acreditados en el PLAN REINICIA AUTO+. - **Procedimiento** : Según lo indicado en el artículo 9. - **Ámbito geográfico** : Todo el territorio nacional.	4
GPT-4.1	Los vehículos subvencionados por el PLAN REINICIA AUTO+ solo pueden adquirirse a través de los puntos de venta acreditados en el propio programa PLAN REINICIA AUTO+. No es posible comprarlos en cualquier concesionario, sino únicamente en aquellos que estén reconocidos y acreditados dentro del programa. El ámbito geográfico para la adquisición de estos vehículos abarca todo el territorio nacional, por lo que puedes acceder a los puntos de venta acreditados en cualquier parte de España. Para adquirir un vehículo subvencionado, debes seguir el procedimiento establecido en el artículo 9 del programa. Tanto los vehículos nuevos matriculados en España a partir del 30 de octubre de 2024, como los vehículos seminuevos que hayan sido titularidad de un concesionario y matriculados a su nombre después del 30 de octubre de 2021, pueden ser objeto de ayuda, siempre que la compra se realice en estos puntos de venta acreditados. En resumen: - Los vehículos subvencionados solo pueden adquirirse en puntos de venta acreditados en el PLAN REINICIA AUTO+. - El ámbito de adquisición es todo el territorio nacional. - Es necesario seguir el procedimiento específico del programa para la compra. No se proporciona en el contexto una lista concreta de los puntos de venta acreditados, pero estos deben estar identificados oficialmente dentro del programa PLAN REINICIA AUTO+.	5

Tabla 4.9: Comparativa de respuestas a pregunta 2

Pregunta	¿Qué entidad está a cargo del seguimiento de la calidad de las aguas continentales afectadas por la DANA en octubre de 2024 en la Confederación Hidrográfica del Júcar?	
Respuesta base	La Presidencia de la Confederación Hidrográfica del Júcar es la entidad encargada del seguimiento de la calidad de las aguas continentales afectadas por la DANA en octubre de 2024.	
Modelo	Respuesta	Accuracy
Llama 3.2	La entidad a cargo del seguimiento de la calidad de las aguas continentales afectadas por la DANA en octubre de 2024 en la Confederación Hidrográfica del Júcar es el Ministerio para la Transición Ecológica y el Reto Demográfico.	1
Mistral Small 3.2	La entidad encargada del seguimiento de la calidad de las aguas continentales afectadas por la DANA en octubre de 2024 en la Confederación Hidrográfica del Júcar es la **Presidencia de la Confederación Hidrográfica del Júcar**. Esta responsabilidad se detalla en el anuncio de formalización de contratos publicado en el Boletín Oficial del Estado (BOE) el 13 de marzo de 2025, bajo el expediente **FP.CAC.002/2025**. El objeto del contrato es específicamente el **seguimiento de la calidad de las aguas continentales** para evaluar la afección causada por las avenidas de agua extraordinarias derivadas de los fuertes temporales provocados por la DANA en octubre de 2024. Además, se proporcionan los siguientes datos de contacto de la entidad: - **Dirección**: Av. Blasco Ibañez 48, Valencia (46010). - **Teléfono**: 963938816. - **Fax**: 963938801. - **Número de identificación fiscal**: Q4617001E. Esta información está respaldada por el anuncio oficial publicado en el BOE, que especifica que la Presidencia de la Confederación Hidrográfica del Júcar es la responsable de este seguimiento.	5
GPT-4.1	La entidad a cargo del seguimiento de la calidad de las aguas continentales afectadas por la DANA en octubre de 2024 en el ámbito de la Confederación Hidrográfica del Júcar es la Presidencia de la Confederación Hidrográfica del Júcar. Esta entidad tiene como objeto la evaluación de la afección causada por las avenidas de agua extraordinarias derivadas de los fuertes temporales provocados por la DANA en dicho periodo. Los datos relevantes de la entidad responsable son: - Nombre: Presidencia de la Confederación Hidrográfica del Júcar - Dirección: Av. Blasco Ibañez 48, Valencia, 46010, España - Teléfono: 963938816 - Fax: 963938801 - Correo electrónico: Bzn_Contratacion@chj.es - Página web: http://www.chj.es/ La actividad principal ejercida por esta autoridad estatal es la protección del medio ambiente, y el seguimiento de la calidad de las aguas continentales se realiza específicamente para evaluar los efectos de la DANA de octubre de 2024 en el ámbito de la Confederación Hidrográfica del Júcar.	5

Tabla 4.10: Comparativa de respuestas a pregunta 3

4.4.1. Resultados de Mistral Small 3.2

El modelo *Mistral Small 3.2*, es sobre el cual hemos realizado el mayor numero de observaciones. El carácter externo y gratuito de este modelo nos ha permitido ejecutar al menos una prueba con cada modelo de embedding disponible localmente en sus diferentes versiones (tamaño 512 y 1024 de documentos) y con distintos *prompts*, en unos tiempos moderadamente bajos.

Cada prueba de modelo de embedding y estrategia de búsqueda tomaba alrededor de 3 horas de ejecución. Si hacemos cuentas del tiempo empleado solo en pruebas de este modelo de lenguaje, obtendríamos un total de 240 horas empleadas para generar los resultados sólo de esta sección (los resultados sin y con *reranking* se obtienen en una misma prueba).

En los siguientes apartados se muestran los resultados obtenidos en tablas y gráficas de las evaluaciones realizadas, junto con breves observaciones de ellos.

Resultados en bases de datos con tamaño de documentos 512

En las primeras pruebas hemos usado las bases de datos con menor tamaño (512). Los resultados pueden verse en la siguiente tabla, que esta ordenaba por el mejor *accuracy* de cada modelo de embedding.

Embedding	Estrategia	Accuracy	Faithfulness	Groundedness	Relevance
bge-m3	tfidf_rr	3.46	4.79	4.71	4.51
bge-m3	tfidf	3.36	4.78	4.59	4.34
bge-m3	bm25_rr	3.3	4.8	4.57	4.37
bge-m3	grafo_rr	3.19	4.75	4.62	4.12
bge-m3	similarity_rr	3.15	4.78	4.65	4.24
bge-m3	bm25	3.14	4.73	4.41	4.15
bge-m3	mmr_rr	3.13	4.79	4.64	4.24
bge-m3	similarity	3.1	4.75	4.57	4.14
bge-m3	grafo	3.03	4.76	4.5	4.1
bge-m3	mmr	2.91	4.79	4.45	4.07
jina	tfidf_rr	3.52	4.77	4.66	4.5
jina	tfidf	3.39	4.8	4.59	4.33
jina	bm25_rr	3.28	4.76	4.6	4.37
jina	mmr_rr	3.15	4.76	4.63	4.21
jina	bm25	3.14	4.67	4.39	4.15
jina	similarity_rr	3.13	4.84	4.66	4.32
jina	grafo_rr	3.03	4.79	4.58	3.97
jina	mmr	3.02	4.79	4.48	4.14
jina	similarity	2.94	4.78	4.59	4.1
jina	grafo	2.92	4.77	4.45	3.85
nomic	tfidf_rr	3.5	4.8	4.68	4.49
nomic	tfidf	3.39	4.77	4.52	4.28
nomic	bm25_rr	3.33	4.78	4.59	4.38
nomic	similarity_rr	3.28	4.81	4.59	4.33
nomic	mmr_rr	3.23	4.78	4.54	4.27
nomic	similarity	3.17	4.76	4.52	4.14
nomic	bm25	3.11	4.75	4.35	4.12
nomic	grafo_rr	3.09	4.76	4.5	4.06
nomic	grafo	3.01	4.77	4.39	3.97
nomic	mmr	2.93	4.74	4.28	3.99
snowflake	tfidf_rr	3.51	4.78	4.67	4.5
snowflake	similarity_rr	3.49	4.82	4.74	4.49
snowflake	similarity	3.44	4.8	4.66	4.38
snowflake	mmr_rr	3.43	4.82	4.69	4.5
snowflake	tfidf	3.39	4.79	4.63	4.33
snowflake	bm25_rr	3.34	4.79	4.61	4.36
snowflake	grafo_rr	3.32	4.8	4.67	4.35
snowflake	grafo	3.3	4.77	4.6	4.23
snowflake	mmr	3.29	4.84	4.62	4.32
snowflake	bm25	3.15	4.74	4.38	4.14

Tabla 4.11: Resultados de Mistral Small 3.2 en embeddings con 512 de tamaño de documentos

Los resultados obtenidos de las pruebas según la tabla anterior, muestran una clara superioridad de la estrategia de búsqueda *TF-IDF* con *reranking* sobre las demás al menos en el *accuracy*, siendo *JINA* el mejor modelo de embedding en este apartado. El modelo *JINA* esta entrenado principalmente en español, por lo que es lógico que obtenga los mejores resultados.

Los demás agentes representan lo bien que es utilizado el contexto en las respuestas o la relevancia de este a la hora de responder, y sus resultados son muy parecidos en las distintas estrategias y modelos. Salvando algunas excepciones, en general los resultados de estos últimos agentes son bastante altos, lo que indica que las respuestas están bien fundamentadas y el contexto recuperado es bueno.

En cuanto al *accuracy*, es un factor que debe mejorar. El mejor resultado, 3.52 sobre 5 no es muy alto, veremos si mejora en las siguientes pruebas.

Como aporte final para este apartado, las siguientes cuatro gráficas muestran el contenido que puede verse en la tabla 4.11 de forma individual y separando los resultado sin y con *reranking*.

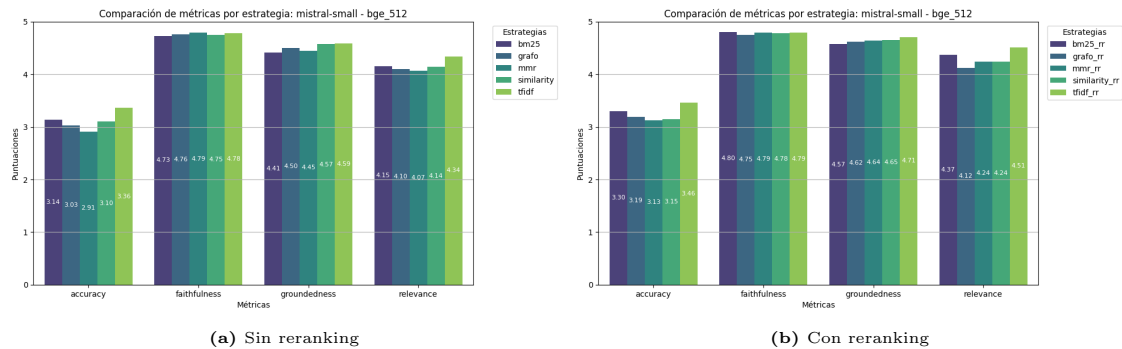


Figura 128: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding BGE-M3 y tamaños de documento 512

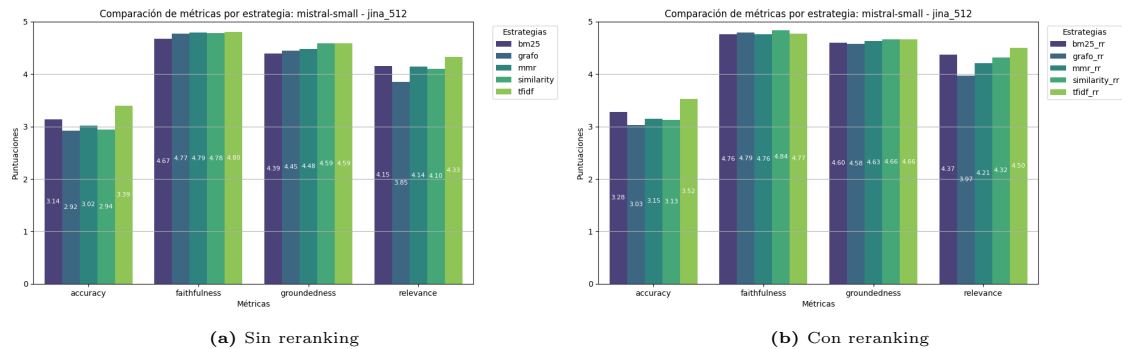


Figura 129: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding JINA y tamaños de documento 512

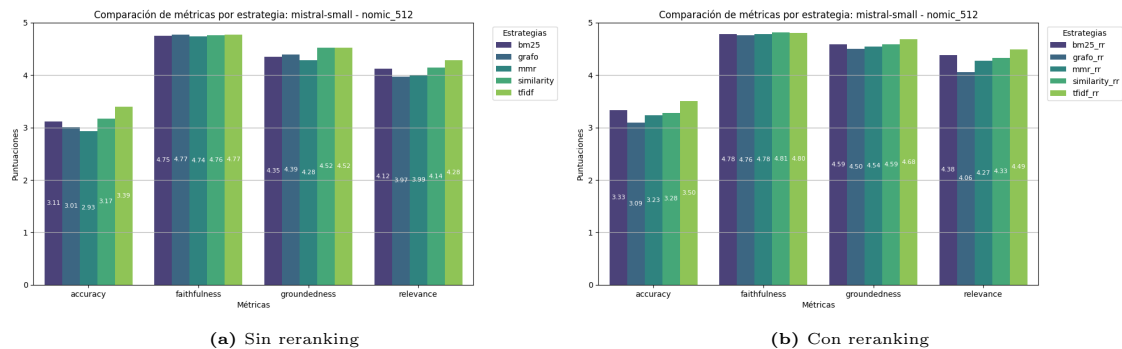


Figura 130: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Nomic-Embed-Text y tamaños de documento 512

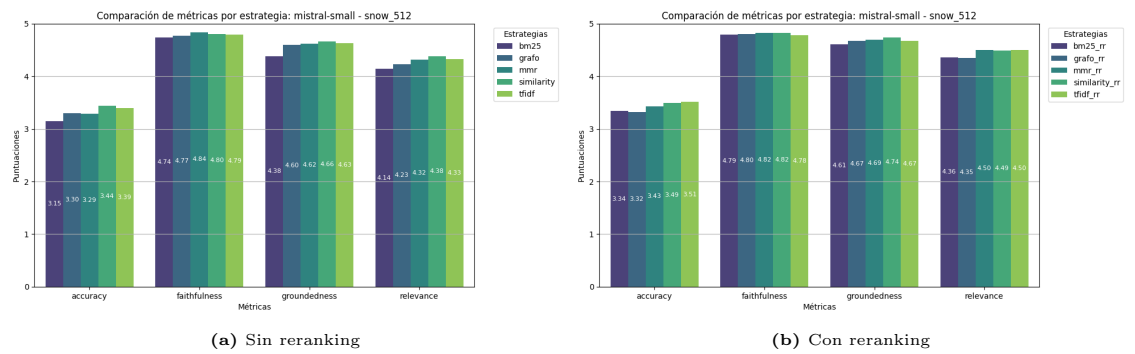


Figura 131: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Snowflakev2 y tamaños de documento 512

Resultados en bases de datos con tamaño de documentos 1024

En la segunda tanda de pruebas se usaron las bases de datos con mayor tamaño (1024). Los resultados pueden verse en la siguiente tabla, que esta ordenaba por el mejor *accuracy* de cada modelo de embedding.

Embedding	Estrategia	Accuracy	Faithfulness	Groundedness	Relevance
bge-m3	tfidf_rr	3.77	4.91	4.85	4.75
bge-m3	tfidf	3.62	4.87	4.69	4.56
bge-m3	bm25_rr	3.58	4.93	4.74	4.56
bge-m3	similarity_rr	3.54	4.88	4.77	4.53
bge-m3	bm25	3.45	4.85	4.58	4.41
bge-m3	mmr_rr	3.44	4.88	4.75	4.52
bge-m3	grafo_rr	3.38	4.89	4.64	4.31
bge-m3	similarity	3.34	4.86	4.73	4.42
bge-m3	mmr	3.29	4.87	4.62	4.39
bge-m3	grafo	3.24	4.86	4.64	4.28
jina	tfidf_rr	3.84	4.94	4.92	4.81
jina	tfidf	3.72	4.86	4.73	4.71
jina	bm25_rr	3.66	4.95	4.81	4.63
jina	bm25	3.51	4.87	4.63	4.49
jina	mmr_rr	3.3	4.88	4.65	4.42
jina	similarity_rr	3.28	4.87	4.66	4.32
jina	similarity	3.19	4.83	4.53	4.21
jina	mmr	3.14	4.81	4.6	4.28
jina	grafo_rr	3.11	4.83	4.52	4.13
jina	grafo	3.02	4.81	4.43	4.04
nomic	tfidf_rr	3.72	4.9	4.82	4.75
nomic	bm25_rr	3.61	4.88	4.73	4.56
nomic	tfidf	3.6	4.86	4.68	4.57
nomic	similarity_rr	3.52	4.88	4.7	4.46
nomic	bm25	3.45	4.87	4.6	4.41
nomic	similarity	3.26	4.84	4.52	4.2
nomic	mmr_rr	3.15	4.87	4.49	4.22
nomic	mmr	2.98	4.82	4.3	3.99
nomic	grafo_rr	0.96	4.64	2.09	1.32
nomic	grafo	0.8	4.69	1.51	0.97
snowflake	similarity_rr	3.69	4.91	4.81	4.64
snowflake	tfidf_rr	3.62	4.94	4.29	4.76
snowflake	tfidf	3.61	4.89	4.68	4.57
snowflake	similarity	3.57	4.88	4.73	4.54
snowflake	bm25_rr	3.56	4.92	4.71	4.54
snowflake	grafo_rr	3.51	4.9	4.8	4.47
snowflake	bm25	3.48	4.84	4.6	4.41
snowflake	grafo	3.43	4.86	4.73	4.44
snowflake	mmr_rr	3.41	4.93	4.15	4.6
snowflake	mmr	3.38	4.9	4.67	4.46

Tabla 4.12: Resultados de Mistral Small 3.2 en embeddings con 1024 de tamaño de documentos

En este caso, podemos observar que el aumento de tamaño de los documentos ha influido positivamente a la puntuación asignada por los agentes. Cada agente ha mejorado en torno a una o dos décimas, siendo el *accuracy* el que obtenido un incremento de más de tres décimas.

La estrategia dominante vuelve a ser *TF-IDF* con *reranking* junto con el modelo de embedding *JINA*, que obtiene los mejores resultados en todos y cada uno de los agentes críticos. Este hecho, refuerza la idea de que este es el mejor modelo a usar con texto escrito en español.

En cuanto a los resultados en sí, aunque el *accuracy* ha mejorado, sigue manteniendo un

valor un poco bajo respecto a nuestras expectativas. Una mejora en el *prompt* podría influir positivamente.

Las demás métricas muestran un funcionamiento sólido de nuestro sistema RAG a la hora de la recuperación y uso del contexto en las respuestas, con valores cercanos al 5 en muchas ocasiones.

Al igual que en el apartado anterior, se muestran las gráficas individuales cuyo contenido esta resumido en la tabla 4.12.

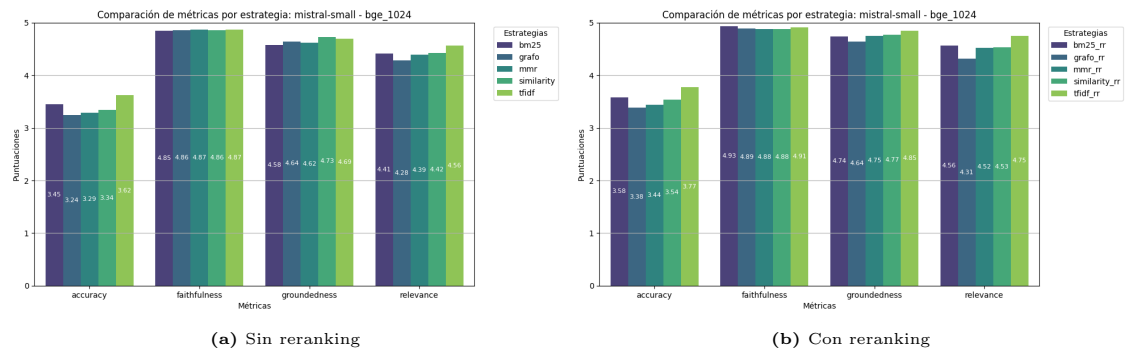


Figura 132: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding BGE-M3 y tamaños de documento 1024

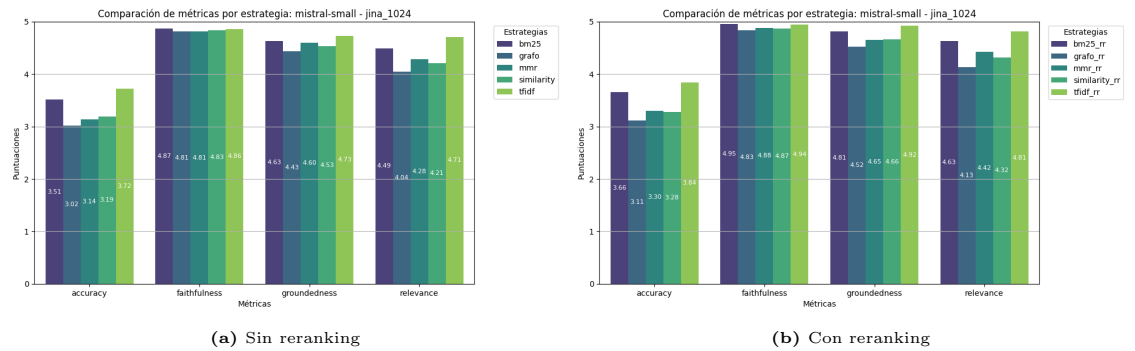


Figura 133: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding JINA y tamaños de documento 1024

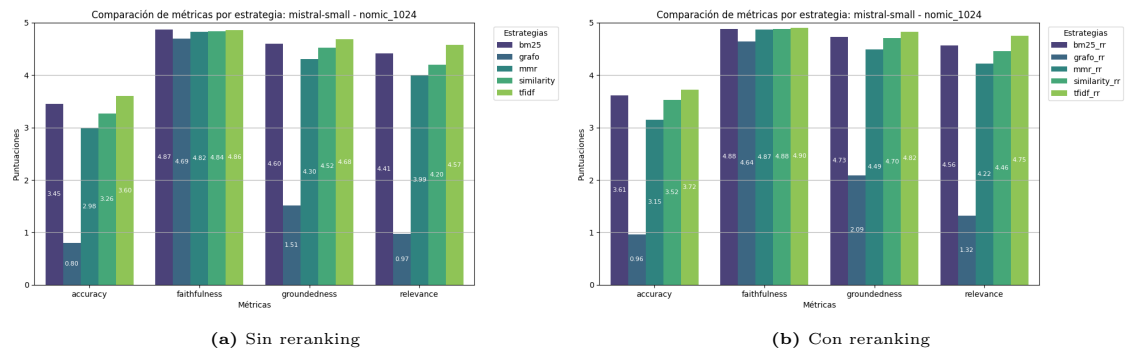


Figura 134: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Nomic-Embed-Text y tamaños de documento 1024

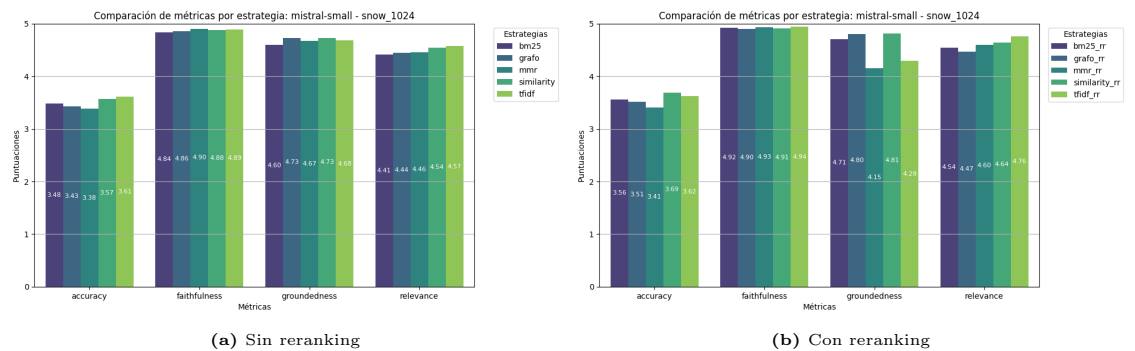


Figura 135: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Snowflakev2 y tamaños de documento 1024

Resultados usando un prompt modificado en bases de datos con tamaño de documentos 1024

En esta tanda de pruebas se hizo una modificación en el *prompt* interno del sistema RAG. En este *prompt* (figura 136), se indica que debe efectuar una respuesta de forma más simple y escueta a la consulta del usuario. El *prompt* por defecto se extiende demasiado en su respuesta, generando estructuras y listas que pueden afectar negativamente a la evaluación del *accuracy*.

```
prompt_test = """
Eres un asistente que responde preguntas frecuente (FAQ) usando SOLO y ÚNICAMENTE el contexto proporcionado.
Si la respuesta NO está en contexto, simplemente dí que no lo sabes y no respondas o te inventes la respuesta.
Tus respuestas deben ser claras, concisas y directas, sin rodeos ni explicaciones innecesarias.

Aquí tienes la pregunta y el contexto:

Pregunta: {question}\n
Contexto: {context}\n
"""
```

Figura 136: Modificación hecha en el prompt

Los resultados se muestran en la siguiente tabla:

Embedding	Estrategia	Accuracy	Faithfulness	Groundedness	Relevance
bge-m3	tfidf_rr	3.72	4.95	4.4	4.82
bge-m3	bm25_rr	3.51	4.8	4.24	4.64
bge-m3	similarity_rr	3.44	4.77	4.09	4.55
bge-m3	tfidf	3.43	4.83	4.07	4.67
bge-m3	bm25	3.38	4.82	3.98	4.52
bge-m3	mmr_rr	3.32	4.86	4.09	4.51
bge-m3	grafo_rr	3.21	4.74	3.96	4.31
bge-m3	similarity	3.12	4.74	3.84	4.38
bge-m3	mmr	3.04	4.83	3.9	4.37
bge-m3	grafo	2.95	4.76	3.76	4.27
jina	tfidf_rr	3.66	4.92	4.3	4.73
jina	bm25_rr	3.55	4.9	4.26	4.66
jina	bm25	3.46	4.79	4.01	4.56
jina	tfidf	3.37	4.81	3.97	4.57
jina	mmr_rr	3.1	4.87	3.98	4.41
jina	similarity_rr	3.09	4.77	3.91	4.37
jina	similarity	2.84	4.72	3.74	4.22
jina	mmr	2.81	4.88	3.66	4.26
jina	grafo_rr	2.77	4.53	3.52	4.12
jina	grafo	2.6	4.48	3.38	4.06
nomic	tfidf_rr	3.67	4.91	4.3	4.75
nomic	bm25_rr	3.39	4.87	4.08	4.56
nomic	tfidf	3.38	4.76	3.95	4.55
nomic	bm25	3.23	4.84	3.81	4.4
nomic	similarity_rr	3.14	4.75	3.77	4.36
nomic	mmr_rr	2.95	4.79	3.65	4.24
nomic	similarity	2.9	4.7	3.44	4.04
nomic	grafo_rr	2.77	4.67	3.44	3.97
nomic	mmr	2.62	4.85	3.15	3.99
nomic	grafo	2.56	4.53	3.22	3.8
snowflake	tfidf_rr	3.65	4.92	4.27	4.75
snowflake	similarity_rr	3.53	4.87	4.24	4.62
snowflake	mmr_rr	3.44	4.89	4.16	4.62
snowflake	tfidf	3.43	4.81	3.99	4.57
snowflake	similarity	3.4	4.86	4.08	4.54
snowflake	grafo_rr	3.36	4.77	4.09	4.47
snowflake	bm25_rr	3.35	4.91	4.09	4.57
snowflake	grafo	3.22	4.76	3.9	4.4
snowflake	bm25	3.21	4.79	3.79	4.4
snowflake	mmr	3.17	4.95	3.97	4.51

Tabla 4.13: Resultados de Mistral Small 3.2 en embeddings 1024 de tamaño de documentos y el prompt modificado

A la vista de los resultados obtenidos de estas pruebas, podemos asegurar que la modificación del *prompt* introducida ha tenido influencia en ellos. La mejora esperada no se ha producido, y hemos tenido resultados más bajos que en el apartado anterior en todas las métricas salvo en *faithfulness*.

La simplificación del *prompt* de la cual esperábamos mejores resultados, ha afectado negativamente a estos. El dato positivo que podemos extraer de esta batería de pruebas es que al modificar el *prompt*, obtenemos una respuesta diferente del modelo de lenguaje, y por consiguiente, unos resultados distintos para nuestros test.

Resultados que se pueden observar individualmente (como en los demás apartados), en las

siguientes gráficas:

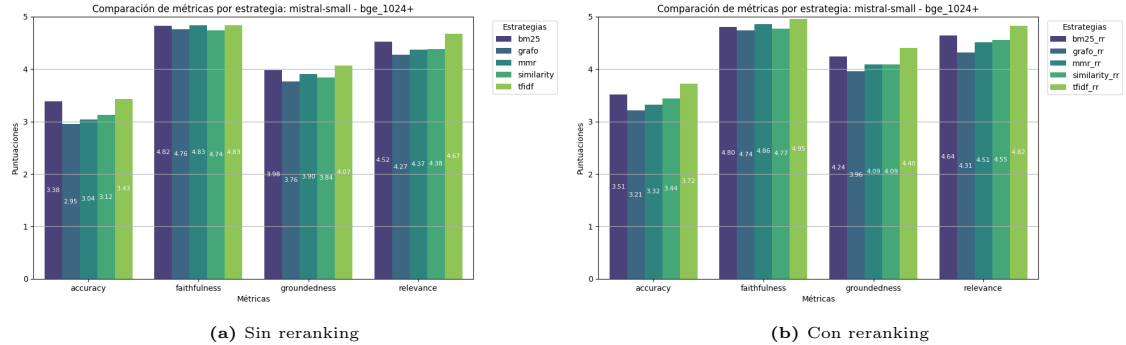


Figura 137: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding BGE-M3, tamaños de documento 1024 y el prompt modificado

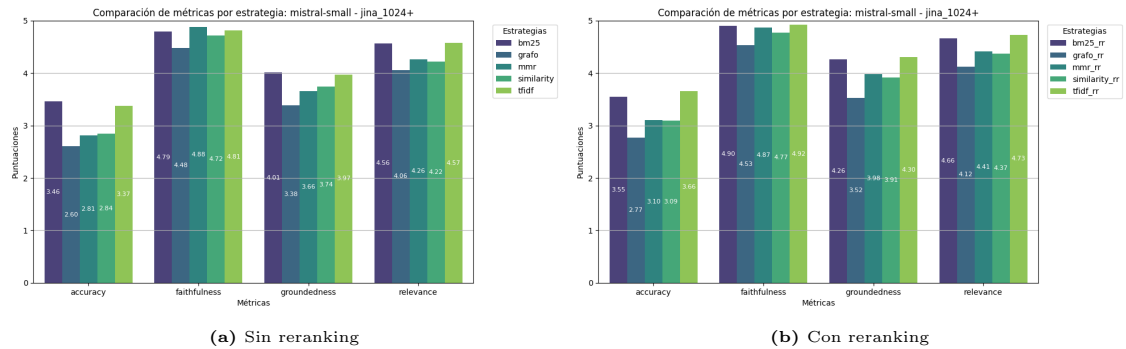


Figura 138: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding JINA, tamaños de documento 1024 y el prompt modificado

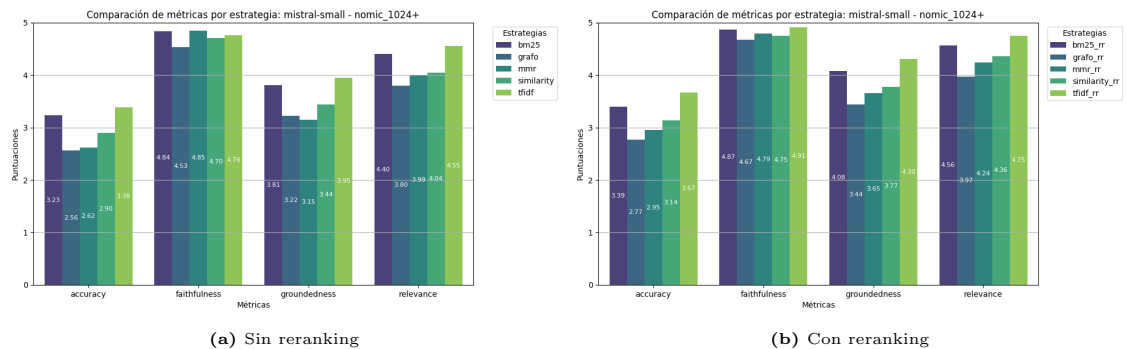


Figura 139: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Nomic-Embed-Text, tamaños de documento 1024 y el prompt modificado

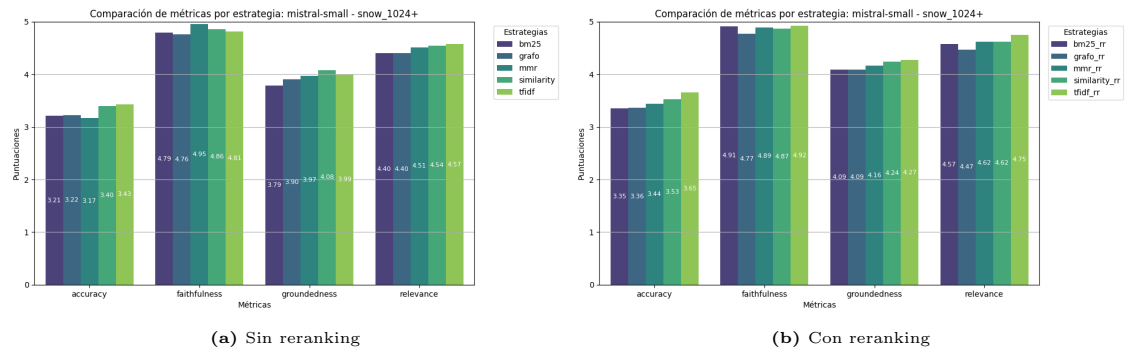


Figura 140: Gráfica de rendimiento del modelo Mistral Small 3.2 usando el modelo de embedding Snowflakev2, tamaños de documento 1024 y el prompt modificado

4.4.2. Resultados de Llama 3.2

Llama 3.2 es el modelo usado para realizar estas pruebas de forma local. Este modelo es el más modesto de los utilizados en el proyecto, contando solo con 3 billones (ingleses) de parámetros. Debido a esto, se espera recibir unos resultados peores a los anteriores mostrados.

Otro factor importante derivado del método de ejecución de este modelo es que, al ser ejecutado localmente, consume bastante tiempo en generar una respuesta. Al depender de la configuración de la máquina donde se use, si esta no cumple con los requisitos necesarios los tiempos de respuesta se alargan considerablemente. En nuestro caso, al no disponer de una tarjeta gráfica *nvidia*, sufrimos esta consecuencia.

Para mitigar la tardanza en el tiempo de respuesta, se planteó usar un 10 % del conjunto de preguntas (unas 30) y extrapolar los resultados. Dichos resultados se obtuvieron en una media de 3 horas y 20 minutos para cada prueba, empleándose alrededor de unas 16 horas y 40 minutos para realizar las pruebas en un solo embedding con la suma de todas las estrategias de búsqueda.

Los resultados obtenidos para un solo embedding se muestran en la siguiente tabla:

Embedding	Estrategia	Accuracy	Faithfulness	Groundedness	Relevance
jina	similarity_rr	3.08	4.17	3.75	4.25
jina	grafo	2.94	4.29	3.18	4.18
jina	similarity	2.76	3.94	3.41	4.24
jina	tfidf_rr	2.5	4.5	3.33	4.67
jina	tfidf	2.4	4.0	3.47	4.33
jina	bm25_rr	2.36	4.18	3.0	4.36
jina	grafo_rr	2.3	4.4	3.2	4.6
jina	mmr	2.29	4.29	3.43	3.86
jina	mmr_rr	1.8	4.1	3.4	3.9
jina	bm25	1.76	3.94	3.0	4.53

Tabla 4.14: Resultados de Llama 3.2 en embeddings 1024 de tamaño de documentos

Y como se esperaba, se obtienen los peores resultados en todas las métricas. Destaca la pobre puntuación obtenida en el *accuracy* en la mayoría de los agentes, algunos no llegan a superar el 50%. Siendo la mejor puntuación de esta, casi 8 décimas peor que el mejor

modelo, esta claro que no es un modelo aconsejable para el uso.

Aunque las demás métricas consiguen resultados más o menos decentes, siguen estando muy por debajo de las mejores puntuaciones obtenidas.

No se ha decidido usar otro modelo de embedding por los malos resultados obtenidos usando *JINA*, el mejor modelo en comparación vistas las otras tablas de pruebas.

En las siguientes figuras, se muestra de forma gráfica los resultados presentes en la tabla 4.14.

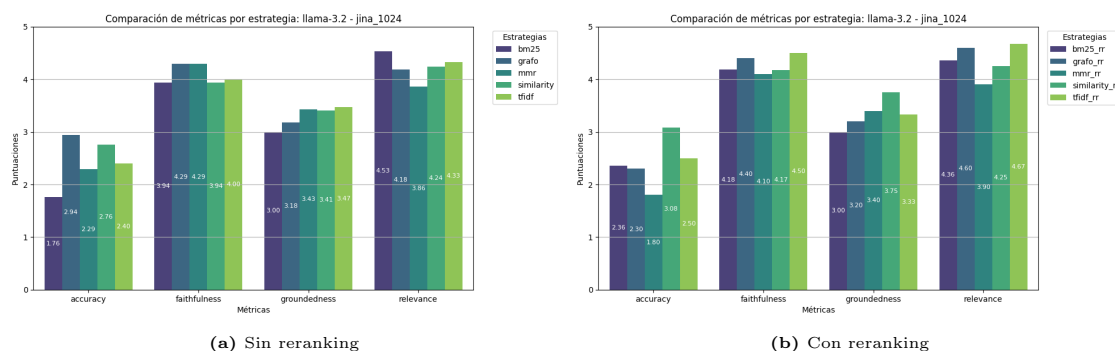


Figura 141: Gráfica de rendimiento del modelo Llama 3.2 usando el modelo de embedding Jina y tamaños de documento 1024

4.4.3. Resultados de GPT-4.1

El modelo *GPT-4.1* es el último usado en las pruebas definidas en nuestro proyecto. Este modelo perteneciente a *OpenAI* es de pago, por lo que hay que controlar el uso que le pedimos a esta plataforma si no queremos gastar mucho dinero. Por este motivo se redujo el conjunto de pruebas a un 25 % del total (75 preguntas) para después extrapolar los resultados conseguidos.

A su vez, el modelo de embedding utilizado para las pruebas también comparte plataforma con este modelo de lenguaje. Se optó por probar ambos, para medir el rendimiento de los modelos de pago en comparación al resto. El modelo de embedding (*text-embedding-3-small*) fue configurado para usar documentos de tamaño 1024.

El tiempo para obtener los resultados fue de 40 minutos por modelo de embedding. Teniendo en cuenta que obtenemos los resultados sin y con *reranking* en la misma prueba, se produjeron cinco ejecuciones en un total de dos horas y media, con un costo aproximado de 15 dólares.

Nota: Cada consulta hecha generaba alrededor de 3000 tokens, y su coste era de 0,01\$.

Los resultados de las pruebas son los mostrados en la siguiente tabla:

Embedding	Estrategia	Accuracy	Faithfulness	Groundedness	Relevance
text-embedding-3-small	tfidf	4.32	4.95	4.91	4.68
text-embedding-3-small	tfidf_rr	4.27	4.92	4.96	4.75
text-embedding-3-small	similarity_rr	3.96	4.89	4.88	4.41
text-embedding-3-small	similarity	3.95	4.88	4.89	4.49
text-embedding-3-small	mmr_rr	3.92	4.89	4.88	4.59
text-embedding-3-small	bm25	3.83	4.91	4.88	4.45
text-embedding-3-small	bm25_rr	3.83	4.92	4.95	4.41
text-embedding-3-small	grafo_rr	3.68	4.91	4.85	4.19
text-embedding-3-small	grafo	3.67	4.87	4.81	4.17
text-embedding-3-small	mmr	3.6	4.87	4.81	4.28

Tabla 4.15: Resultados de GPT-4.1 en embeddings 1024 de tamaño de documentos

Como se puede apreciar en la tabla anterior, los resultados obtenidos son los mejores obtenidos hasta ahora, con la única excepción de la métrica de relevancia.

En el caso del *accuracy*, las estrategias *TF-IDF* sin y con *reranking* superan los 4.25 de puntuación, lo cual indica una respuesta muy parecida a la esperada. Para las demás estrategias, el *accuracy* siempre supera a la mejor obtenida anteriormente.

Las métricas *faithfulness* y *groundedness* se mantienen muy altas, rozando el 5 en las mejores y siempre superando la puntuación mínima de 4.8. Es una buena manera de saber que, con estos modelos se utiliza muy bien el contexto recuperado para generar las respuestas.

En cuanto a la relevancia, aún no siendo el valor más alto obtenido en estas pruebas, el mejor conseguido sigue manteniendo una puntuación muy alta (4.75). A excepción de algunas estrategias, las puntuaciones obtenidas en esta métrica son generalmente muy altas.

En las siguientes figuras, podemos ver el contenido de la tabla 4.15 de forma gráfica.

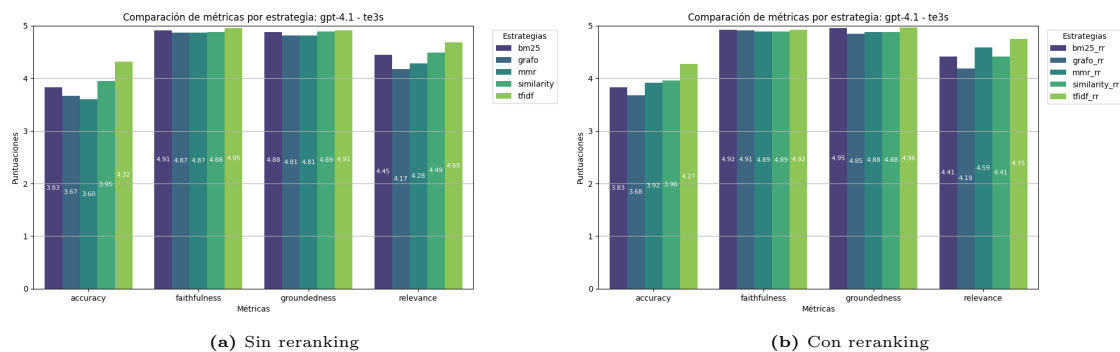


Figura 142: Gráfica de rendimiento del modelo GPT-4.1 usando el modelo de embedding text-embedding-3-small y tamaños de documento 1024

4.5. Conclusión sobre los resultados

Tras realizar las pruebas y, habiendo consultado los resultados; podemos concluir aportando el conocimiento adquirido de estos. Para ello, vamos a responder una lista de preguntas relacionadas con el rendimiento del RAG ante cambios o modificaciones en su

configuración. Y como colofón, explicaremos las deducciones finales obtenidas sobre los resultados.

¿Cómo afecta el tamaño del LLM a la calidad de la respuesta en un sistema RAG? Los modelos de lenguaje más grandes (con mayor número de parámetros) obtienen mejores resultados. Los resultados obtenidos por *Llama 3.2* (tabla 4.14) son inferiores a los otros dos modelos de lenguaje (tablas 4.12 y 4.15). Esto se debe al tamaño de los modelos, *Llama 3.2* (3B) es mucho más pequeño que *Mistral Small 3.2* (24B) y *GPT-4.1* (desconocido, aunque sabemos que la versión GPT3 cuenta con 175B de parámetros). Por lo tanto, podemos asegurar que conseguiremos mejores respuestas con modelos más grandes.

¿Pueden diferencias sutiles en el prompt afectar significativamente la alineación de la recuperación y la generación? En nuestro caso, solo afecta a la generación de respuesta, ya que la recuperación no se utiliza un *prompt* (recupera documentos relevantes a la consulta). En la prueba realizada modificando el *prompt* (tabla 4.13), pedimos una respuesta más simple y escueta, y los resultados consecuente fueron peores a los conseguidos anteriormente. Debido a esto, concluimos que el *prompt* usado por defecto es mejor generando una respuesta más parecida a la esperada y, que la modificación de este influye en los resultados.

¿Cómo impacta el tamaño del fragmento de documento recuperado en la calidad de la respuesta? Hemos comprobado este factor en las dos primeras tandas de pruebas (tablas 4.11 y 4.12). Los resultados obtenidos muestran que un tamaño mayor es mejor para generar una respuesta. Así que, podemos decir que la respuesta generada se ve favorecida si dispone de documentos recuperados con un mayor tamaño de contexto.

¿Cómo impacta el tamaño de la base de conocimiento en el rendimiento general? Hemos trabajado con bases de conocimiento que oscilaban entre 45MB y 65MB, por lo que no hemos apreciado diferencias considerables en el tiempo de recuperación de documentos. Este tiempo era casi inmediato para todas las bases de conocimiento, al contrario que el tiempo de creación de la bases de conocimiento, variando entre los 12 minutos (figura 87) y 35 minutos (figura 88) para las bases de datos locales, y los 40 segundos para el embedding *text-embedding-3-small* (figura 89), que opera vía API desde *OpenAI*. Así pues, tenemos dos opciones: si valoramos más el tiempo de recuperación, la elección es una base local gratuita dado a su nula diferencia; en cambio, si valoramos más el tiempo de creación, los métodos de pago son muy superiores.

¿Afecta la incorporación de documentos multilingües a las respuestas del sistema RAG? En nuestro caso, al manejar siempre documentos escritos en español (BOE), no hemos podido comprobar este factor. Podemos responder a esta pregunta diciendo que los modelos de embedding que mejor se han comportado con nuestra base de conocimiento son los multilingües, especialmente *JINA* (entrenado en español e inglés) y *text-embedding-3-small* (de pago).

¿El centrarse en unas pocas oraciones recuperadas agudiza las respuestas de RAG? Negativo, hemos demostrado en las pruebas que se obtienen mejores resultados al aumentar el tamaño de contexto de los documentos. En cambio, si la pregunta se refiere a reducir los documentos recuperados para mejorar la respuesta, podemos decir que el filtrado de ellos realizado por el *reranker* mejora siempre las puntuaciones asignadas por los agentes.

En resumen, los resultados muestran dos estrategias claras que podemos usar para recomendar un sistema u otro, basándonos sólo en el presupuesto que estemos dispuestos a emplear como factor determinante.

La primera estrategia es la ofrecida por la combinación de los modelos de pago (embedding y generación), ya que siempre obtienen los mejores resultados. Recordemos que el coste de la consulta era de alrededor de 0,01\$, un precio asequible pero que se puede disparar ante una gran demanda.

En cambio la segunda estrategia es, usar el modelo de lenguaje *Mistral Small 3.2* y embeddings locales. A la vista de los resultados, el uso de este modelo junto a una base vectorial *JINA* puede ser bastante competente. Como se ha comentado, el embedding *JINA* es un modelo bilingüe español-inglés, por lo que es ideal para nuestra base de conocimiento.

En cuanto a la estrategia de búsqueda, los mejores resultados los obtienen los métodos tradicionales: *TF-IDF*, *BM25* y similaridad coseno. Destacando la mayoría de veces *TF-IDF* sobre el resto.

Capítulo 5 : Conclusiones y Líneas Futuras

En este capítulo presentamos las conclusiones alcanzadas tras haber realizado este proyecto. Asimismo, finalizaremos explicando posibles mejoras y vías de desarrollo que podría seguir el proyecto si decidimos realizar futuras expansiones.

5.1. Conclusiones

Inicialmente este proyecto fue planeado como una continuación de mi Trabajo de Fin de Grado y de mi experiencia como investigador de la UGR especializado en el desarrollo de sistemas conversacionales. En él, aplicaría mis conocimientos adquiridos sobre la plataforma RASA.

Sin embargo, este es el ejemplo más claro de lo rápido que cambian los tiempos, sobre todo en el ámbito de la informática. En apenas unos años, hemos visto el resurgimiento de los modelos de lenguaje, que llevados de la mano de nuevos paradigmas de diseño, han dejado obsoletos a los enfoques más tradicionales. Ya nadie va a perder el tiempo entrenando y desarrollando un modelo específico cuando de una manera inmediata, podemos recibir una respuesta a través de las plataformas donde están alojados estos nuevos modelos.

Así pues, durante la realización de este proyecto tuvimos que dejar todo lo aprendido y apostar por este *nuevo* enfoque. Lo cual nos llevó a aprender nuevos frameworks de diseño y a usar diferentes modelos tanto de lenguaje como de vectorización (embedding), dado el carácter del proyecto. Además de utilizar plataformas modernas de pago por uso (servicios cloud), utilizadas hoy en día por la mayoría de las compañías del sector TIC.

En este sentido, el desarrollo de la aplicación fue prácticamente guiado a través de la gran número de tutoriales o de la magnífica documentación oficial que poseen estas nuevas tecnologías. *LangChain* -una de las librerías más usadas-, permite integraciones con multitud de aplicaciones, lo que ocasionó una rápida implementación y adaptación del código, y conexión con las principales plataformas que sirven los distintos modelos de lenguaje.

En cuanto a la interfaz, más de lo mismo. *Streamlit* utiliza un enfoque más moderno que inyecta código *Python* para crear una interfaz web. Como ya hemos comentado, con *Streamlit* podemos acceder a bastantes elementos de personalización (inputs, selectbox, radio buttons) propios de HTML desde el *backend*, lo que personalmente me ayudo al estar más familiarizado con este tipo de desarrollos.

Respecto a los resultados, tenemos varias conclusiones que podemos ver en la siguiente lista:

1. Hemos demostrado la superioridad del enfoque *Advanced RAG* en todos los casos de comparación, exceptuando la prueba hecha con *OpenAI* (tabla 4.15). El filtrado de documentos realizado tras la primera recuperación ofrecido por el modelo encargado de hacer el *reranking* [52], mejora la calidad de las respuestas de manera ostensible.
2. El modelo de embedding también es importante. Aunque los modelos usados en el proyecto son todos multilingües, *JINA* [24] está especializado en nuestro idioma (bilingüe español-inglés). A la prueba de los resultados, es el más recomendable para su uso en

este contexto.

3. *TF-IDF* se muestra como la estrategia de búsqueda más sólida en las pruebas realizadas, seguida de *BM25* y la búsqueda por similitud coseno en la mayoría de los casos.

Para finalizar con los resultados, hay un motivo por el cual hemos realizado la mayoría de las pruebas con el modelo de lenguaje *Mistral Small 3.2* [16]. Este modelo es gratuito y de libre acceso a través de la API provista por *Mistral* en su página web. La calidad y velocidad de respuesta que obtuvimos, hizo posible la realización de todas las pruebas que necesitábamos. Por otro lado, el modelo local usado *Llama3.2* [15], era mucho más lento en la obtención de respuestas (aún usando un conjunto muy reducido) y la calidad de ellas no era muy buena (tabla 4.14), por lo que descartamos hacer más. Y si miramos al modelo de *OpenAI*, los motivos fueron dos: primero económico y segundo porque en la primera tanda de pruebas consiguió mejores resultados que todas las anteriores. Así pues, cumplimos el mini-objetivo de demostrar que un modelo más grande mejora los resultados y finalizamos las pruebas.

En resumen, pienso que este Trabajo de Fin de Máster ha cambiado mi percepción como desarrollador de sistemas conversacionales. Desarrollar con *RASA* era más complicado en codificación y lento en entreno, y aunque los resultados eran buenos, siempre estaba el riesgo de que no tuviese consistencia en la respuesta, ya sea al usar palabras distintas a las usadas en el entreno o porque el propio modelo se salte pasos. Con el nuevo enfoque empleado en el proyecto, hemos tenido un desarrollo rápido y versátil, y me ha aportado algo de la experiencia en estos nuevos paradigmas que tanto se busca en entornos profesionales.

Y sobre el tema del proyecto, este trabajo se inserta en un ámbito en el que ya han surgido aplicaciones orientadas a dar respuesta a la DANA, como *Ajuda Dana* y *TuCocheDana*. Asimismo, proyectos como *Justicio* han mostrado cómo el enfoque RAG puede transformar la manera en que accedemos e interpretamos el BOE. En este contexto, nuestro TFM presenta una combinación ambos enfoques, con el propósito de ayudar a la ciudadanía a encontrar información oficial, veraz y comprensible en un momento de necesidad.

En definitiva, este trabajo refleja cómo el uso responsable de la inteligencia artificial puede convertirse en una herramienta clave para acercar la documentación oficial a la ciudadanía. Si se orienta hacia la transparencia, la accesibilidad y la veracidad, la IA no solo facilita la gestión de la información en situaciones críticas como la DANA, sino que también contribuye a construir un futuro mejor informado y más participativo.

5.2. Líneas futuras

En este último apartado, comentaremos posibles mejoras que pueden añadirse al proyecto que, ya sea por falta de tiempo o recursos, no se han efectuado. Estas ideas han ido apareciendo durante la realización del proyecto.

En la interfaz han surgido varias ideas de mejora en la personalización tanto de la consulta como de la creación o actualización de la fuente de conocimiento. Aspectos como poder seleccionar la base de datos a la que queremos hacer la consulta, o poder modificar la configuración de los documentos o su embedding, ayudan a que la aplicación sea más manejable para un usuario no experto y no obliga a cambiar estos datos en el código como se hace ahora.

Igualmente, se planteó añadir una página para realizar los test del sistema RAG en la interfaz. Esta parte se añadiría a una página nueva, en la que se podrían editar los siguientes parámetros:

- **Dataset de preguntas:** Manteniendo el formato visto en el apartado 4.1, se puede usar el conjunto de preguntas que deseemos. También se puede añadir un parámetro numérico para controlar el porcentaje o fragmento de preguntas a usar en el test.
- **Botones de selección de:** base de datos, estrategia de búsqueda, reranking, embedding y LLM.

Los resultados serían mostrados dinámicamente en la interfaz, ya que *Streamlit* proporciona elementos como una barra de progreso que podemos actualizar fácilmente usando el número de preguntas. Pienso que sería una actualización muy interesante, la cual no se ha realizado por falta de tiempo.

Con relación a los test de nuestro RAG, aunque hemos cubierto bastantes posibilidades, no hemos comprobado los resultados que pudiésemos haber obtenido al probar el aprendizaje en contexto (*In-Context-Learning*) que mencionamos en el Estado del Arte. Añadiendo al *prompt* un mínimo porcentaje del conjunto de preguntas, haría que el LLM reconociese el formato y, supuestamente, respondiese mejor a la consulta del usuario.

Finalmente, aunque el proyecto es totalmente funcional, necesita los requerimientos ubicados en el repositorio [58]. Este factor no sería necesario se estuviese contenerizado ya sea en *docker* u otra plataforma parecida. De esta manera funcionaría en cualquier sistema bajo cualquier plataforma gestora de contenedores. Queda como futura actualización.

: Bibliografía

- [1] RTVE. *Un mes de la DANA, en mapas e imágenes: de las inundaciones al lento reinicio del día a día*. 2024. URL: <https://www.rtve.es/noticias/20241129/dana-valencia-graficos-mapas-imagenes/16345347.shtml>.
- [2] Joan Lluís Ferrer. *Así llenaron de bulos las redes sociales la tragedia de la DANA de Valencia*. 2024. URL: <https://www.elperiodico.com/es/medio-ambiente/20250901/llenaron-bulos-redes-sociales-tragedia-121120294>.
- [3] Natassha Selvaraj. *What is Retrieval Augmented Generation (RAG)?* 2025. URL: <https://www.datacamp.com/blog/what-is-retrieval-augmented-generation-rag>.
- [4] elastic.co. *What is RAG (retrieval augmented generation)?* 2024. URL: <https://www.elastic.co/what-is/retrieval-augmented-generation>.
- [5] BOE. *BOE*. 2025. URL: <https://www.boe.es/>.
- [6] Vaibhav Khobragade. *The Journey of Large Language Models: Evolution, Application, and Limitations*. 2024. URL: <https://medium.com/@researchgraph/the-journey-of-large-language-models-evolution-application-and-limitations-c72461bf3a6f>.
- [7] Sidwendluian Nana et al. *Deep Learning and Web Applications Vulnerabilities Detection: An Approach Based on Large Language Models*. 2024. URL: https://www.researchgate.net/publication/382802591_Deep_Learning_and_Web_Applications_Vulnerabilities_Detection_An_Approach_Based_on_Large_Language_Models.
- [8] Yunfan Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2024. URL: <https://arxiv.org/abs/2312.10997>.
- [9] Conor Bronsdon. *Comparing RAG and Traditional LLMs: Which Suits Your Project?* 2024. URL: <https://www.galileo.ai/blog/comparing-rag-and-traditional-llms-which-suits-your-project>.
- [10] Pratik Bhavsar. *Understanding LLM Hallucinations Across Generative Tasks*. 2023. URL: <https://www.galileo.ai/blog/deep-dive-into-llm-hallucinations-across-generative-tasks>.
- [11] Mehmet Ozkaya. *The RAG Architecture: Ingestion and Retrieval with Embeddings and Vector Search*. 2024. URL: <https://mehmetozkaya.medium.com/the-rag-architecture-ingestion-and-retrieval-with-embeddings-and-vector-search-9bb86adb0ff5>.
- [12] Wayne Xin Zhao et al. *A Survey of Large Language Models*. 2025. URL: <https://arxiv.org/abs/2303.18223>.
- [13] Zena Assaad. *ChatGPT just passed the Turing test. But that doesn't mean AI is now as smart as humans*. 2025. URL: <https://theconversation.com/chatgpt-just-passed-the-turing-test-but-that-doesnt-mean-ai-is-now-as-smart-as-humans-253946>.

- [14] Shijie Wu et al. *BloombergGPT: A Large Language Model for Finance*. 2023. URL: <https://arxiv.org/abs/2303.17564>.
- [15] Meta. *llama3.2*. 2024. URL: <https://ollama.com/library/llama3.2>.
- [16] Mistral AI. *mistralai/Mistral-Small-3.2-24B-Instruct-2506*. 2025. URL: <https://huggingface.co/mistralai/Mistral-Small-3.2-24B-Instruct-2506>.
- [17] Mistral AI. *Mistral Small 3.1*. 2025. URL: <https://mistral.ai/news/mistral-small-3-1>.
- [18] Alex Olteanu. *GPT 4.1: Características, acceso, comparación GPT-4o y más*. 2025. URL: <https://www.datacamp.com/es/blog/gpt-4-1>.
- [19] OpenAI. *Introducing GPT-4.1 in the API*. 2025. URL: <https://openai.com/index/gpt-4-1/>.
- [20] Nayab Hassan. *Embedding Models Explained: A Guide to NLP's Core Technology*. 2024. URL: <https://medium.com/@nay1228/embedding-models-a-comprehensive-guide-for-beginners-to-experts-0cfc11d449f1>.
- [21] Nomic Team. *Introducing Nomic Embed: A Truly Open Embedding Model*. 2024. URL: <https://www.nomic.ai/blog/posts/nomic-embed-text-v1>.
- [22] Jianlv Chen et al. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. 2024. URL: <https://ollama.com/library/bge-m3>.
- [23] Snowflake. *Snowflake Arctic Embed 2.0: Rendimiento Multilingüe y Escalabilidad Empresarial*. 2024. URL: <https://ollama.com/library/snowflake-arctic-embed2>.
- [24] Jina AI. *Jina Embeddings v2 Base ES: Representaciones Bilingües Español-Inglés de Alto Rendimiento*. 2024. URL: <https://ollama.com/jina/jina-embeddings-v2-base-es>.
- [25] OpenAI. *Nuevos modelos de integración y actualizaciones de API*. 2024. URL: <https://openai.com/es-ES/index/new-embedding-models-and-api-updates/>.
- [26] Rasa. *Conversational AI with Language Models*. 2024. URL: <https://rasa.com/docs/learn/concepts/calm/>.
- [27] Rasa. *codellama 13b calm demo*. 2024. URL: https://huggingface.co/rasa/cmd_gen_codellama_13b_calm_demo.
- [28] Geshan Manandhar. *What is Ollama and how to use it: a quick guide [part 1]*. 2025. URL: <https://geshan.com.np/blog/2025/02/what-is-ollama/>.
- [29] GeeksForGeeks. *Ollama Explained: Transforming AI Accessibility and Language Processing*. 2024. URL: <https://www.geeksforgeeks.org/ollama-explained-transforming-ai-accessibility-and-language-processing/>.
- [30] LM Studio. *LM Studio Docs*. 2025. URL: <https://lmstudio.ai/docs/app>.
- [31] Chroma. *ChromaDB Docs*. 2025. URL: <https://docs.trychroma.com/docs/overview/introduction>.
- [32] Abid Ali Awan. *Chroma DB Tutorial: A Step-By-Step Guide*. 2024. URL: <https://www.datacamp.com/tutorial/chromadb-tutorial-step-by-step-guide>.

- [33] chromaDB. *Getting Started*. 2025. URL: <https://docs.trychroma.com/docs/overview/getting-started>.
- [34] Pinecone. *Pinecone Docs*. 2025. URL: <https://docs.pinecone.io/guides/get-started/overview>.
- [35] Woyera. *Pinecone vs. Chroma: The Pros and Cons*. 2023. URL: <https://medium.com/@woyera/pinecone-vs-chroma-the-pros-and-cons-2b0b7628f48f>.
- [36] LangChain. *LangChain Docs*. 2025. URL: <https://python.langchain.com/docs/introduction/>.
- [37] Logan Kilpatrick. *What is Langchain and why should I care as a developer?* 2023. URL: <https://medium.com/around-the-prompt/what-is-langchain-and-why-should-i-care-as-a-developer-b2d952c42b28>.
- [38] Kusalani Tharunya. *What is LangChain and How to Use It*. 2024. URL: <https://blog.stackademic.com/what-is-langchain-and-how-to-use-it-c9a656b80cea>.
- [39] Llamaindex. *Llamaindex Docs*. 2025. URL: <https://docs.llamaindex.ai/en/stable/>.
- [40] Bhavik Jikadara. *Understanding LlamaIndex: Features and Use Cases*. 2024. URL: <https://bhavikjikadara.medium.com/understanding-llamaindex-features-and-use-cases-c433c3246e86>.
- [41] GeeksForGeeks. *What is LlamaIndex?* 2024. URL: <https://www.geeksforgeeks.org/what-is-llamaindex/>.
- [42] Tamanna. *LangChain vs. LlamaIndex, A Comprehensive Comparison for Retrieval-Augmented Generation (RAG)*. 2024. URL: <https://medium.com/@tam.tamanna18/langchain-vs-llamaindex-a-comprehensive-comparison-for-retrieval-augmented-generation-rag-0adc119363fe>.
- [43] Streamlit. *Streamlit Docs*. 2025. URL: <https://docs.streamlit.io/>.
- [44] Dayanithi. *Streamlit in 3 Minutes*. 2023. URL: <https://medium.com/data-and-beyond/streamlit-d357935b9c>.
- [45] Dora Gurova. *What is Streamlit: All Why's and How's Answered*. 2025. URL: <https://uibakery.io/blog/what-is-streamlit>.
- [46] Ajuda Dana. 2024. URL: <https://ajudadana.es/>.
- [47] Santi Riesco. *'Ajuda Dana', una plataforma digital unificada para coordinar la ayuda y la búsqueda de personas tras la DANA*. 2024. URL: <https://www.rtve.es/noticias/20241105/ajuda-dana-plataforma-unificada-digital-para-coordinar-ayuda-busqueda-personas-tras-dana/16316884.shtml>.
- [48] Marta Sangrà. *Tu Coche DANA: una plataforma para poder encontrar los coches perdidos*. 2024. URL: <https://valenciasecreta.com/tu-coche-dana/>.
- [49] justicio.es. *Pregunta a Justicio*. 2025. URL: <https://justicio.es/boe/20250903>.
- [50] Juan Diego Polo. *Justicio, un chatbot entrenado con la legislación estatal, autonómica y europea*. 2024. URL: <https://www.whatsnew.com/2024/03/15/justicio-un-chatbot-entrenado-con-la-legislacion-estatal-autonomica-y-europea/>.

- [51] confilegal. *El BOE, ahora sí, al alcance de los juristas: Justicio lanza una nueva forma de consultarlo fácil, ágil y gratuita*. 2025. URL: <https://confilegal.com/20250625-el-boe-ahora-si-al-alcance-de-los-juristas-justicio-lanza-una-nueva-forma-de-consultarlo-facil-agil-y-gratuita/>.
- [52] BAAI. *BAAI/bge-reranker-v2-m3*. 2025. URL: <https://huggingface.co/BAAI/bge-reranker-v2-m3>.
- [53] Marcin Rutecki. *RAG: MMR Search in LangChain*. 2024. URL: <https://www.kaggle.com/code/marcinrutecki/rag-mmr-search-in-langchain>.
- [54] Boci Peng et al. *Graph Retrieval-Augmented Generation: A Survey*. 2024. arXiv: 2408.08921 [cs.AI]. URL: <https://arxiv.org/abs/2408.08921>.
- [55] proyectos ágiles. *Qué es SCRUM*. URL: <https://proyectosagiles.org/que-es-scrum/>.
- [56] Pablo Valenzuela Álvarez. *GeneRAG*. 2025. URL: <https://pvalenz23.atlassian.net/jira/software/projects/TFM/summary>.
- [57] BOE. *API de datos abiertos*. URL: <https://www.boe.es/datosabiertos/api/api.php>.
- [58] Pablo Valenzuela Álvarez. *GeneRAG*. 2025. URL: <https://github.com/Valenz23/GeneRAG>.
- [59] GitHub. *Understanding GitHub Actions*. URL: <https://docs.github.com/es/actions/get-started/understanding-github-actions>.
- [60] LangChain. *How to recursively split text by characters*. URL: https://python.langchain.com/docs/how_to/recursive_text_splitter/.
- [61] LangChain. *TF-IDF*. URL: https://python.langchain.com/docs/integrations/retrievers/tf_idf/.
- [62] LangChain. *BM25*. URL: <https://python.langchain.com/docs/integrations/retrievers/bm25/>.
- [63] LangChain. *GraphRAG*. URL: https://python.langchain.com/docs/integrations/retrievers/graph_rag/?vector-store=chroma.
- [64] Pablo Valenzuela Álvarez. *preguntas-mistral*. 2025. URL: https://docs.google.com/spreadsheets/d/1jYbFqGvP_v-M2BAeYZK_QieRD56pkHrLAePY6CVkgc0/edit?gid=0#gid=0.
- [65] Pablo Valenzuela Álvarez. *Resumen resultados*. 2025. URL: https://docs.google.com/spreadsheets/d/1PnUr-vEbaMKFC_LsKC1bwn3-bDg7liZA48kGq21mRKs/edit?usp=sharing.